

计算智能与启发式算法

硕士生课程讲义

第三部分：神经计算

第一节：神经计算与人工神经网络的历史

神经计算

神经计算是以人工神经网络为基础的一种智能计算方法。它是智能计算的重要基础和核心，也是智能计算乃至智能科学技术的一个重要研究领域。

神经计算的特点

- 可以充分逼近任意复杂的非线性关系
- 所有定量或定性的信息都等势分布贮存于网络内的各神经元，故有很强的鲁棒性和容错性
- 采用并行分布处理方法，使得快速进行大量运算成为可能
- 可学习和自适应不知道或不确定的系统
- 能够同时处理定量、定性知识。

人工神经网络的历史

萌芽期（20世纪40年代）

- 人工神经网络的研究最早可以追溯到人类开始研究自己的智能的时期。
- 1943年，心理学家McCulloch和数学家Pitts建立了著名的阈值加权和模型，简称为M-P模型。发表于数学生物物理学会刊《Bulletin of Mathematical Biophysics》
- 1949年，心理学家D. O. Hebb提出神经元之间突触联系是可变的假说——Hebb学习律。

人工神经网络的历史

第一高潮期（1950~1968）

- 以Marvin Minsky, Frank Rosenblatt, Bernard Widrow等为代表人物，代表作是单级感知器（Perceptron）。
- 可用电子线路模拟。
- 人们乐观地认为几乎已经找到了智能的关键。许多部门都开始大批地投入此项研究，希望尽快占领制高点。

人工神经网络的历史

反思期（1969~1982）

- M. L. Minsky和S. Papert, 《Perceptron》, MIT Press, 1969年
- “异或”运算不可表示
- 二十世纪70年代和80年代早期的研究成果
- 认识规律：认识——实践——再认识

人工神经网络的历史

第二高潮期（1983~1990）

1982年，J. Hopfield提出循环网络

- 建立ANN稳定性的判别依据
- 阐明了ANN与动力学的关系
- 用非线性动力学的方法来研究ANN的特性
- 指出信息被存放在网络中神经元的联接上

人工神经网络的历史

第二高潮期（1983~1990）

- 1984年， J. Hopfield设计研制了后来被人们称为Hopfield网的电路。较好地解决了著名的TSP问题，找到了最佳解的近似解，引起了较大的轰动。
- 1985年， 圣地牙哥加州大学的Hinton、Rumelhart等人所在的并行分布处理（PDP）小组的研究者在Hopfield网络中引入了随机机制，提出所谓的Boltzmann机。

人工神经网络的历史

第二高潮期（1983~1990）

- 1986年，并行分布处理小组的Rumelhart等研究者重新独立地提出多层网络的学习算法—BP算法，较好地解决了多层网络的学习问题。
- 1988年，RBF神经网络
- 90年代早期，vapnik提出SVM
- 国内首届神经网络大会1990年12月在北京举行

人工神经网络的历史

再认识与应用研究期（1991~）

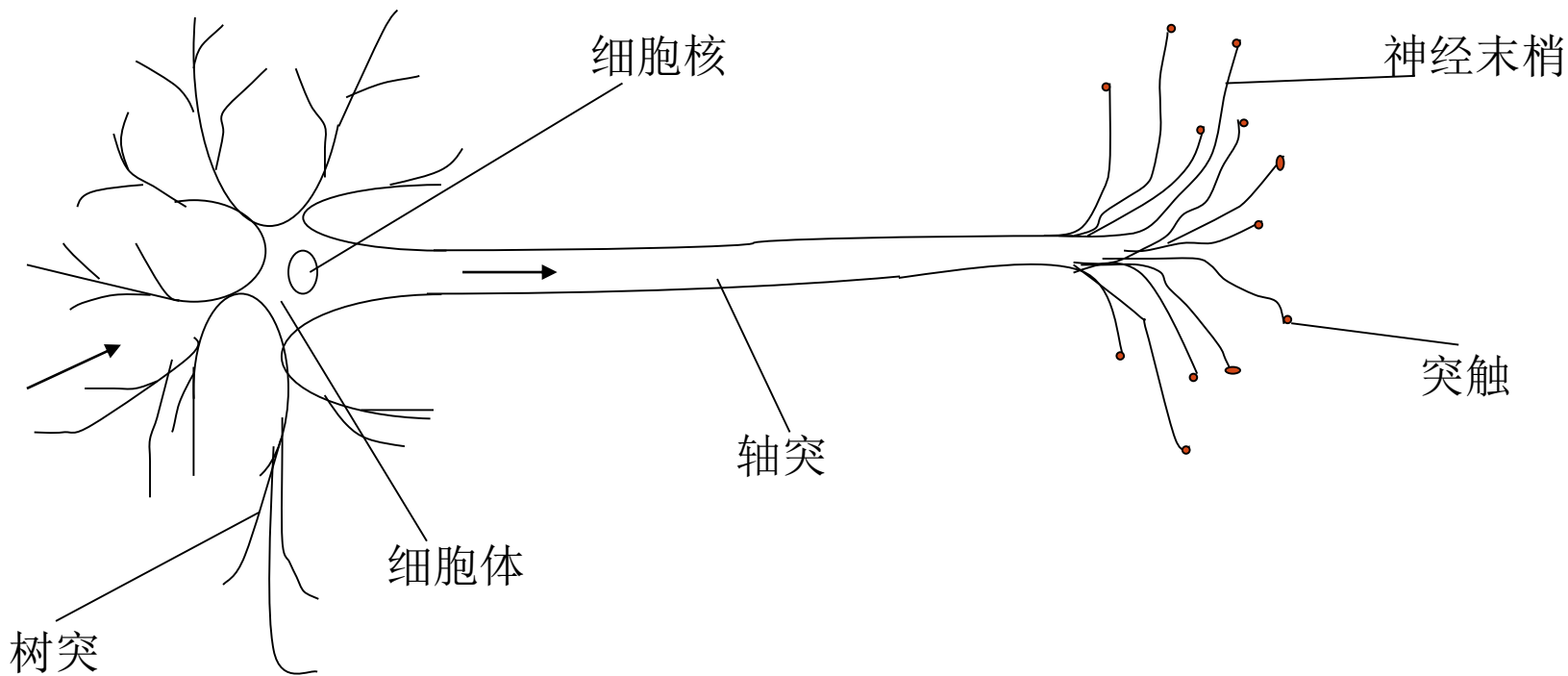
- 存在应用面还不够宽、结果不够精确等问题
- 改进现有模型、算法，以提高网络的训练速度和运行的准确度
- 算法的集成
- 希望在理论上寻找新的突破，建立新的专用/通用模型和算法
- 进一步对生物神经网络进行研究，不断地丰富对人脑的认识。

生物神经系统

生物神经系统是人工神经网络的基础。
人工神经网络是对人脑神经系统的简化、抽象和模拟，具有人脑功能的许多基本特征。

生物神经元的结构

它由细胞体(Soma)、轴突(Axon)和树突(Dendrite)三个主要部分组成



细胞体是神经元的主体，用于处理由树突接受的其它神经元传来的信号。**轴突**是由细胞体向外延伸出的所有纤维中最长的一条分枝，用来向外传递神经元产生的输出电信号。**树突**是指由细胞体向外延伸的除轴突以外的其它所有分支。树突的长度一般较短，但数量很多，它是神经元的输入端，用于接受从其它神经元的突触传来的信号。

生物神经元的功能

1. 神经元的抑制与兴奋

抑制状态是指神经元在没有产生冲动时的工作状态。兴奋状态是指神经元产生冲动时的工作状态。

通常情况下，神经元膜电位约为-70毫伏，膜内为负，膜外为正，处于抑制状态。当神经元受到外部刺激时，其膜电位随之发生变化，即膜内电位上升、膜外电位下降，当膜内外的电位差大于阈值电位（约+40毫伏）时，神经元产生冲动而进入兴奋状态。

2. 神经元内神经冲动的传导

神经冲动在神经元内的传导是一种电传导过程，神经冲动沿神经纤维传导的速度却在3.2--320km/s之间，且其传导速度与纤维的粗细、髓鞘的有无有一定关系。一般来说，有髓鞘的纤维的传导速度较快，而无髓鞘的纤维的传导速度较慢。

人脑神经系统的联结机制

1.人脑神经系统的联结规模

人脑大约由 10^{11} -- 10^{12} 个神经元所组成，其中每个神经元大约有 3×10^4 个突触。小脑中的每个神经元大约有 10^5 个突触，并且每个突触都可以与别的神经元的一个树突相连。

人脑神经系统就是由这些巨量的生物神经元经广泛并行互连所形成的一个高度并行性、非常复杂的神经网络系统。

2.人脑神经系统的分布功能

人脑神经系统的记忆和处理功能是有机的结合在一起的，每个神经元既具有存储功能，同时又具有处理能力。

从结构上看，人脑神经系统又是一种分布式系统。人们通过对脑损坏病人所做的神经生理学研究，没有发现大脑中的哪一部分可以决定其余所有各部分的活动，也没有发现在大脑中存在有用于驱动和管理整个智能处理过程的任何中央控制部分。

人类大脑的各个部分是协同工作、相互影响的。在大脑中，不仅知识的存储是分散的，而且其控制和决策也是分散的。

人工神经网络的工作原理

- 人脑由百亿条神经组成。每条神经平均连结到其它几千条神经。通过这种连结方式，神经可以收发不同数量的能量。
- 神经的一个非常重要的功能是它们对能量的接受并不是立即作出响应，而是将它们累加起来，当这个累加的总和达到某个**临界阈值**时，它们将它们自己的那部分能量发送给其它的神经。
- 大脑通过调节这些连结的数目和强度进行学习。尽管这是个生物行为的简化描述。但同样可以充分有力地被看作是神经网络的模型。

人工神经网络的工作原理

人的思维有逻辑性和直观性两种不同的基本方式。

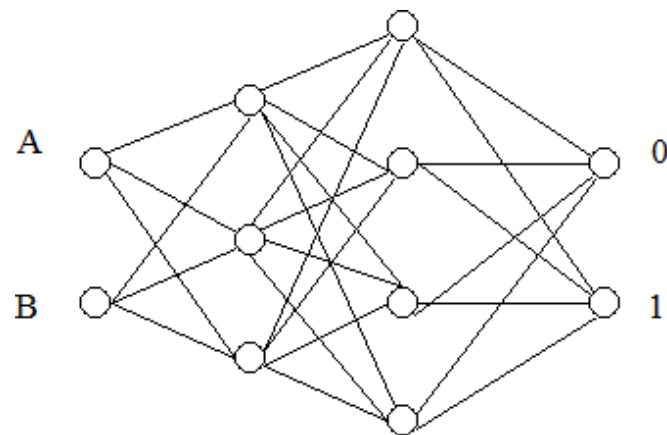
- 逻辑性的思维是指根据逻辑规则进行推理的过程；它先将信息化成概念，并用符号表示，然后，根据符号运算按串行模式进行逻辑推理；这一过程可以写成串行的指令，让计算机执行。
- 直观性的思维是将分布式存储的信息综合起来，结果是忽然间产生想法或解决问题的办法。这种思维方式的根本之点在于以下两点：
 - 信息是通过神经元上的兴奋模式分布储在网络上；
 - 信息处理是通过神经元之间同时相互作用的动态过程来完成的。

人工神经网络的工作原理

人工神经网络就是模拟人思维的第二种方式。这是一个非线性动力学系统，其特色在于信息的分布式存储和并行协同处理。虽然单个神经元的结构极其简单，功能有限，但大量神经元构成的网络系统所能实现的行为却是极其丰富多彩的。

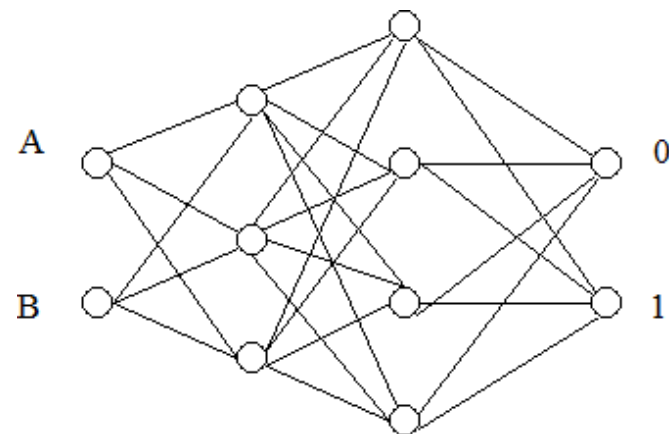
例子

- 人工神经网络首先要以一定的学习准则进行学习，然后才能工作。
- 现以人工神经网络对手写“A”、“B”两个字母的识别为例进行说明，规定当“A”输入网络时，应该输出“1”，而当输入为“B”时，输出为“0”。
- 网络学习的准则应该是：如果网络作出错误的的判决，则通过网络的学习，应使得网络减少下次犯同样错误的可能性。



例子

- 首先，给网络的各连接权值赋予 $(0, 1)$ 区间内的随机值，将“A”所对应的图象模式输入给网络，网络将输入模式加权求和、与门限比较、再进行非线性运算，得到网络的输出。在此情况下，网络输出为“1”和“0”的概率各为50%，也就是说是完全随机的。
- 如果输出为“1”(结果正确)，则使连接权值不变或增大，以便使网络再次遇到“A”模式输入时，仍然能作出正确的判断。



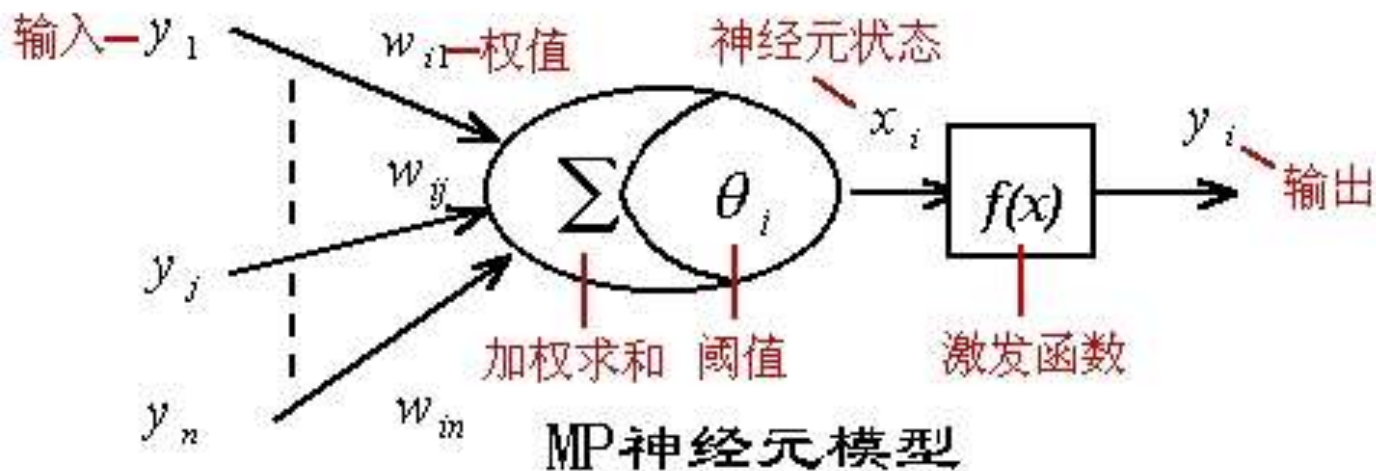
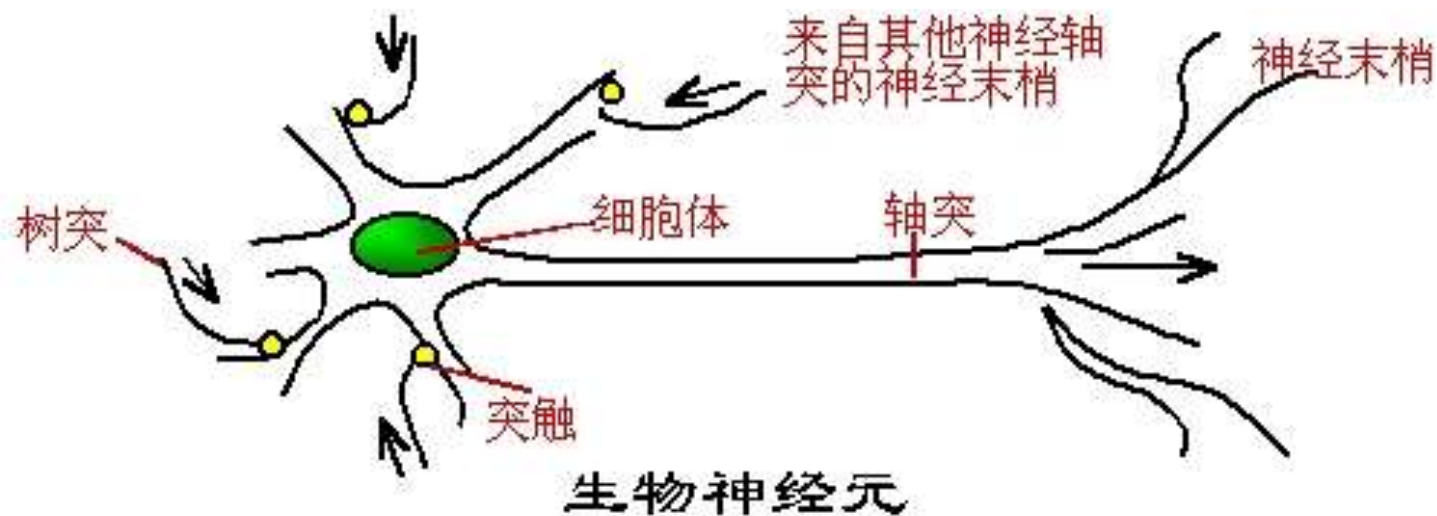
例子

- 如果输入为“B”，输出为“1”(即结果错误)，则把网络连接权值朝着减小的方向调整，其目的在于使网络下次再遇到“B”模式输入时，减小犯同样错误的可能性。
- 如此操作调整，当给网络轮番输入若干个手写字母“A”、“B”后，经过网络按以上学习方法进行若干次学习后，网络判断的正确率将大大提高。这说明网络对这两个模式的学习已经获得了成功，它已将这两个模式分布地记忆在网络各个连接权值上。当网络再次遇到其中任何一个模式时，能够作出迅速、准确的判断和识别。
- 一般说来，网络中所含的神经元个数越多，则它能记忆、识别的模式也就越多。

人工神经元模型

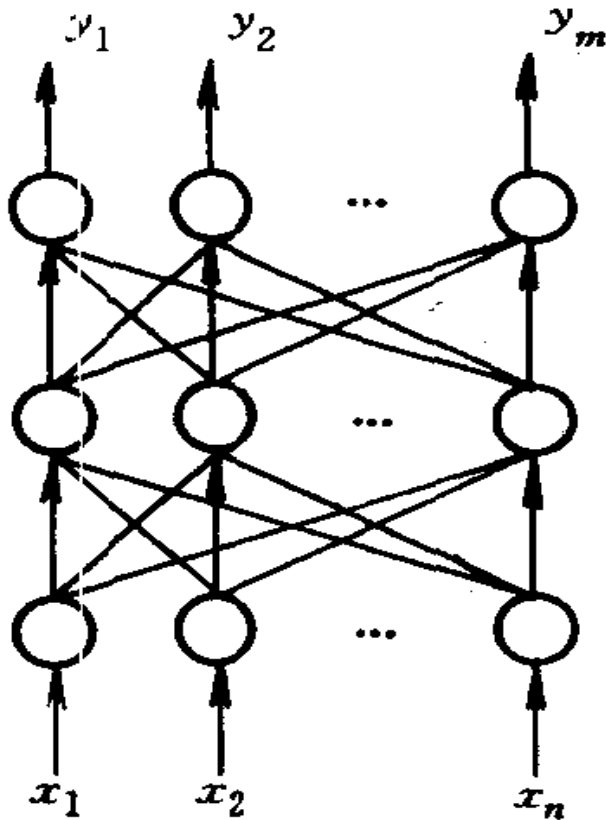
- 神经元是构成神经网络的最基本单元。
- 人工神经元模型应该具有生物神经元的基本特性。

神经元模型



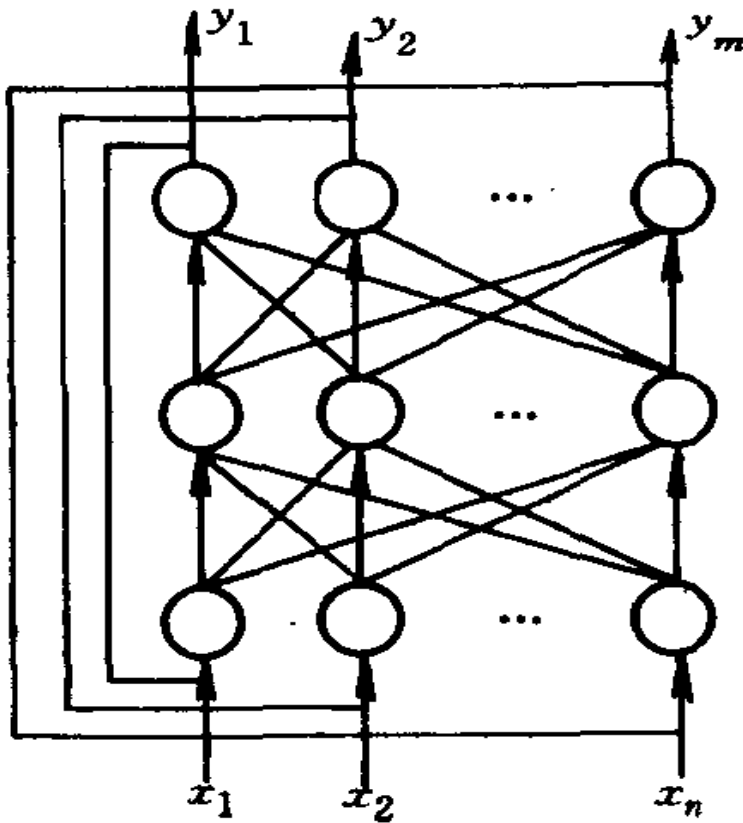
人工神经网络的类型

1. 前向网络



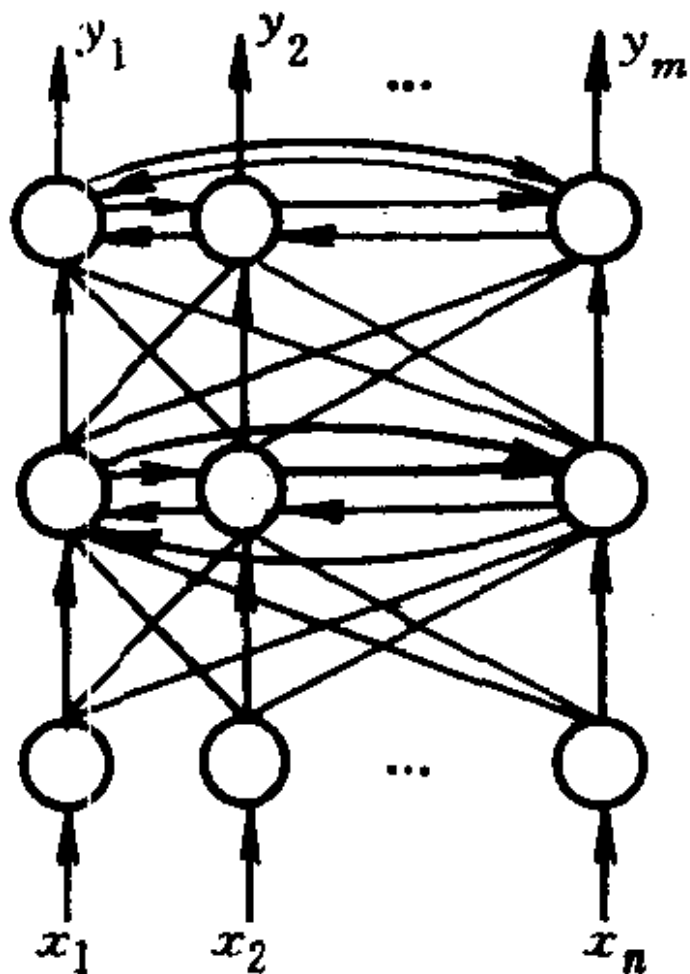
- 网络的结构如左图所示，网络中的神经元是分层排列的，每个神经元只与前一层的神经元相连接。最上一层为输出层，隐含层的层数可以是一层或多层。前向网络在神经网络中应用很广泛。
- 感知器就属于这种类型。

2. 从输出到输入有反馈的前向网络



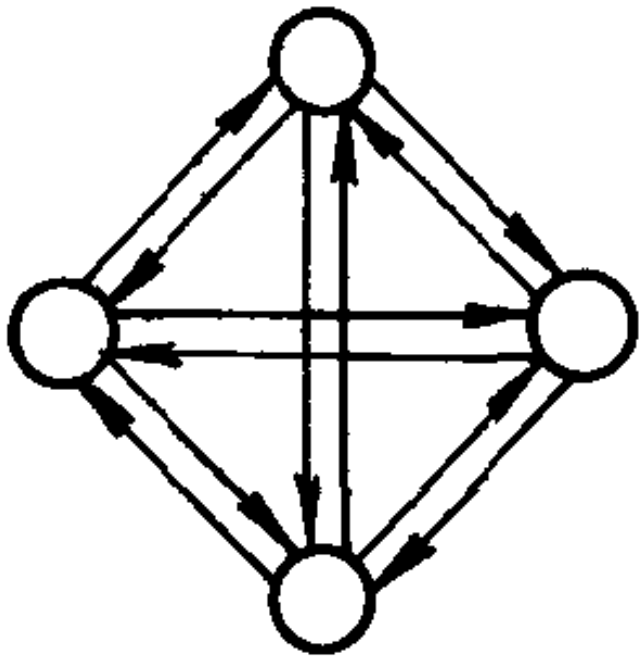
- 网络的结构如左图所示。网络的本身是前向型的，与前一种不同的是从输出到输入有反馈回路。
- Fukushima网络就属于这种类型。

3. 层内互连前向网络



- 可以实现同一层神经元之间横向抑制或兴奋的机制，从而限制层内能同时动作的神经数
- 可把层内神经元分为若干组，让每组作为一个整体来动作。
- 一些自组织竞争型神经网络就属于这种类型。

4. 互连网络

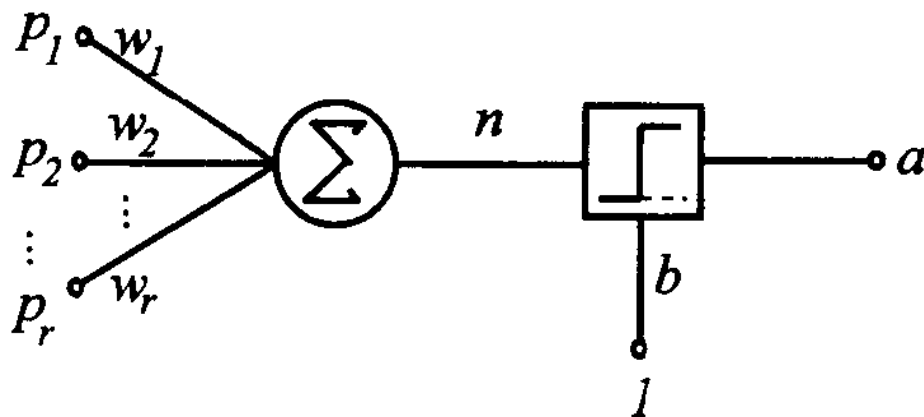


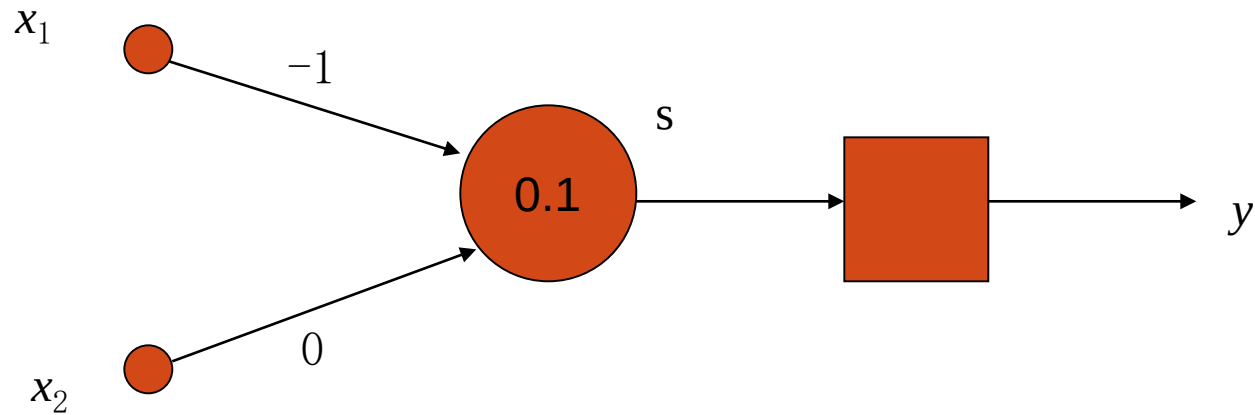
- 互连网络有局部互连和全互连两种。
- 全互连网络中的每个神经元都与其他神经元相连。
- 局部互连是指互连只是局部的，有些神经元之间没有连接关系。
- Hopfield网络和Boltzmann机属于互连网络的类型。

第二节：感知器

感知器 (Perceptron)

- **感知器**是由美国计算机科学家罗森布拉特 (F. Roseblatt) 于1957年提出的。感知器可谓是最早的人工神经网络。
- 单层感知器是一个具有一层神经元、采用阈值激活函数的前向网络。
- 通过对网络权值的训练，可以使感知器对一组输入矢量的响应达到元素为0或1的目标输出。

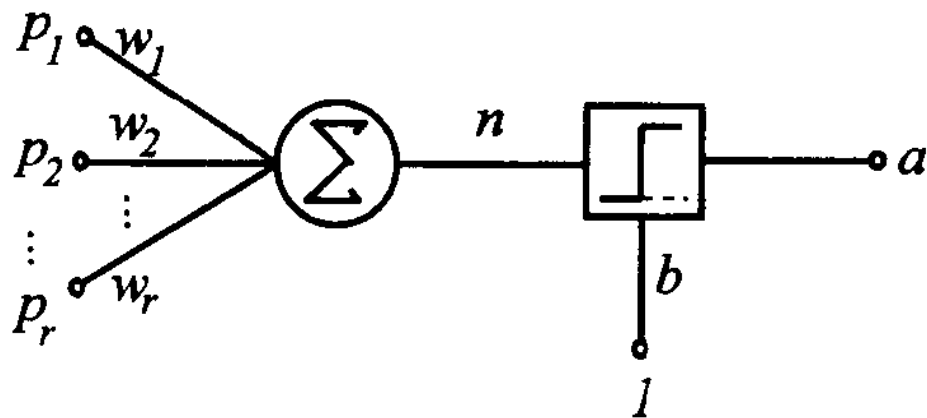




如果： $x_1=1$ ， $x_2=2$ ， 阈值为0.1

神经元计算： $s = 1 \times (-1) + 2 \times 0 - 0.1 = -1.1$

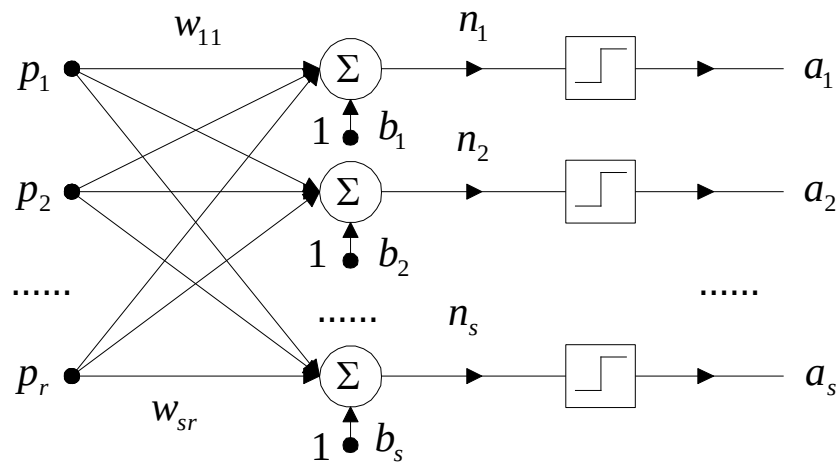
计算最后输出： $y = 0$ ($s = -1.1 < 0$)



- 每一个输入分量 p_j ($j=1, 2, \dots, r$) 通过一个权值分量 w_j ，进行加权求和，并作为阈值函数的输入。偏置 b 的加入使得网络多了一个可调参数，为使网络输出达到期望的目标矢量提供了方便。
- 感知器特别适合解决简单的模式分类问题。
- F. Roseblatt 已经证明，如果两类模式是线性可分的（指存在一个超平面将它们分开），则算法一定收敛。

感知器的网络结构

- 感知器的网络是由单层的 s 个感知神经元，通过一组权值 $\{\omega_{ij}\}(i=1, 2\dots s; j=1, 2\dots r)$ 与 r 个输入相连组成。对于具有输入矢量 $\mathbf{P}_r$ 和目标矢量 $\mathbf{T}_s$ 的感知器网络的简化结构，如图所示。



感知器简化结构图

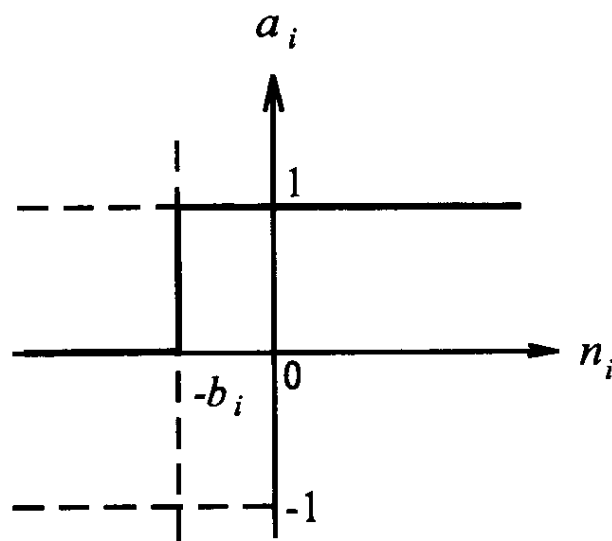
感知器的网络结构

根据网络结构，可以写出第*i*个输出神经元(*i*=1, 2, ..., *s*)的加权输入和*n_i*及其输出*a_i*为：

$$n_i = \sum_{j=1}^r w_{ij} p_j \qquad a_i = f(n_i + b_i)$$

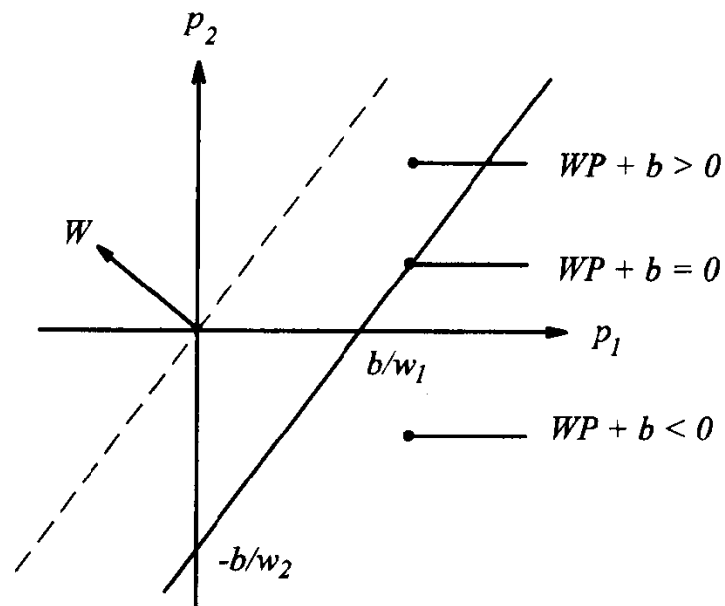
感知器的输出值是通过测试加权输入和值落在阈值函数的左右来进行分类的，即有：

$$a_i = \begin{cases} 1 & n_i + b_i \geq 0 \\ 0 & n_i + b_i < 0 \end{cases}$$



感知器的图形解释

为了简单起见，以下输出为一个节点的网络情况来进行作图解释。



- 以输入矢量 $\mathbf{r}=2$ 为例，对于选定的权值 w_1 、 w_2 和 b ，可以在以 p_1 和 p_2 分别作为横、纵坐标的输入平面内画出 $\mathbf{W} \times \mathbf{P} + b = w_1 p_1 + w_2 p_2 + b = 0$ 的轨迹，它是一条直线
- 直线上及线以上部分的所有 p_1 、 p_2 值均使 $w_1 p_1 + w_2 p_2 + b \geq 0$ ，这些点通过由 w_1 、 w_2 和 b 构成的感知器则使其输出为1；
- 直线以下部分的点则使感知器的输出为0。

感知器学习规则

对于输入矢量 P ，输出矢量 A ，目标矢量为 T 的感知器网络，感知器的学习规则是根据以下输出矢量可能出现的几种情况来进行参数调整的。

- ① 如果第 i 个神经元的输出是正确的，即有： $a_i = t_i$ ，那么与第 i 个神经元联接的权值 w_{ij} 和偏置值 b_i 保持不变；
- ② 如果第 i 个神经元的输出是0，但期望输出为1，即有 $a_i = 0$ ，而 $t_i = 1$ ，此时权值修正算法为：
新的权值 $w_{ij} = \text{旧的权值} w_{ij} + \text{输入矢量} p_j$
新的偏置 $b_i = \text{旧偏置} b_i + 1$ ；
- ③ 如果第 i 个神经元的输出为1，但期望输出为0，即有 $a_i = 1$ ，而 $t_i = 0$ ，此时权值修正算法为：
新的权值 $w_{ij} = \text{旧的权值} w_{ij} - \text{输入矢量} p_j$
新的偏置 $b_i = \text{旧偏置} b_i - 1$ 。

感知器学习规则

- 感知器学习规则的实质为：权值的变化量等于正负输入矢量。

用矢量矩阵来表示为：

$$\begin{aligned} \mathbf{W} &= \mathbf{W} + \mathbf{E}\mathbf{P}^T \\ \mathbf{B} &= \mathbf{B} + \mathbf{E} \end{aligned}$$

此处， $\mathbf{E}$ 为误差矢量，有 $\mathbf{E} = \mathbf{T} - \mathbf{A}$ 。

感知器的学习规则属于梯度下降法，该法则已被证明：如果解存在，则算法在有限次的循环迭代后可以收敛到正确的目标矢量。

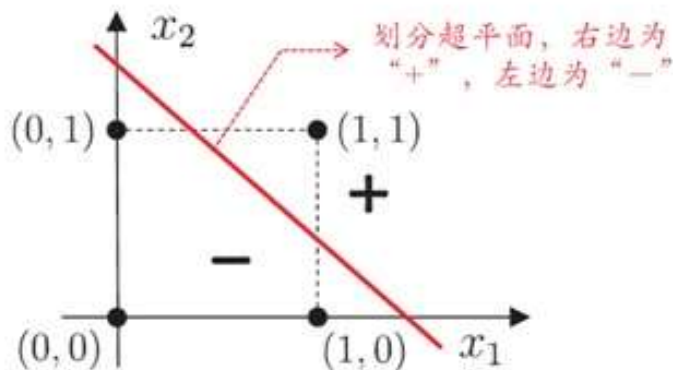
感知器设计训练的步骤

- 1) 对于所要解决的问题，确定输入矢量 P ，目标矢量 T ，并由此确定各矢量的维数以及确定网络结构大小的神经元数目： r ， s 和 q ；
- 2) 参数初始化：
 - a) 赋给权矢量 w 在 $(-1, 1)$ 的随机非零初始值；
 - b) 给出最大训练循环次数 max_epoch ；
- 3) 网络表达式：根据输入矢量 P 以及最新权矢量 W ，计算网络输出矢量 A ；
- 4) 检查：检查输出矢量 A 与目标矢量 T 是否相同，如果是，或已达最大循环次数训练结束，否则转入5)；
- 5) 学习：感知器按学习规则调整权矢量，并返回3)。

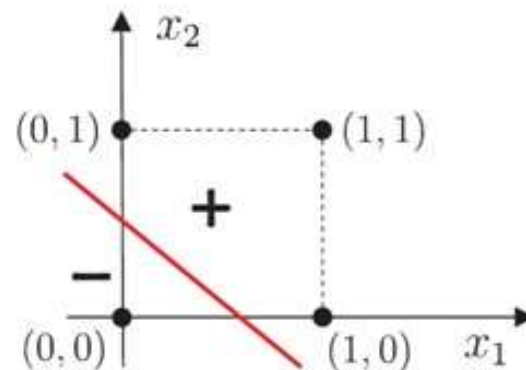
注意：

- 若网络训练成功，那么训练后的网络在网络权值的作用下，对于被训练的每一组输入矢量都能够产生一组对应的期望输出；
- 若在设置的最大训练次数内，网络未能够完成在给定的输入矢量 P 的作用下，使 $A=T$ 的目标，则可以通过改用新的初始权值与偏置，并采用更长训练次数进行训练。

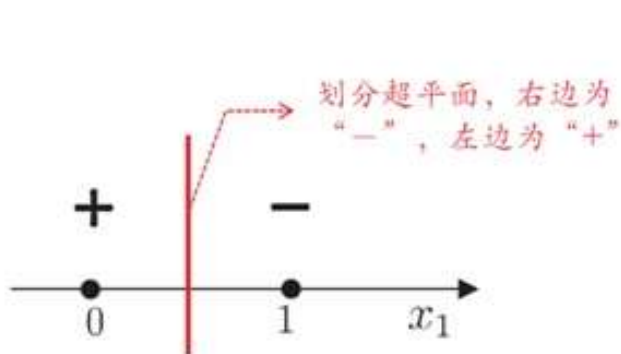
感知机求解与、或、非、异或问题



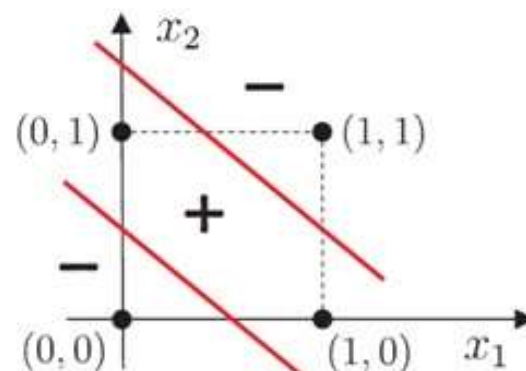
(a) “与”问题 ($x_1 \wedge x_2$)



(b) “或”问题 ($x_1 \vee x_2$)



(c) “非”问题 ($\neg x_1$)



(d) “异或”问题 ($x_1 \oplus x_2$)

感知机学习能力

- 感知器的输出只能取0或1。
- 若两类模式线性可分, 则感知机的学习过程一定会收敛; 否则感知机的学习过程将会发生震荡。
- 单层感知机的学习能力非常有限, 只能解决线性可分问题
- 事实上, 与、或、非问题是线性可分的, 因此感知机学习过程能够求得适当的权值向量。而异或问题不是线性可分的, 感知机学习不能求得合适解
- 对于非线性可分问题, 如何求解? → 多层感知机

多层感知机

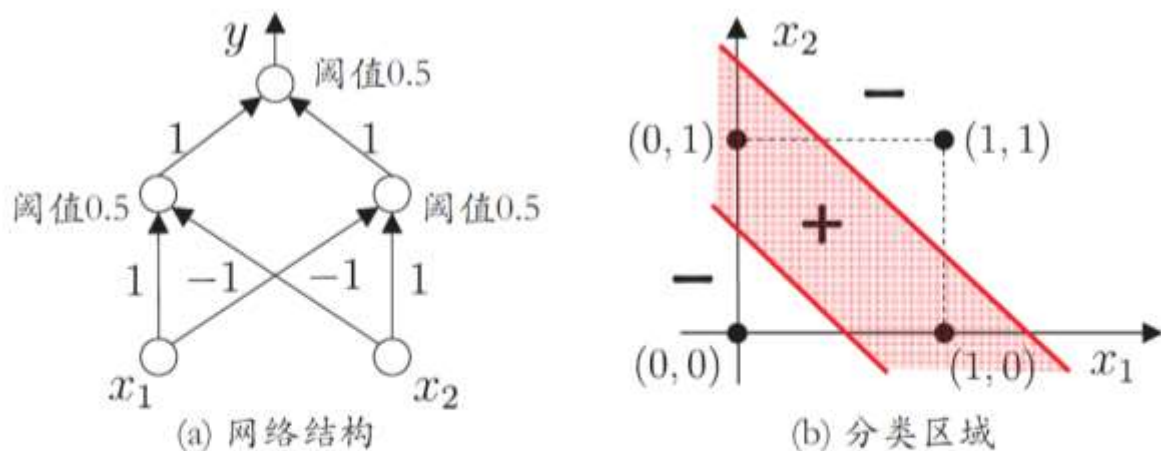


图 5.5 能解决异或问题的两层感知机

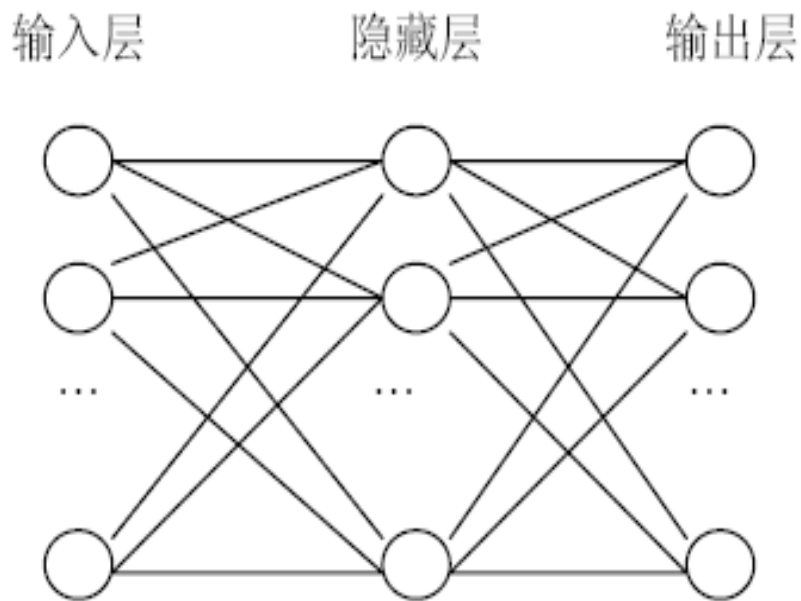
- 输出层与输入层之间的一层神经元, 被称之为**隐层或隐含层**, 隐含层和输出层神经元都是具有激活函数的功能神经元

第三节：反向传播网络

反向传播网络

- 感知器神经网络为单层网络结构，其传递函数都十分简单，感知器为单边符号函数。
- 推广到多层网络，并且将传递函数变为非线性可微分函数，这就是反向传播网络 (Back Propagation Network, 简称BP网络) 是对非线性可微分函数进行权值训练的多层网络。
- 在人工神经网络的实际应用中，80%~90%的人工神经网络模型是采用BP网络或它的变化形式，是前向网络的核心部分。

BP网络模型与结构



- 神经网络通常由输入层、输出层和隐藏层组成，输入层的神经元个数等于样本的特征数，
- 输出层就是分类判决层，它的神经元个数等于样本的类别数。
- 特点：对噪声数据具有较高的承受能力，但训练需要花费较长的时间。

BP算法

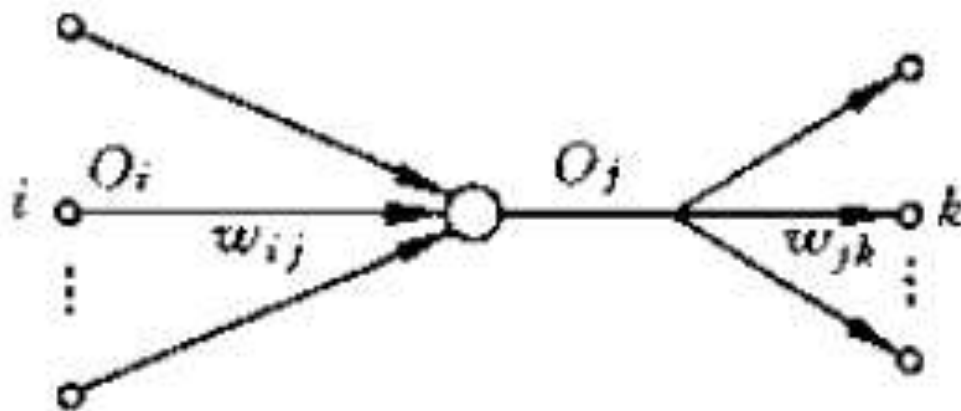
- BP算法通过迭代地处理一组训练样本，将每个样本的网络预测与实际知道的类标号比较，进行学习。
- 对于每个训练样本，修改连接权值，使得网络预测与实际类之间的均方误差最小。
- 这种修改“后向”地进行，即由输出层，经由每个隐藏层，到第一个隐藏层。

BP算法修改权值方法的推导

- 研究处于某一层的第 j 个计算单元，脚标 i 代表其前层第 i 个单元，脚标 k 代表后层第 k 个单元， O_j 代表本层输出， w_{ij} 是前层到本层的权值，如图所示。
- 当输入某个样本时，从前到后对每层各单元作如下计算（正向计算）

$$net_j = \sum_i w_{ij} O_i$$

$$O_j = f(net_j)$$



BP算法修改权值方法的推导

对于输出层而言， $\hat{y}_j = O_j$ 是实际输出值， y_j 是理想输出值，此样本下的误差为：

$$E = \frac{1}{2} \sum_j (y_j - \hat{y}_j)^2$$

为使式子简化，定义局部梯度

$$\delta_j = \frac{\partial E}{\partial net_j}$$

考虑权值 w_{ij} 对误差的影响，可得

$$\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial net_j} \frac{\partial net_j}{\partial w_{ij}} = \delta_j O_i$$

权值修正应使误差最快地减少，修正量为

$$\Delta w_{ij} = -\eta \delta_j O_i$$

$$w_{ij}(t+1) = w_{ij}(t) + \Delta w_{ij}(t)$$

BP算法修改权值方法的推导

如果节点 j 是输出单元, 则 $O_j = \hat{y}_j$

$$\delta_j = \frac{\partial E}{\partial \hat{y}_j} \frac{\partial \hat{y}_j}{\partial net_j} = -(y_j - \hat{y}_j) f'(net_j)$$

如果节点 j 不是输出单元, 由图可知, O_j 对后层的全部节点都有影响。因此,

$$\delta_j = \frac{\partial E}{\partial net_j} = \sum_k \frac{\partial E}{\partial net_k} \cdot \frac{\partial net_k}{\partial O_j} \cdot \frac{\partial O_j}{\partial net_j} = \sum_k \delta_k w_{jk} f'(net_j)$$

对于Sigmoid函数

$$y = f(x) = \frac{1}{1 + e^{-x}}$$

有
$$f'(x) = \frac{e^{-x}}{(1 + e^{-x})^2} = y(1 - y)$$

或者当
$$y = f(x) = \tanh x$$

时有
$$f'(x) = 1 - \tanh^2 x = 1 - y^2$$

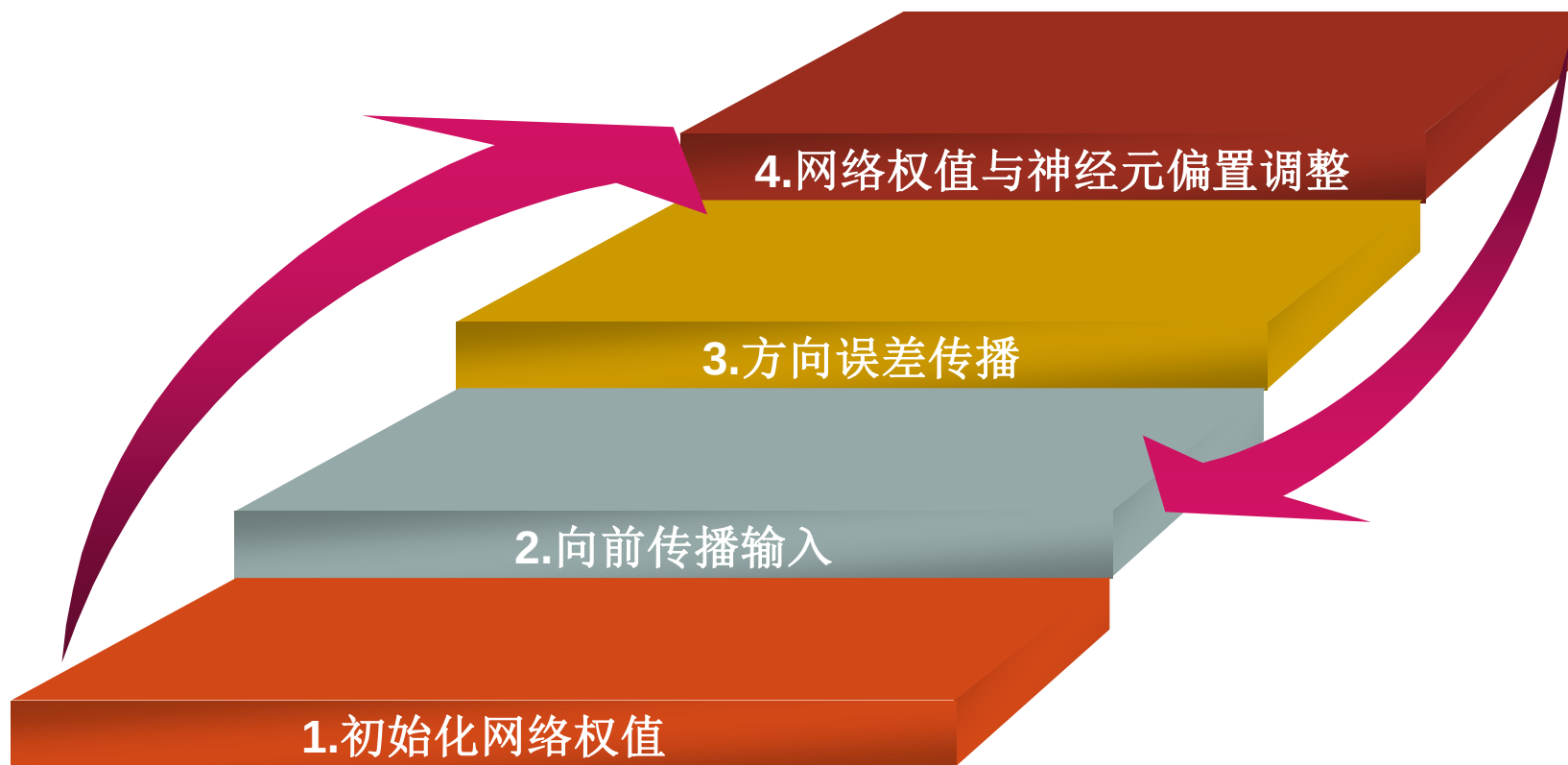
BP算法修改权值方法的推导

- 在实际计算时，为了加快收敛速度，往往在权值修正量中加上前一次的权值修正量，一般称之为惯性项，即

$$\Delta w_{ij}(t) = -\eta \delta_j O_i + \alpha \Delta w_{ij}(t-1)$$

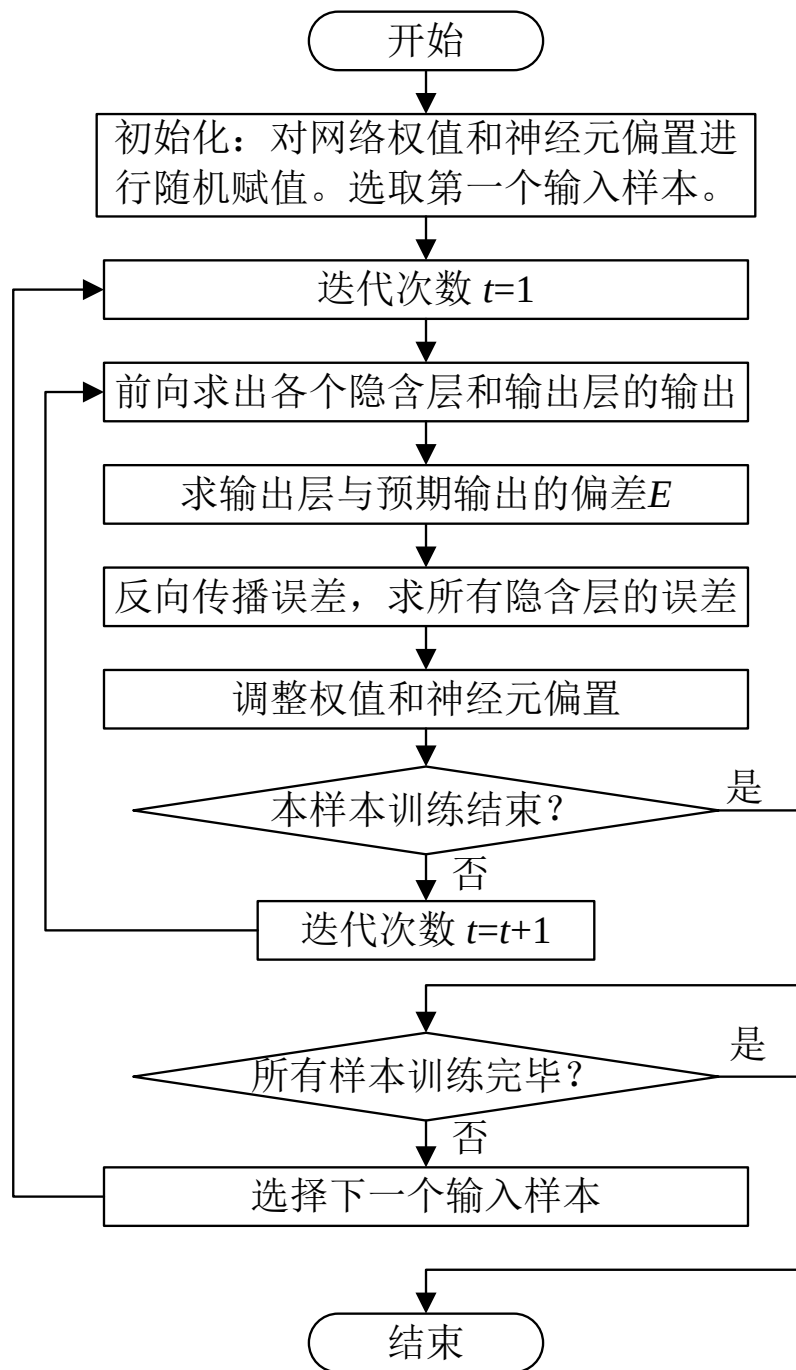
- η 为学习率

算法流程



BP算法

- 反向传播算法解决了隐层权值修正问题，但它是用梯度法求非线性函数极值，因而有可能陷入局部极小点，不能保证收敛到全局极小点。
- 终止条件：
 - 更新权值较小
 - 正确分类的样本百分比(系统总误差小于 ε)
 - 超过预先指定的训练周期



//功能：BP神经网络训练过程的伪代码

procedure BPNN

Initialization, include the ω_{ij} and θ_i

for each sample X

while not stop

//forwards propagation of the input

for each unit j in the
hidden and output layer

//calculate the output O_j ;

$$O_j = \frac{1}{1 + e^{-S_j}} = \frac{1}{1 + e^{-(\sum_i \omega_{ij} O_i + \theta_j)}}$$

for each unit j in the output layer

//calculate the error E_j ;

$$E_j = O_j(1 - O_j)(T_j - O_j);$$

//back propagation of the error

for each unit j in the hidden layer

//calculate the error E_j ;

$$E_j = O_j(1 - O_j) \sum_k \omega_{jk} E_k$$

//adjust the network parameters

for each network weight ω_{ij}

$$\omega_{ij} = \omega_{ij} + (l) E_j O_j;$$

for each biased θ_j

$$\theta_j = \theta_j + (l) E_j;$$

end of while not stop

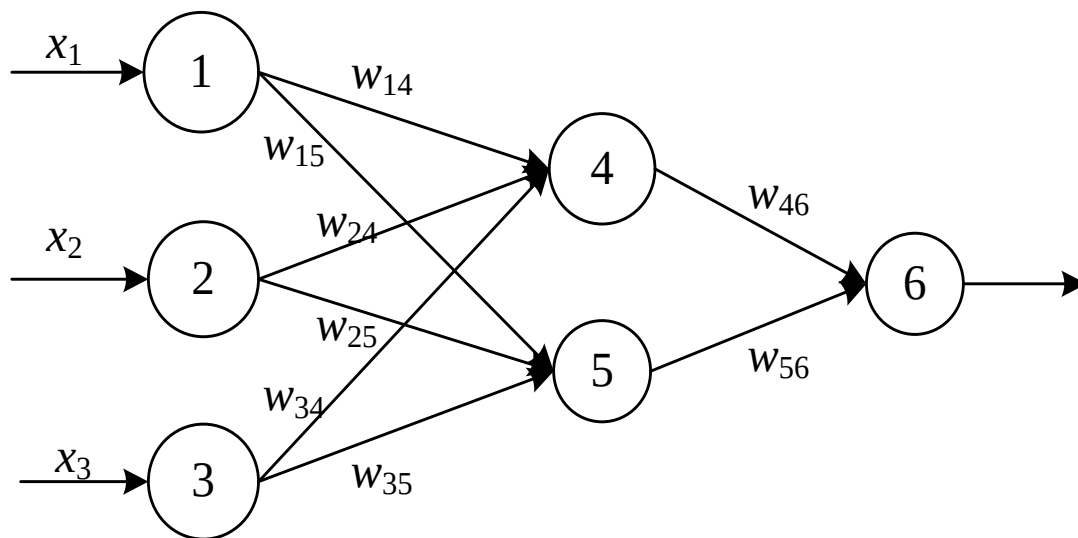
end of for each sample X

end of procedure

应用举例

- 已知一个前馈型神经网络例子如下图所示。设学习率 $\eta=0.9$ ，当前的训练样本为 $x=\{1,0,1\}$ ，而且预期分类标号为1。下表给出了当前该网络的各个连接权值和神经元偏置。求该网络在当前训练样本下的训练过程。

w_{14}	w_{15}	w_{24}	w_{25}	w_{34}	w_{35}	w_{46}	w_{56}	θ_4	θ_5	θ_6
0.2	-0.3	0.4	0.1	-0.5	0.2	-0.3	-0.2	-0.4	0.2	0.1



分类系统训练过程

w_{14}	w_{15}	w_{24}	w_{25}	w_{34}	w_{35}	w_{46}	w_{56}	θ_4	θ_5	θ_6
0.2	-0.3	0.4	0.1	-0.5	0.2	-0.3	-0.2	-0.4	0.2	0.1

第一步：前向计算每个神经元的输入加权和 S_j 和输出 O_j

单元 j	加权和 S_j	输出 O_j
4	$w_{14} \times x_1 + w_{24} \times x_2 + w_{34} \times x_3 + \theta_4 = 0.2 \times 1 + 0.4 \times 0 + (-0.5) \times 1 + (-0.4) = -0.7$	$1/(1+e^{-(0.7)})=0.332$
5	$w_{15} \times x_1 + w_{25} \times x_2 + w_{35} \times x_3 + \theta_5 = -0.3 \times 1 + 0.1 \times 0 + 0.2 \times 1 + 0.2 = 0.1$	$1/(1+e^{-0.1})=0.525$
6	$w_{46} \times O_4 + w_{56} \times O_5 + \theta_6 = -0.3 \times 0.332 + (-0.2) \times 0.525 + 0.1 = -0.105$	$1/(1+e^{-(0.105)})=0.474$

第二步：后向传播误差 E_j

单元 j	误差 E_j (T_j 是期望输出, 本例题为1)
6	$O_j(1-O_j)(T_j-O_j) = 0.474 \times (1-0.474) \times (1-0.474) = 0.1311$
5	$O_j(1-O_j) \sum_k \omega_{jk} E_k = 0.525 \times (1-0.525) \times (0.1311) \times (-0.2) = -0.0065$
4	$O_j(1-O_j) \sum_k \omega_{jk} E_k = 0.332 \times (1-0.332) \times (0.1311) \times (-0.3) = -0.0087$

分类系统训练过程

第三步：更新权值或者偏置

权值或偏置	新值
w_{46}	$w_{ij} + \Delta w_{ij} = w_{ij} + (l)O_i E_j = -0.3 + 0.9 \times 0.332 \times 0.1311 = -0.261$
w_{56}	$w_{ij} + \Delta w_{ij} = w_{ij} + (l)O_i E_j = -0.2 + 0.9 \times 0.525 \times 0.1311 = -0.138$
w_{14}	$w_{ij} + \Delta w_{ij} = w_{ij} + (l)O_i E_j = 0.2 + 0.9 \times 1 \times (-0.0087) = 0.192$
w_{15}	$w_{ij} + \Delta w_{ij} = w_{ij} + (l)O_i E_j = -0.3 + 0.9 \times 1 \times (-0.0065) = -0.306$
w_{24}	$w_{ij} + \Delta w_{ij} = w_{ij} + (l)O_i E_j = 0.4 + 0.9 \times 0 \times (-0.0087) = 0.4$
w_{25}	$w_{ij} + \Delta w_{ij} = w_{ij} + (l)O_i E_j = 0.1 + 0.9 \times 0 \times (-0.0065) = 0.1$
w_{34}	$w_{ij} + \Delta w_{ij} = w_{ij} + (l)O_i E_j = -0.5 + 0.9 \times 1 \times (-0.0087) = -0.508$
w_{35}	$w_{ij} + \Delta w_{ij} = w_{ij} + (l)O_i E_j = 0.2 + 0.9 \times 1 \times (-0.0065) = 0.194$
θ_6	$\theta_j + \Delta \theta_j = \theta_j + (l)E_j = 0.1 + 0.9 \times 0.1311 = 0.218$
θ_5	$\theta_j + \Delta \theta_j = \theta_j + (l)E_j = 0.2 + 0.9 \times (-0.0065) = 0.194$
θ_4	$\theta_j + \Delta \theta_j = \theta_j + (l)E_j = -0.4 + 0.9 \times (-0.0087) = -0.408$

第四节：径向基网络

径向基网络的历史

- 1985年，Powell提出了多变量插值的径向基函数（Radial-Basis Function, RBF）方法。1988年，Broomhead和Lowe首先将RBF应用于神经网络设计，构成了径向基函数神经网络，即RBF神经网络。
- 结构上看，RBF神经网络属于多层前向神经网络。它是一种三层前向网络：
 - 输入层由信号源节点组成；
 - 第二层为隐含层，隐单元的个数由所描述的问题而定，隐单元的变换函数是对中心点径向对称且衰减的非负非线性函数；
 - 第三层为输出层，它对输入模式的作用做出响应。

径向基网络的思想

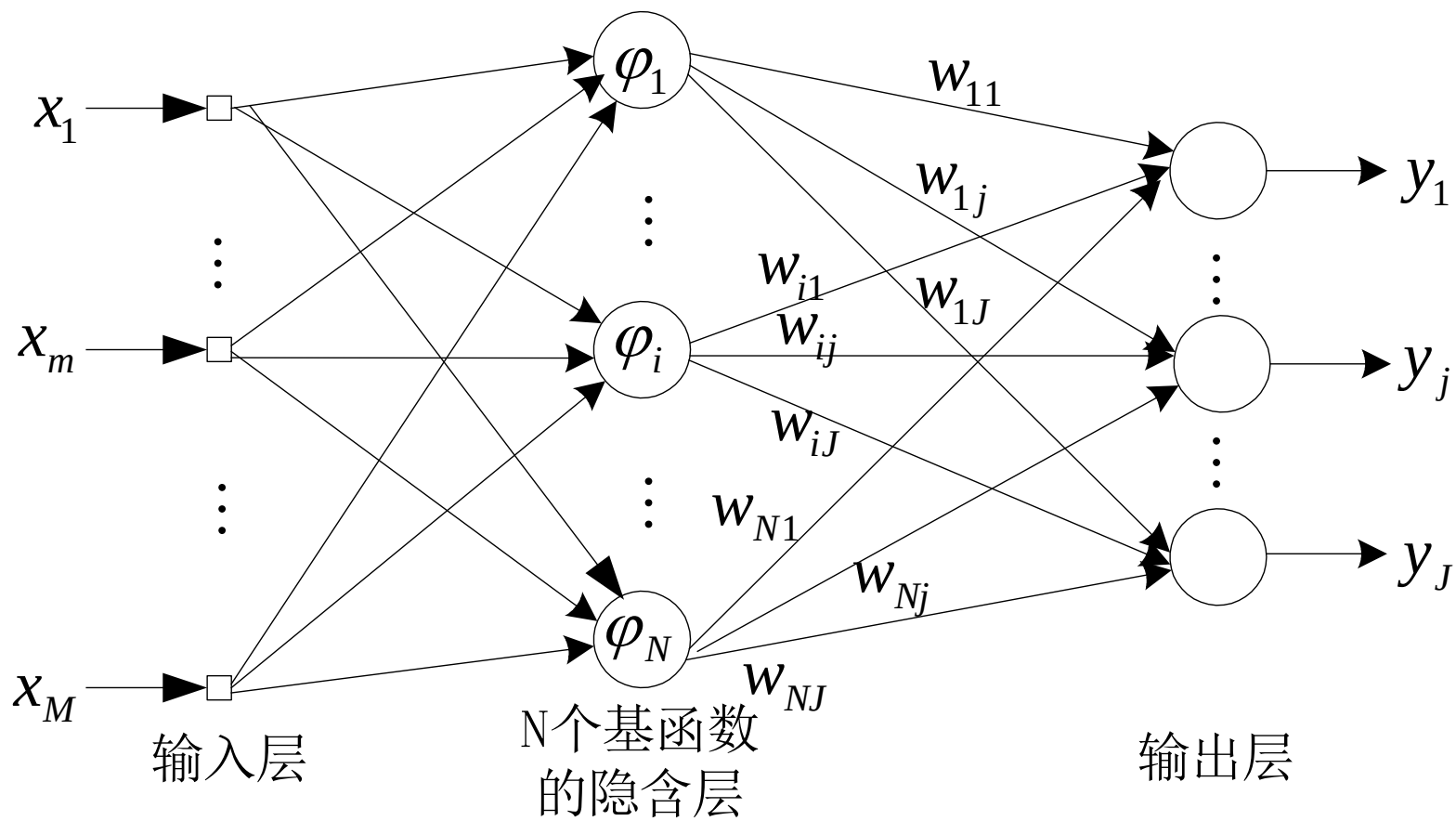
- **RBF神经网络的基本思想是：**用径向基函数（**RBF**）作为隐单元的“基”，构成隐含层空间，隐含层对输入矢量进行变换，将低维的模式输入数据变换到高维空间内，通过对隐单元输出的加权求和得到输出。
- **RBF神经网络主要用于解决模式分类和函数逼近等问题。**在数学上，**RBF神经网络结构的合理性可由 Cover 定理**得到保证。即对于一个模式问题，在高维数据空间中可能解决在低维空间中不易解决的分类问题。它以径向基函数作为隐节点的激活函数，具有收敛速度快、逼近精度高、网络规模小等特点。

径向基网络的分类

RBF网络是单隐层的前向网络。根据隐单元的个数，RBF网络有两种模型：

- **正规化网络（Regularization Network）**
- **广义网络（Generalized Network）。**

径向基网络



(1) 正规化网络

正规化网络的隐单元就是训练样本，所以正规化网络其隐单元的个数与训练样本的个数相同。

- 当网络输入训练样本 X_k 时，网络第 j 个输出神经元的实际输出为：

$$y_{kj}(X_k) = \sum_{i=1}^N w_{ij} \varphi(X_k, X_i)$$

- 一般“基函数”选为格林函数记为：

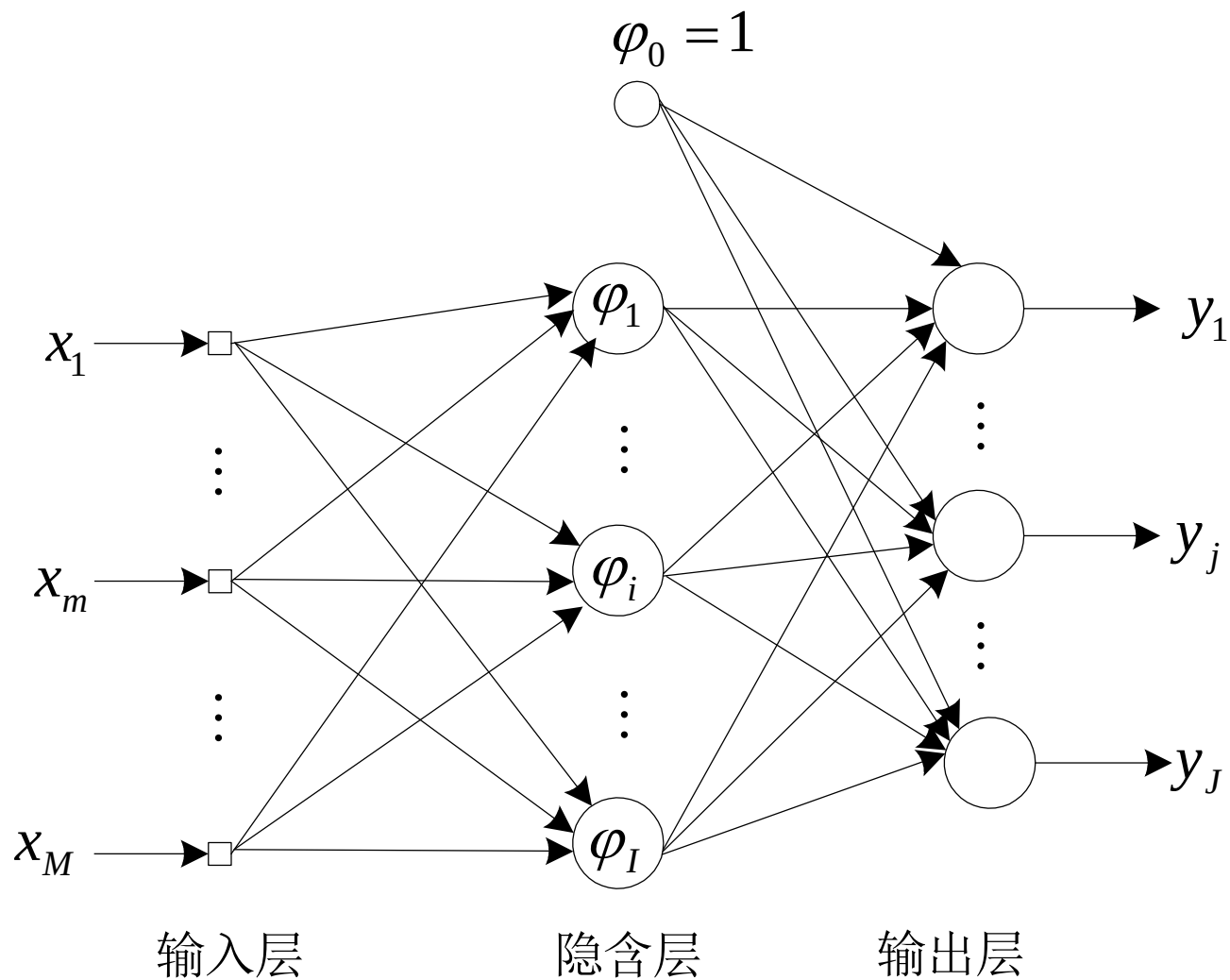
$$\varphi(X_k, X_i) = G(X_k, X_i)$$

- 当格林函数 $G(X_k, X_i)$ 为高斯函数时：

$$G(X_k, X_i) = G(\|X_k - X_i\|) = \exp\left(-\frac{1}{2\sigma_i^2} \sum_{m=1}^M (x_{km} - x_{im})^2\right)$$

(2) 广义网络

- 正规化网络的训练样本 X_i 与“基函数” $\phi(X_k, X_i)$ 是一一对应的，当 N 很大时，网络的实现复杂，且在求解网络的权值时容易产生病态问题（*ill conditioning*）。
- 解决这一问题的方法是减少隐层神经元的个数。



隐层神经元个数小于训练样本数

当网络输入训练样本 X_k 时，网络第 j 个输出神经元的实际输出为：

$$y_{kj}(X_k) = w_{0j} + \sum_{i=1}^I w_{ij} \varphi(X_k, \mathbf{t}_i)$$

当“基函数”为高斯函数时：

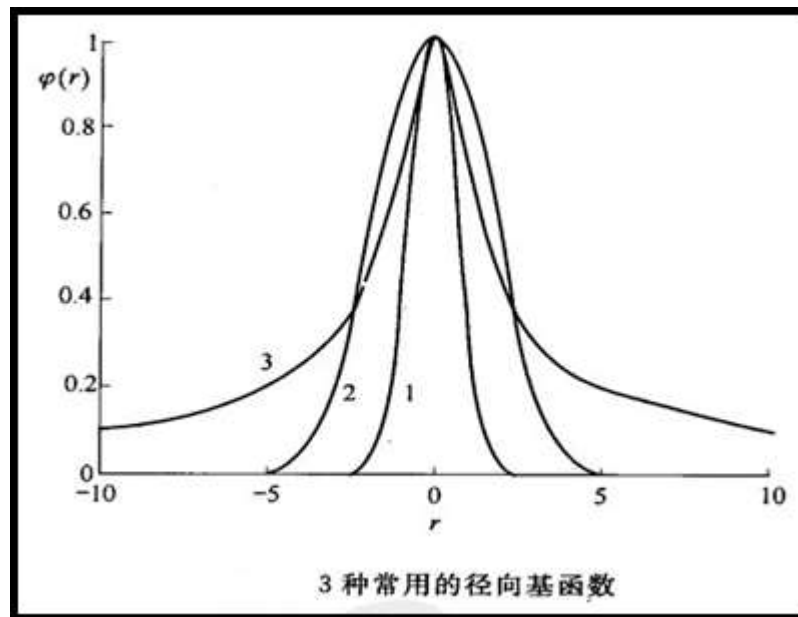
$$\begin{aligned} \varphi(X_k, \mathbf{t}_i) &= G(\|X_k - \mathbf{t}_i\|) \\ &= \exp\left(-\frac{1}{2\sigma_i^2} \sum_{m=1}^M (x_{km} - t_{im})^2\right) \end{aligned}$$

1. Gauss (高斯) 函数: $\varphi(r) = \exp\left(-\frac{r^2}{2\sigma^2}\right)$

2. 反演S型函数: $\varphi(r) = \frac{1}{1 + \exp\left(\frac{r^2}{\sigma^2}\right)}$

3. 拟多二次函数: $\varphi(r) = \frac{1}{(r^2 + \sigma^2)^{1/2}}$

σ 称为基函数的扩展常数或宽度, σ 越小, 径向基函数的宽度越小, 基函数就越有选择性。



RBF网络的学习算法

- RBF网络要学习的参数有三个：中心、方差和权值。
- 根据径向基函数中心选取方法的不同，RBF网络有多种学习方法，其中最常用的四种学习方法有：随机选取中心法，自组织选取中心法，有监督选取中心法和正交最小二乘法。
- 自组织选取中心法由两个阶段构成：
 - ① 自组织学习阶段，即学习隐层基函数的中心与方差的阶段。
 - ② 有监督学习阶段，即学习输出层权值的阶段。

(1) 学习中心

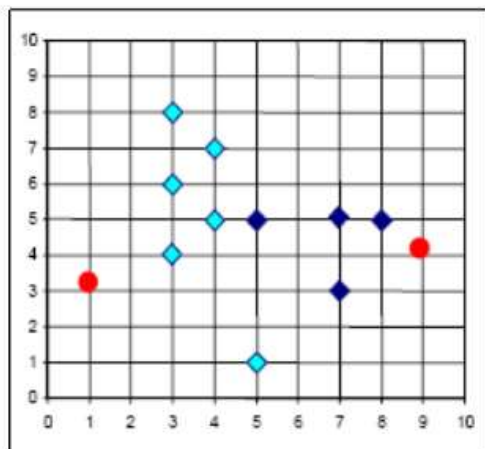
- 自组织学习过程要用到聚类算法，常用的聚类算法是K-均值聚类算法。假设聚类中心有 I 个（ I 的值由先验知识决定），设 $t_i(n)$ （ $i=1,2,\dots,I$ ）是第 n 次迭代时基函数的中心，K-均值聚类算法具体步骤如下：
- 第一步：初始化聚类中心，即根据经验从训练样本集中随机选取 I 个不同的样本作为初始中心 $t_i(0)$ （ $i=1, 2, \dots, I$ ）。

- 第二步：随机输入训练样本 X_k 。
- 第三步：寻找训练样本 X_k 离哪个中心最近，即找到 $i(X_k)$ 使其满足 $i(X_k) = \arg \min \| X_k - \mathbf{t}_i(n) \|^i$
- 第四步：调整基函数的中心。

$$\mathbf{t}_i(n+1) = \begin{cases} \mathbf{t}_i(n) + \eta[X_k(n) - \mathbf{t}_i(n)] & i = i(X_k) \\ \mathbf{t}_i(n) & \text{其他} \end{cases}$$

- 第五步： $n=n+1$ 转到第二步，直到学完所有的训练样本且中心的分布不再变化。

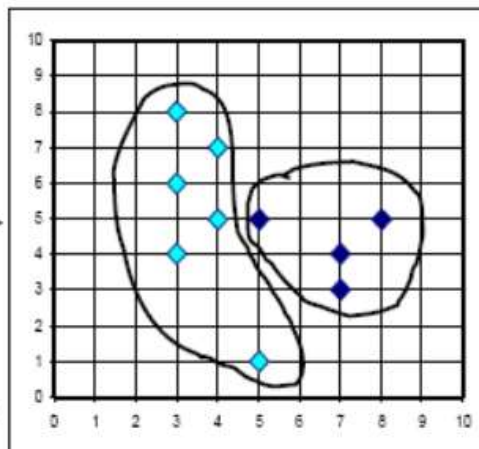
K-means算法过程



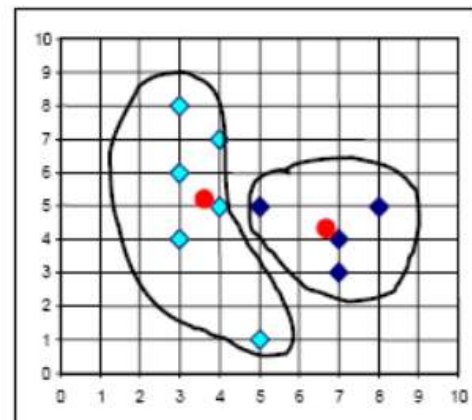
$K=2$

Arbitrarily
choose K object
as initial cluster
center

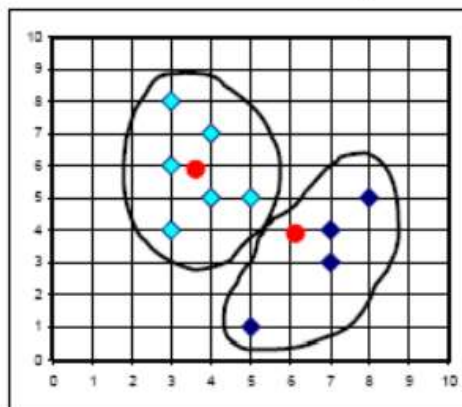
Assign
each
objects
to
most
similar
center



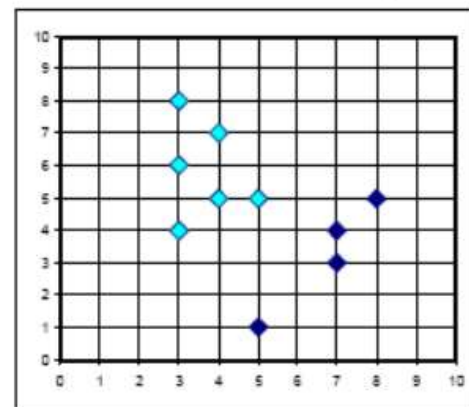
Update
the
cluster
means



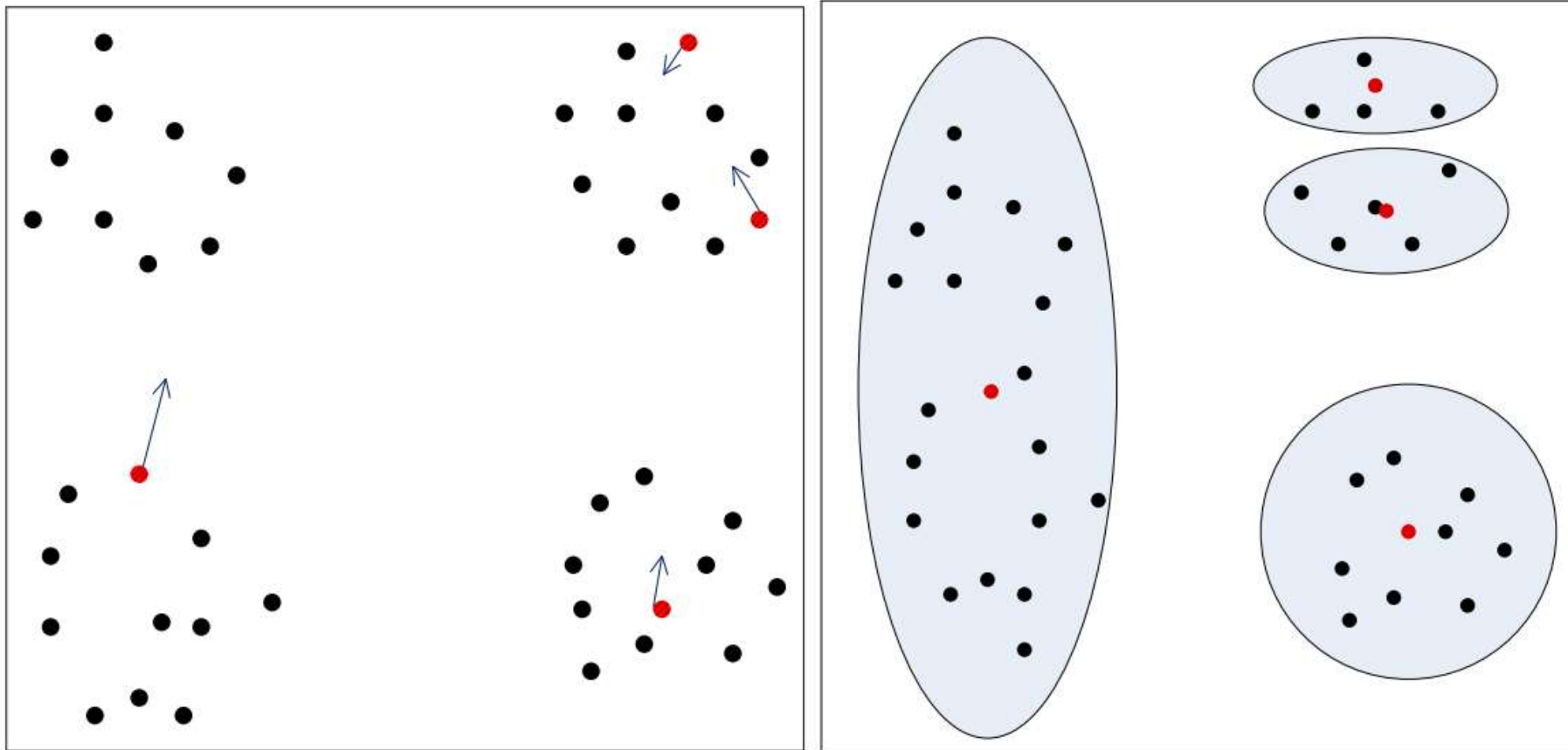
reassign



Update
the
cluster
means



K-means初值敏感



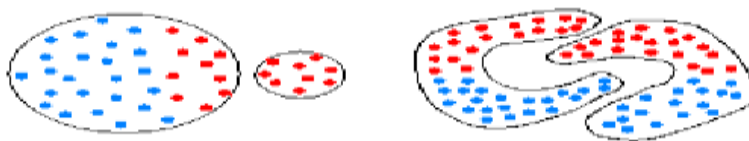
K-means聚类总结

- 算法复杂度 $O(nkt)$ ，其中 t 是迭代次数。
- 优点：
 - ①是解决聚类问题的一种经典算法，简单、快速，对处理大数据集，该算法保持可伸缩性和高效率
 - ②当结果簇是密集的，而簇与簇之间区分明显时，它的效果较好
 - ③常常用于其他聚类算法的初始阶段，如谱聚类

K-means聚类总结

- 缺点:

- ① 在簇的平均值被定义的情况下才能使用，不适涉及有分类属性的数据
- ② 必须事先给出 k （要生成的簇的数目），而且对初值敏感，对于不同的初始值，可能会导致不同结果。
- ③ 不适合于发现非凸面形状的簇或者大小差别很大的簇



- ④ 对噪声和孤立点数据敏感
- ⑤ 算法常以局部最优结束

(2) 确定方差

- 当RBF选用高斯函数，即：

$$G(\| X_k - \mathbf{t}_i \|) = \exp\left(-\frac{1}{2} \sum_{i=1}^I \frac{\| X_k - \mathbf{t}_i \|^2}{\sigma_i^2}\right)$$

- 方差为：

$$\sigma_1 = \sigma_2 = \cdots \sigma_I = \frac{d_{\max}}{\sqrt{2I}}$$

- I 为隐单元的个数， $d_{\max}$ 为所选取中心之间的最大距离。

(3) 学习权值

- 权值的学习可以用LMS方法，也可以直接用伪逆的方法求解，即：

$$\mathbf{W} = \mathbf{G}^+ \mathbf{D}$$

- 式中 $\mathbf{D}=[d_1, \dots, d_k, \dots, d_N]^T$ 是期望响应， $\mathbf{G}^+$ 是矩阵 $\mathbf{G}$ 的伪逆

$$\mathbf{G}^+ = (\mathbf{G}^T \mathbf{G})^{-1} \mathbf{G}^T$$

$$\mathbf{G} = \{g_{ki}\} \quad g_{ki} = \exp\left(-\frac{I}{d_{\max}^2} \|X_k - \mathbf{t}_i\|^2\right)$$
$$k = 1, 2, \dots, N \quad i = 1, 2, \dots, I$$

需注意几点：

- ① K -均值聚类算法的终止条件是网络学完所有的训练样本且中心的分布不再变化。在实际应用中只要前后两次中心的变化小于预先设定的值 ε 即

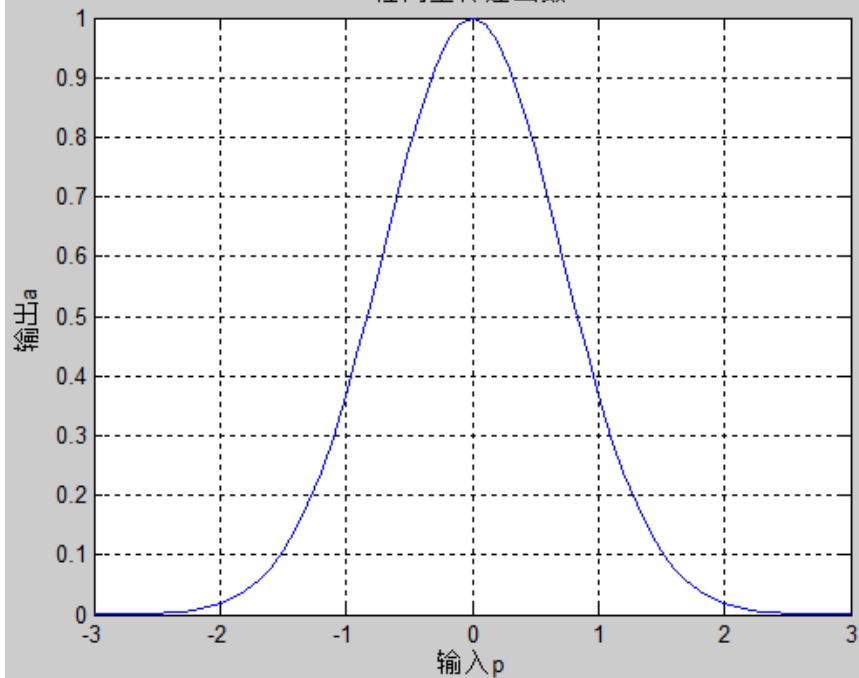
$$|t_i(n+1)-t_i(n)|<\varepsilon \quad (i=1,2,\dots,I),$$

就认为中心的分布不再变化了。

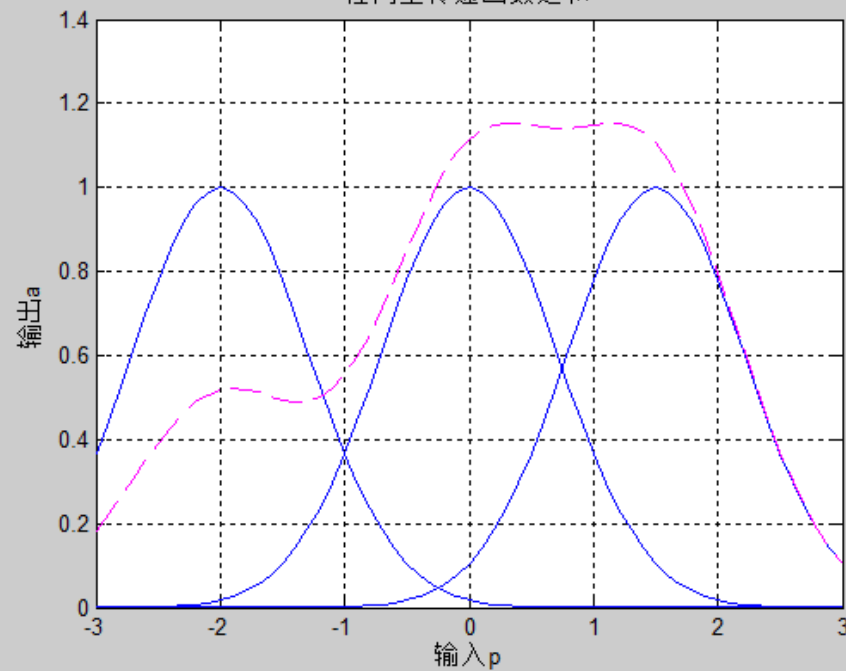
- ② 自组织选取中心法时设所有的基函数其方差都是相同的，实际上每个基函数都有自己的方差，需要在训练过程中根据自身的情况确定。

RBF神经网络工作原理

径向基传递函数



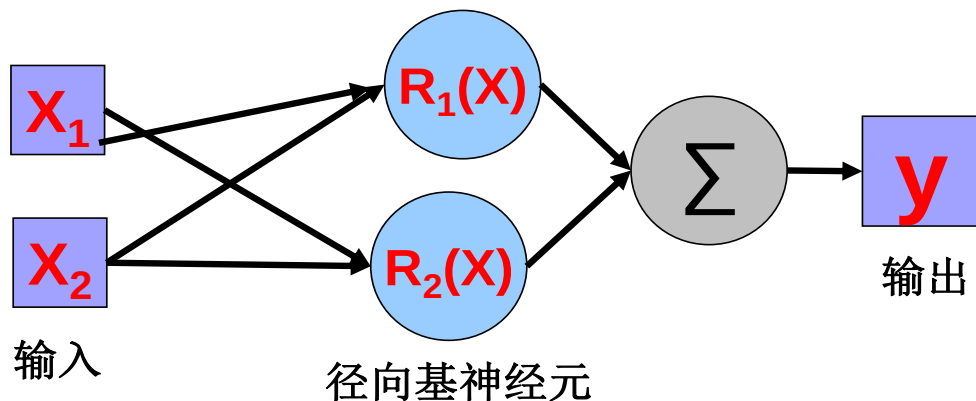
径向基传递函数之和



RBF神经网络工作原理



RBF解决同或/异或问题



同或

x_1	x_2	y
1	0	0
0	1	0
0	0	1
1	1	1

$$R_1(x) = e^{-\|x - \mu_1\|^2} \quad \mu_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$R_2(x) = e^{-\|x - \mu_2\|^2} \quad \mu_2 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

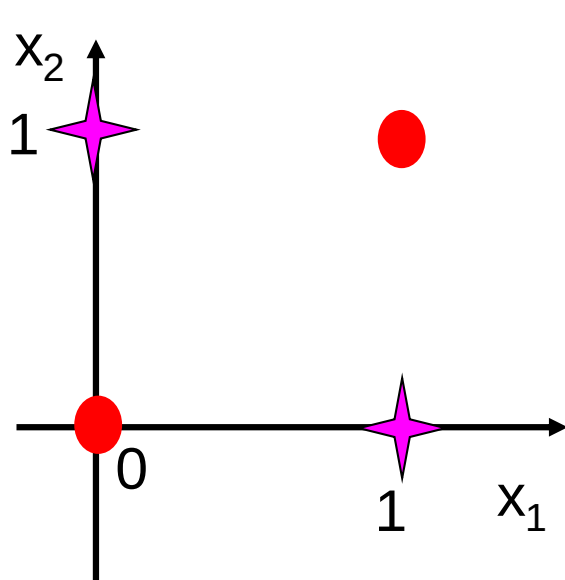
x_1	x_2	$R_1(X)$	$R_2(X)$	Σ
1	0	0.3679	0.3679	<1
0	1	0.3679	0.3679	<1
0	0	0.1353	1	>1
1	1	1	0.1353	>1

$$\|x - \mu_1\|^2 = (x_1 - 1)^2 + (x_2 - 1)^2 = 1$$

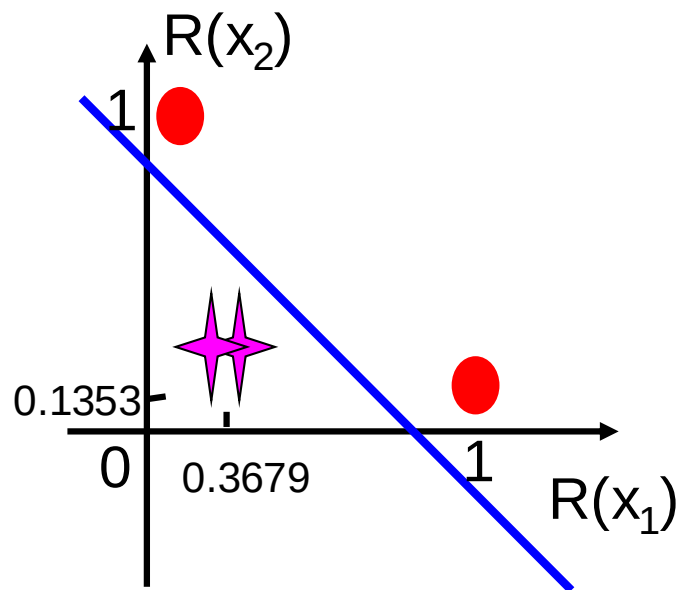
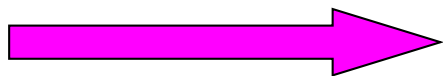
偏置为1

$$R_1(x) = e^{-\|x - \mu_1\|^2} = e^{-1} = 0.3679$$

RBF解决同或/异或问题



空间变换前



空间变换后

RBF网络与多层感知器的比较

- **RBF网络与多层感知器都是非线性多层前向网络，它们都是通用逼近器。对于任一个多层感知器，总存在一个RBF网络可以代替它，反之亦然。但是，这两个网络也存在着很多不同点：**
 - ① **RBF网络只有一个隐层，而多层感知器的隐层可以是一层也可以是多层的。**
 - ② **多层感知器的隐层和输出层其神经元模型是一样的。而RBF网络的隐层神经元和输出层神经元不仅模型不同，而且在网络中起到的作用也不一样。**

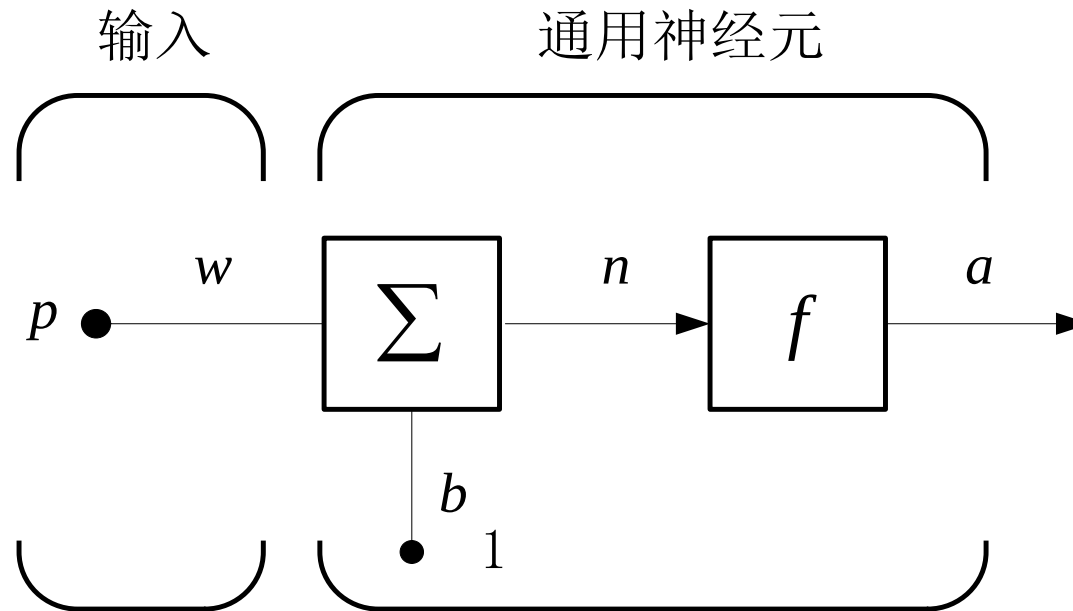
③ **RBF**网络的隐层是非线性的，输出层是线性的。

当用多层感知器解决模式分类问题时，它的隐层和输出层通常选为非线性的。当用多层感知器解决非线性回归问题时，通常选择线性输出层。

④ 多层感知器是对非线性映射的全局逼近，而**RBF**网络使用局部指数衰减的非线性函数对非线性输入输出映射进行局部逼近。这也意味着，逼近非线性输入输出映射，要达到相同的精度，**RBF**网络所需要的参数要比多层感知器少得多。

第五节：神经网络的结构

单输入人工神经元

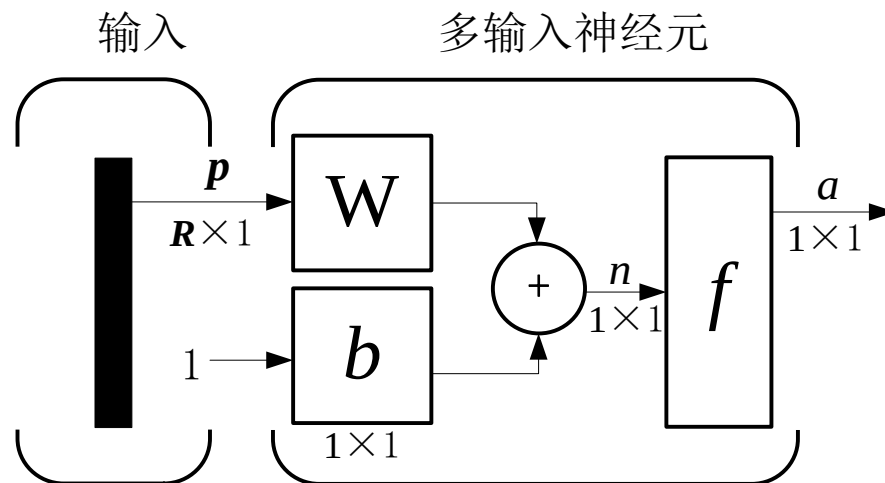
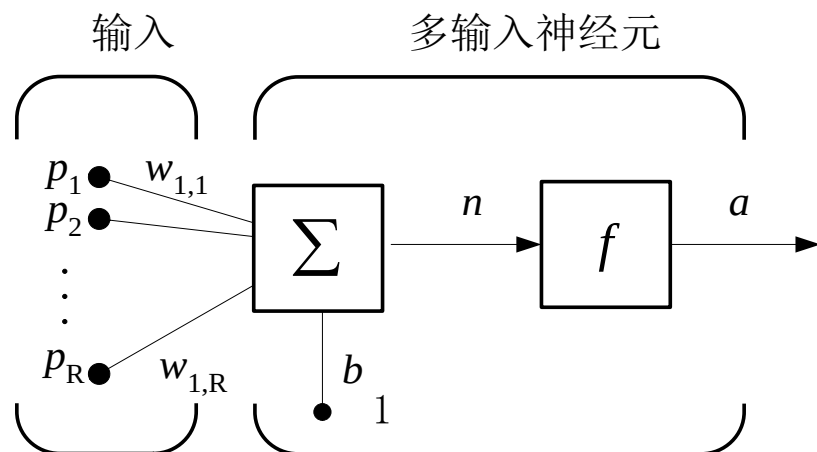


净输入

$$a = f(w p + b)$$

神经元输出 传输函数 权值 输入 偏置(值)

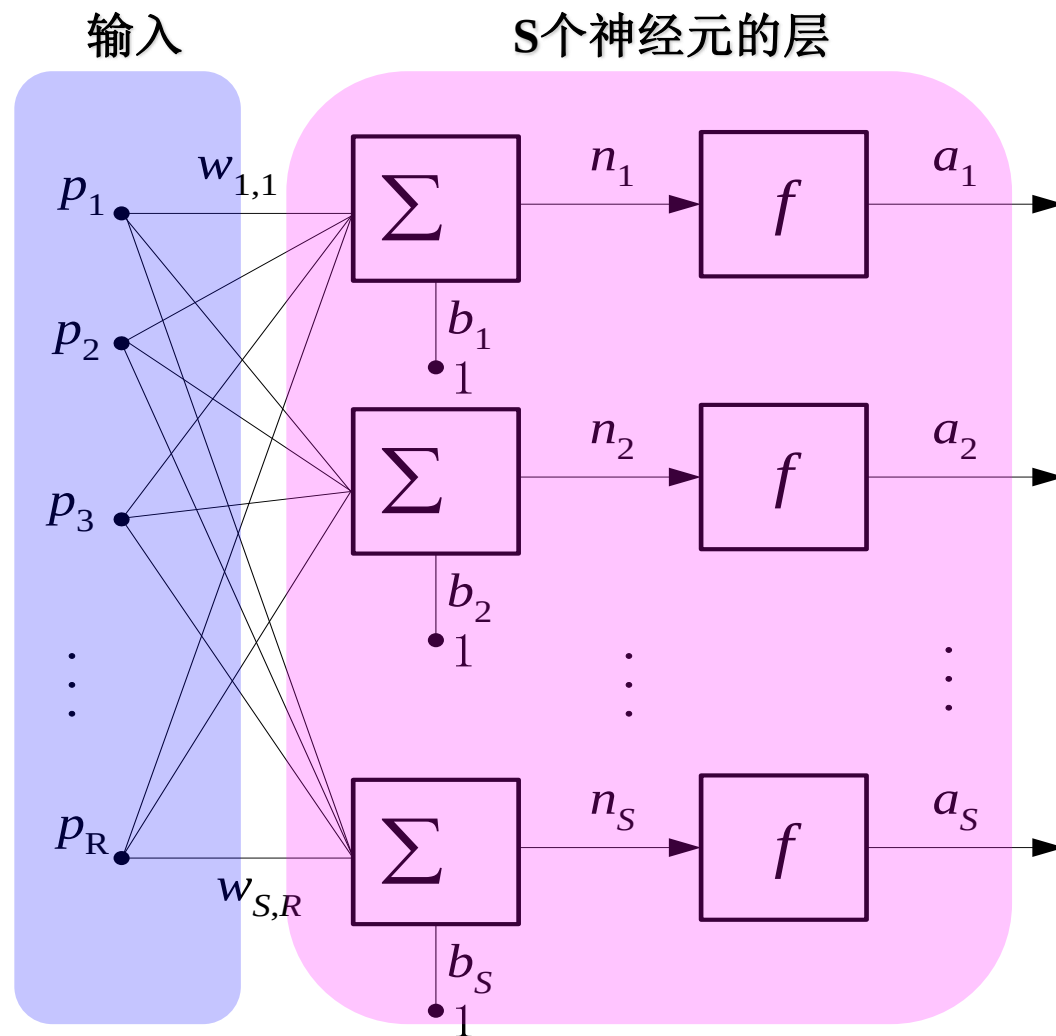
多输入神经元



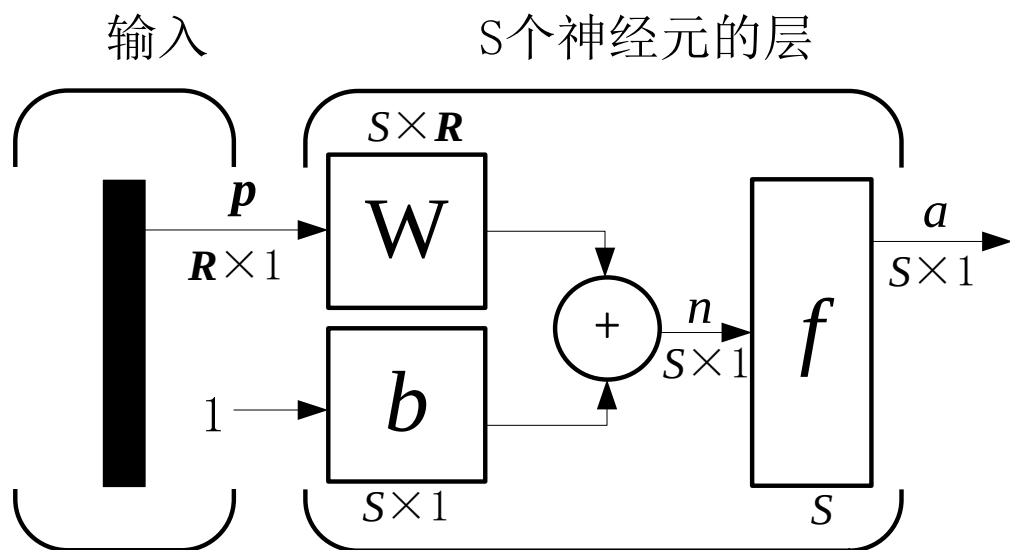
简化符号

$$a = f(\mathbf{W}p + b)$$

网络结构—神经元的层



神经元的层简化模型



$$W = \begin{bmatrix} w_{1,1} & w_{1,2} & \cdots & w_{1,R} \\ w_{2,1} & w_{2,2} & \cdots & w_{2,R} \\ \vdots & \vdots & \ddots & \vdots \\ w_{S,1} & w_{S,2} & \cdots & w_{S,R} \end{bmatrix}$$

$$a = f(Wp + b)$$

$$p = \begin{bmatrix} p_1 \\ p_2 \\ \vdots \\ p_R \end{bmatrix} \quad b = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_S \end{bmatrix} \quad a = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_S \end{bmatrix}$$

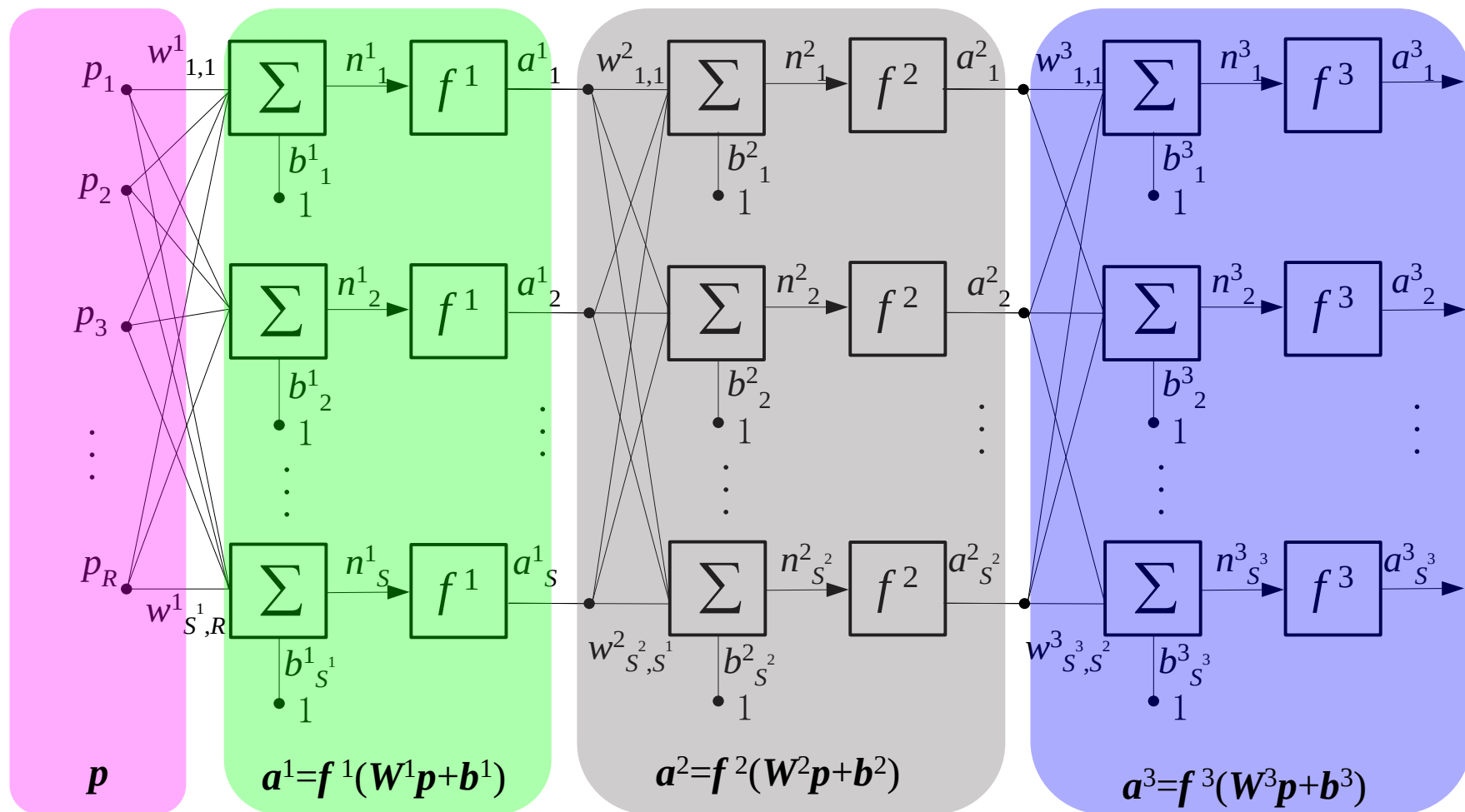
多层神经网络 (3层)

输入

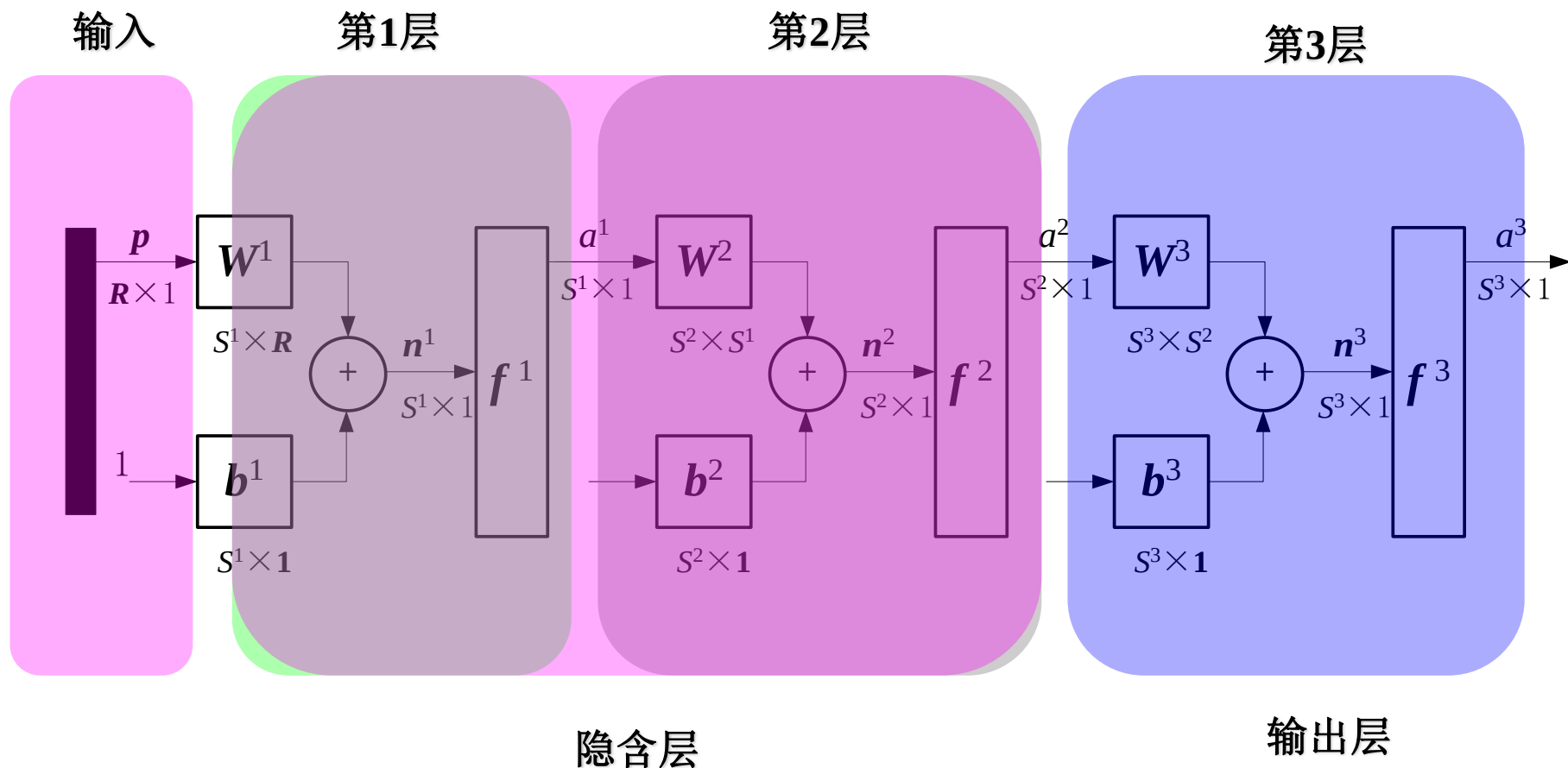
第1层

第2层

第3层



多层神经网络（3层）简化表示



$$a^3 = f^3(W^3 f^2(W^2 f^1(W^1 p + b^1) + b^2) + b^3)$$

多层神经网络讨论

- 多层网络的功能强于单层网络
- 网络结构的确定：
 - ❧ 输入个数
 - ❧ 输出个数
 - ❧ 隐含神经元数
 - ❧ 激活函数
 - ❧ 网络层数
 - ❧ 神经网络权值

多层前馈网络的问题

- 多层前馈网络表示能力

只需要一个包含足够多神经元的隐层, 多层前馈神经网络就能以任意精度逼近任意复杂度的连续函数

- 多层前馈网络局限

- 神经网络由于强大的表示能力, 经常遭遇过拟合。表现为: 训练误差持续降低, 但测试误差却可能上升。

- 如何设置隐层神经元的个数仍然是个未决问题。实际应用中通常使用“试错法”调整。

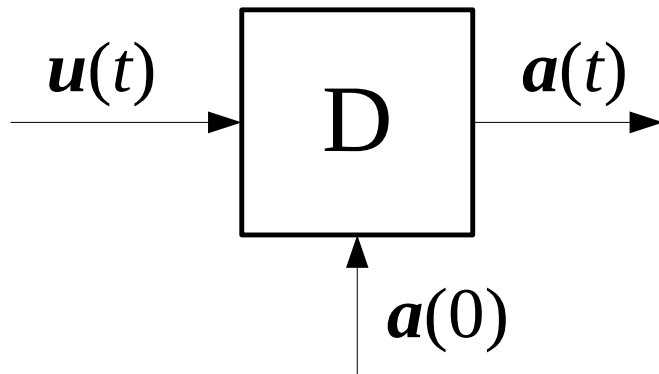
- 缓解过拟合的策略

- 早停: 在训练过程中, 若训练误差降低, 但验证误差升高, 则停止训练。

- 正则化: 在误差目标函数中增加一项描述网络复杂程度的部分, 例如连接权值与阈值的平方和。

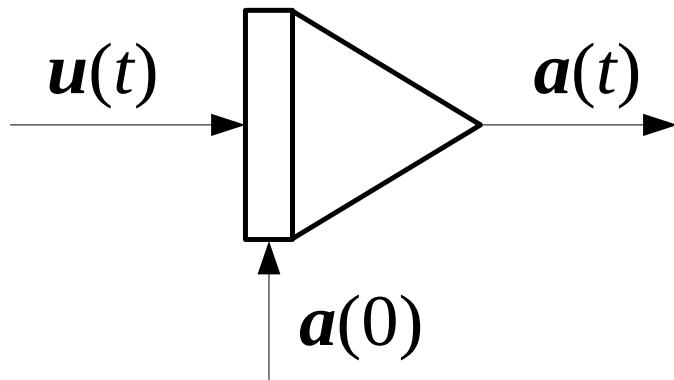
递归网络—构造块

- 延时模块(Delays)



$$a(t) = u(t-1)$$

- 积分模块(Integrators)

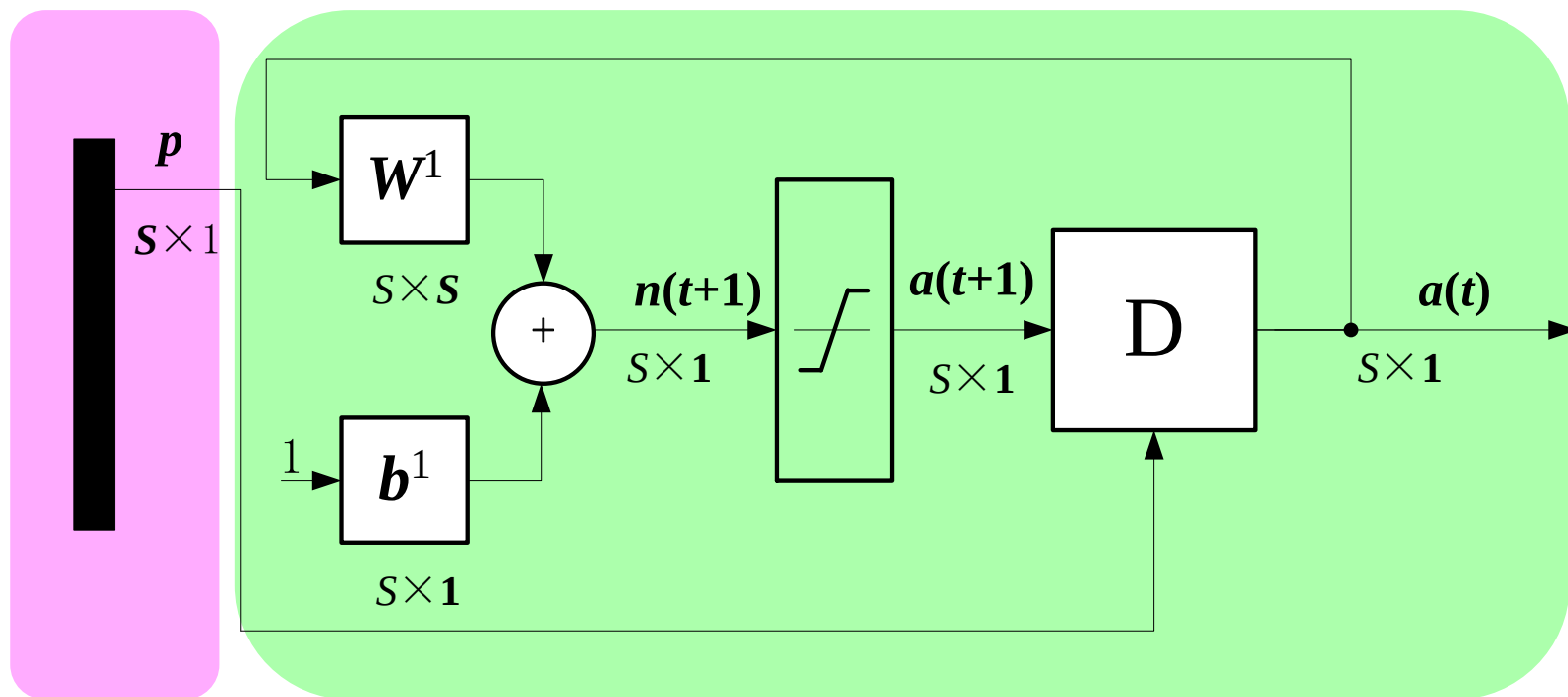


$$a(t) = \int_0^t u(\tau) d\tau + a(0)$$

递归网络 (Recurrent Network)

初始条件

递归层



$$\mathbf{a}(1) = \text{satlins}(\mathbf{W}\mathbf{a}(0) + \mathbf{b}) = \text{satlins}(\mathbf{W}\mathbf{p} + \mathbf{b})$$

$$\mathbf{a}(2) = \text{satlins}(\mathbf{W}\mathbf{a}(1) + \mathbf{b})$$

...

$$\mathbf{a}(t+1) = \text{satlins}(\mathbf{W}\mathbf{a}(t) + \mathbf{b})$$

递归神经网络简单讨论

- 时间性
- 前向网络的特点是将神经元分层，每层神经元之间没有信息交流，一层一层同步计算。
- 反馈神经网络则将整个网络视为整体，神经元之间相互作用，计算也是整体的。

Hopfield神经网络

Hopfield网络是单层对称全反馈网络，根据激活函数选取的不同，可分为离散型和连续性两种（DHNN, CHNN）。

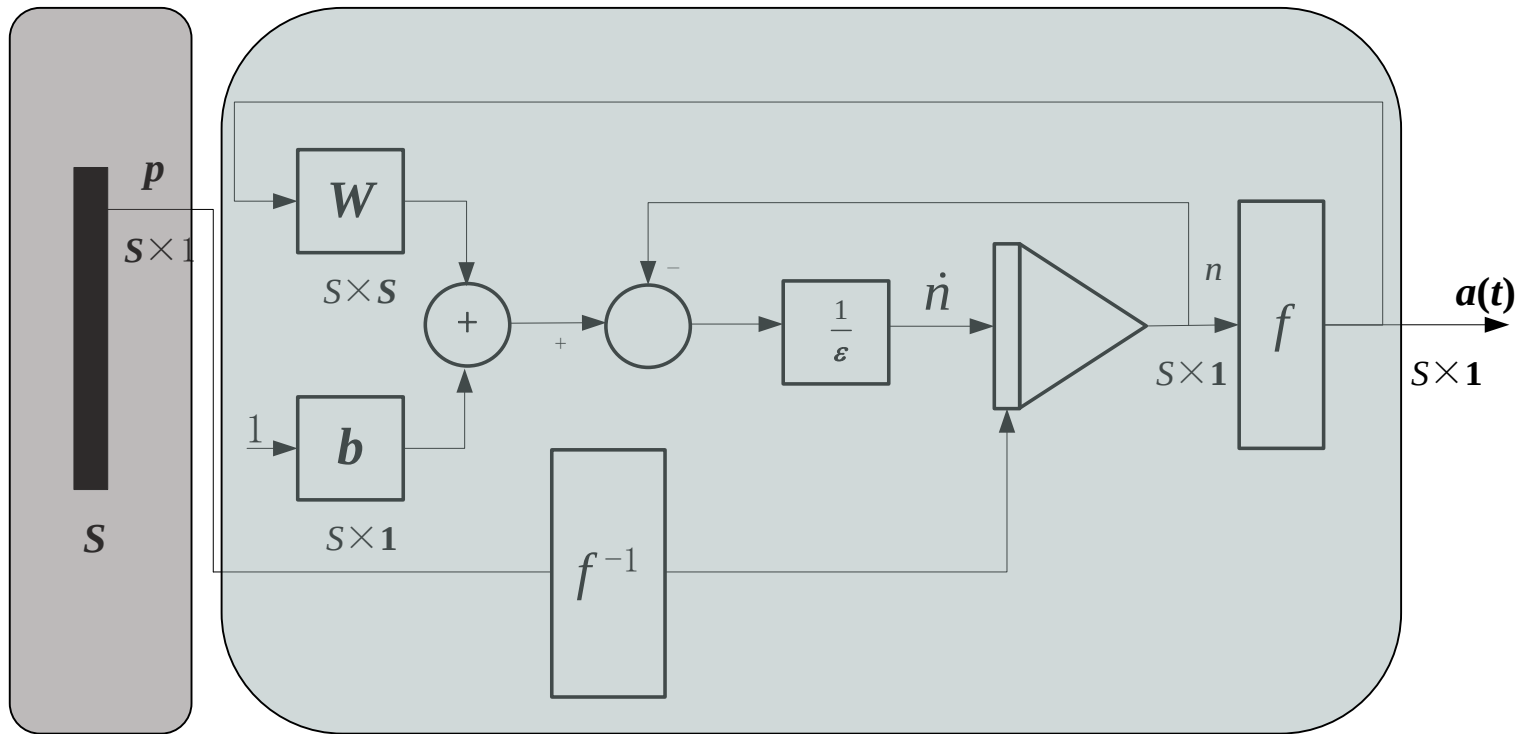
DHNN：激活函数为Hardlim，主要用于联想记忆

CHNN：激活函数为S型函数，主要用于优化计算

Hopfield网络

输入

递归层



$$\mathbf{n}(0) = \mathbf{f}^{-1}(\mathbf{P}), \mathbf{a}(0) = \mathbf{P}$$

$$\epsilon \frac{d\mathbf{n}}{dt} = -\mathbf{n} + \mathbf{W}\mathbf{f}(\mathbf{n}) + \mathbf{b}$$

非线性系统状态演变的形式

在Hopfield网络中，由于反馈的存在，其加权输入和 u_i ， $i=1\sim n$ 为网络状态，网络的输出为 $y_1\sim y_n$ ，则 u, y 的变化过程为一个非线性动力学系统。可用非线性差（微）分方程来描述。一般有如下的几种状态演变形式：

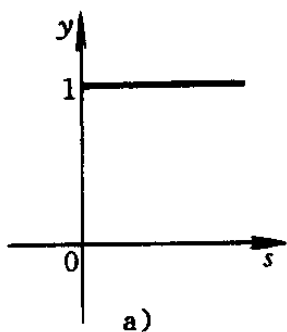
- （1）渐进稳定
- （2）极限环
- （3）混沌现象
- （4）状态轨迹发散

第六节：激活函数

激活函数

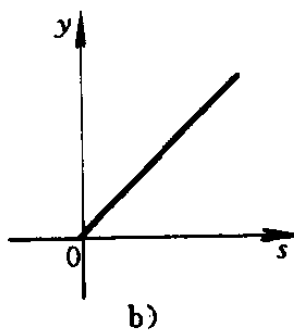
激活函数的基本作用：

- ①控制输入对输出的激活作用；
- ②对输入、输出进行函数转换；
- ③将可能无限域的输入变换成指定的有限范围内的输出。



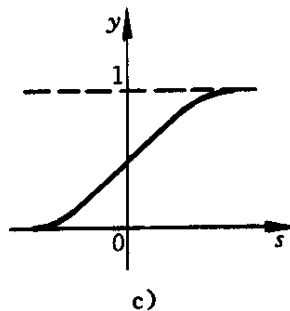
阈值单元

$$\sigma(s) = \begin{cases} 1 & s \geq 0 \\ 0 & s < 0 \end{cases}$$



线性单元

$$y = \sigma(s) = s$$

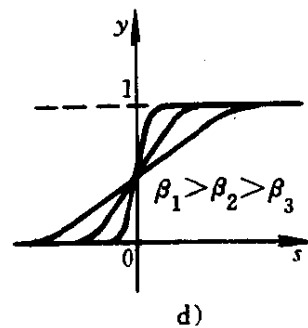


非线性单元

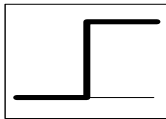
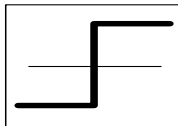
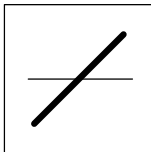
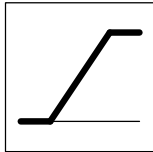
S型 (Sigmoid) 函数

$$\sigma(s) = \frac{1}{1 + e^{-s}}$$

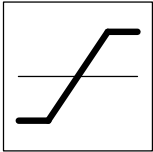
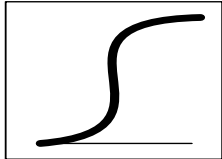
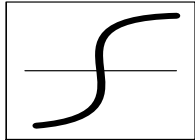
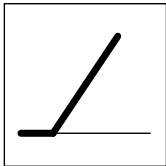
$$\sigma(s) = \tanh(\beta s)$$



常见激活函数

名称	输入/输出关系	图标	Matlab函数
硬极限函数	$\begin{cases} a = 0 & n < 0 \\ a = 1 & n \geq 0 \end{cases}$		Hardlim
对称极限函数	$\begin{cases} a = -1 & n < 0 \\ a = 1 & n \geq 0 \end{cases}$		Hardlims
线性函数	$a = n$		Pureline
饱和线性函数	$\begin{cases} a = 0 & n < 0 \\ a = n & 0 \leq n \leq 1 \\ a = 1 & n \geq 1 \end{cases}$		Satlin

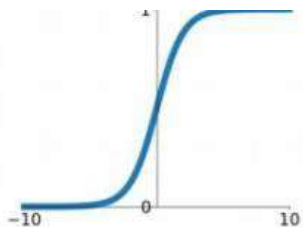
常见激活函数

对称饱和线性函数	$\begin{cases} a = -1 & n < 0 \\ a = n & 0 \leq n \leq 1 \\ a = 1 & n \geq 1 \end{cases}$		Satlin
对数S型函数	$a = \frac{1}{1 + e^{-n}}$		Logsig
双曲正切S型函数	$a = \frac{e^n - e^{-n}}{e^n + e^{-n}}$		Tansig
正线性函数	$\begin{cases} a = 0 & n < 0 \\ a = n & n \geq 0 \end{cases}$		Poslin
竞争函数	$\begin{cases} a = 1 & \max(n) \\ a = 0 & \text{其它} \end{cases}$		Compet

常见激活函数

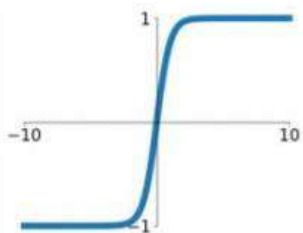
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



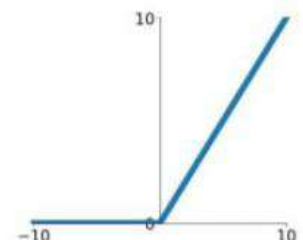
tanh

$$\tanh(x)$$



ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

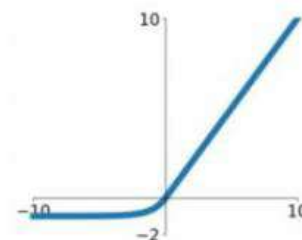


Maxout

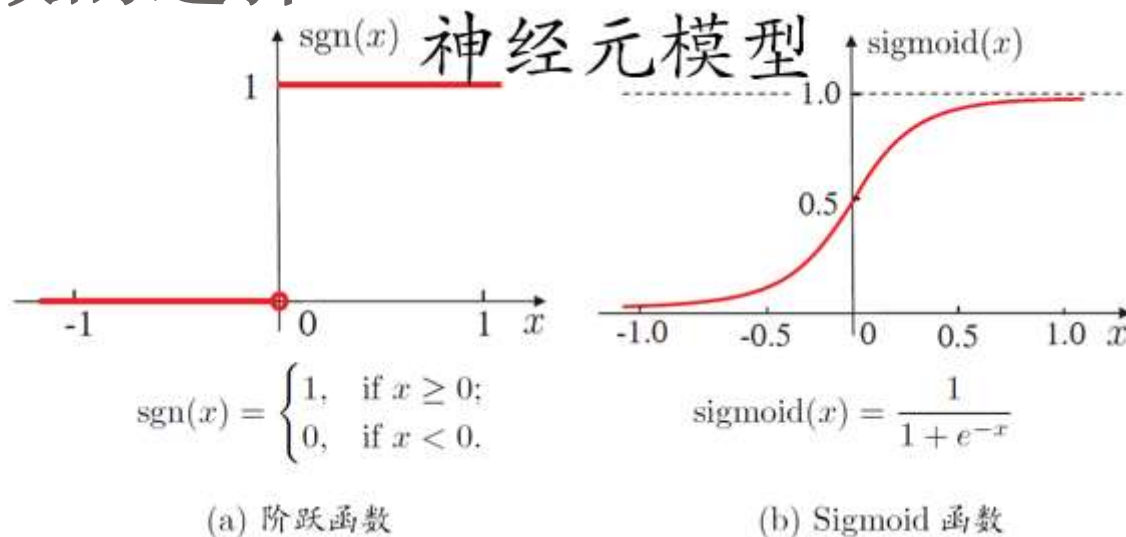
$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



激活函数的选择



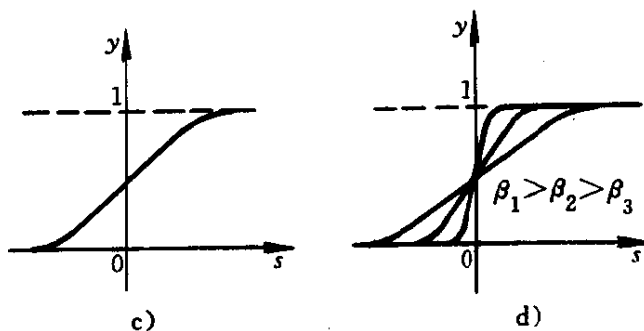
- 理想激活函数是阶跃函数, 0表示抑制神经元而1表示激活神经元
- 阶跃函数具有不连续、不光滑等不好的性质

激活函数的选择

- Sigmoid 非线性激活函数

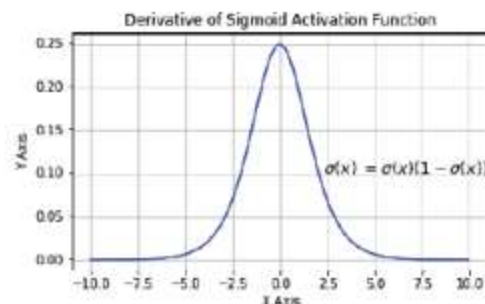
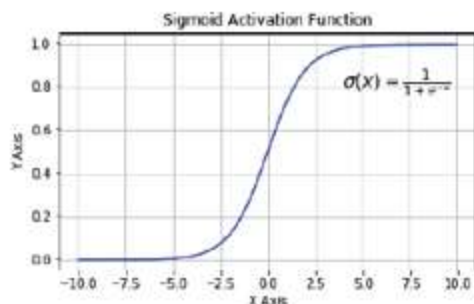
$$\sigma(x) = 1 / (1 + e^{-x})$$

- Sigmoid函数输入一个实值的数，然后将其压缩到0~1的范围内。特别地，大的负数被映射成0，大的正数被映射成1。
- Sigmoid函数在历史上流行过一段时间，因为它能够很好的表达“激活”的意思，未激活就是0，完全饱和的激活则是1。



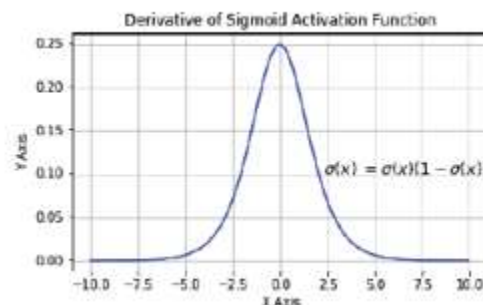
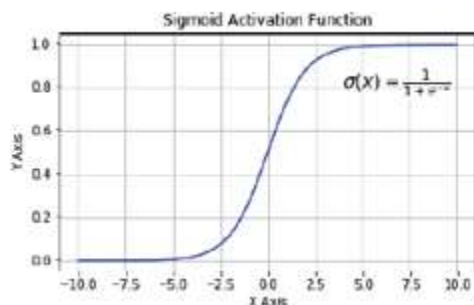
Vanishing gradients

- Sigmoid容易饱和，并且当输入非常大或者非常小的时候，神经元的梯度就接近于0了。
- 这就使得反向传播接近于0的梯度，导致最终权重基本没什么更新，就无法递归地学习到输入数据了。
- 参数的初始值尽量避免饱和。如果初始值很大，大部分神经元可能都会处在饱和的状态而把梯度消失，导致网络变的很难学习。

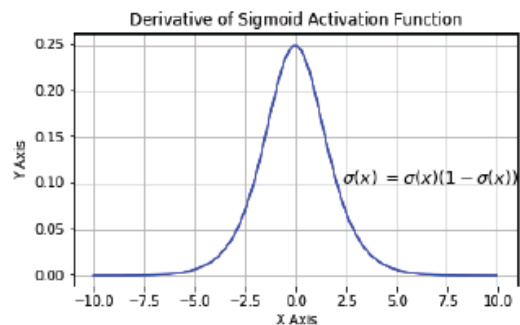
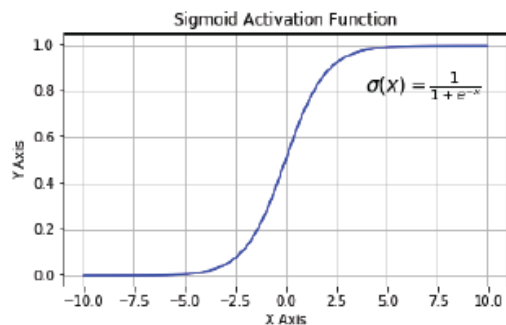


Zero centered

- Sigmoid 的输出不是0均值的，这是不希望的，因为这会导致后层的神经元的输入是非0均值的信号，这会对梯度产生影响。
- 假设后层神经元的输入都为正(e.g. $x > 0$ elementwise in $f = w^T x + b$), 那么对 w 求局部梯度则都为正，这样在反向传播的过程中 w 要么都往正方向更新，要么都往负方向更新，导致有一种捆绑的效果，使得收敛缓慢。
- 如果是按batch去训练，那么每个batch可能得到不同的符号（正或负），那么相加一下这个问题还是可以缓解。

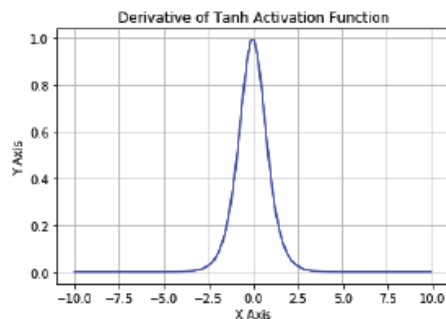
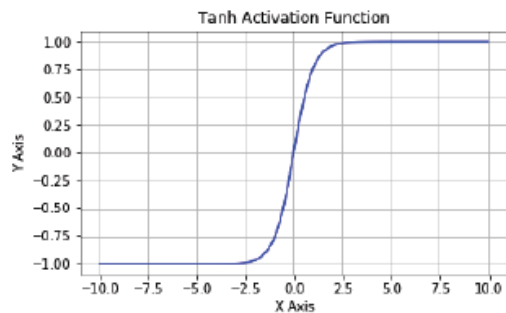


激活函数的选择



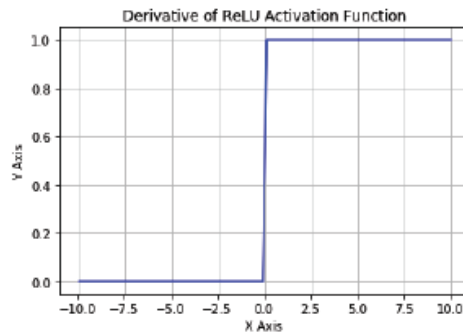
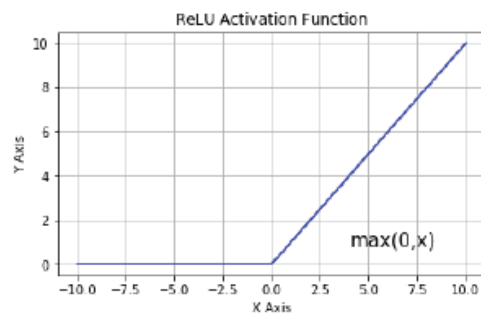
Sigmoid

- Vanishing gradients
- Not zero centered



Tanh

- Vanishing gradients



ReLU

- Not zero centered

激活函数的选择建议

- Sigmoid和tanh函数在输入过小和过大的时候都会出现梯度消失和梯度爆炸的问题，这不利于网络层数加深和优化；
- ReLU不会导致梯度消失和爆炸的问题，但是因为 x 小于 0 时函数值恒为 0，会导致一些神经元无法激活；
- Leaky ReLU计算高效、快速收敛、在正区域内不会饱和，导数总是不为零，这能减少静默神经元的出现，允许基于梯度的学习一定程度上缓解了 dead ReLU 问题；
- ELU不会有Dead ReLU问题,输出的均值接近0，zero-centered缺点就是计算量稍大；
- 当前最常用的是 ReLU。Sigmoid 一般不在隐藏层使用，对于二分类问题，可以在最后的输出层使用。

第七节：网络训练学习

训练学习方法



有监督学习

在有监督的学习方式中，网络的输出和期望的输出（即教师信号）进行比较，然后根据两者之间的差异调整网络的权重，最终使差异变小。



无监督学习

在无监督的学习方式中，输入模式进入网络后，网络按照一预先设定的规则（如竞争规则）自动调整权重，使网络最终具有模式分类等功能。



再励学习

再励学习是介于上述两者之间的一种学习方法

学习规则（无监督）

- Hebb学习规则

- 这个规则是由Donald Hebb在1949年提出的
- 基本规则：如果处理单元从另一个处理单元接受到一个输入，并且如果两个单元都处于高度活动状态，这时两单元间的连接权重就要被加强
- 是一种无监督学习方法，它只根据神经元连接间的激活水平改变权重，因此此这种方法又称为相关学习或并联学习

学习规则（无监督）

● 竞争式学习规则

- 竞争式学习属于无监督学习方式。这种学习方式是利用不同层间的神经元发生兴奋性联接以及同一层内距离很近的神经元间发生同样的兴奋性联接，而距离较远的神经元产生抑制性联接
- 本质在于神经网络中高层次的神经元对低层次神经元的输入模式进行竞争识别

● Kohonen学习规则

- 该规则是由Teuvo Kohonen在研究生物系统学习的基础上提出的，只用于无监督训练的网络
- 最靠近输入向量的神经元权值向量得到修正，使之更靠近输入向量，其结果是获胜的神经元在下一次获胜的输入向量出现时，获胜的可能性会更大；而对于那些与输入向量相差很远的神经元权值向量，获胜的可能性变得很小。

学习规则（无监督）

● 概率式学习规则

- 从统计力学、分子热力学和概率论中关于系统稳态能量的标准出发，进行神经网络学习的方式称概率式学习
- 误差曲线可能会出现局部的最小值，在网络学习时，应尽可能摆脱误差的局部最小值，而达到真正的误差最小值
- 神经网络处于某一状态的概率主要取决于在此状态下的能量，能量越低，概率越大。同时，此概率还取决于温度参数 T 。 T 越大，不同状态出现概率的差异便越小，较容易跳出能量的局部极小点而到全局的极小点； T 越小时，情形正相反。概率式学习的典型代表是Boltzmann机学习规则。

学习规则（有监督）

- Delta (δ) 学习规则
 - Delta规则是最常用的学习规则，其要点是改变单元间的连接权重来减小系统实际输出与应有的输出间的误差
 - 这个规则也叫Widrow-Hoff学习规则，首先在Adaline模型中应用，也可称为最小均方差规则
- 后向传播学习规则
 - BP网络的学习算法是在Delta规则基础上发展起来的，可在多层网络上有效地学习
 - 后向传播学习，是目前应用最为广泛的神经网络学习规则
- 梯度下降学习规则
 - 梯度下降学习规则的要点为在学习过程中，保持误差曲线的梯度下降
 - 误差曲线可能会出现局部的最小值，在网络学习时，应尽可能摆脱误差的局部最小值，而达到真正的误差最小值

再励学习（增强学习）

- 与有监督学习类似，比较适合控制等工程应用系统
- 无确定目标输出，对应一个级别，而该级别对应网络性能测度
- 外部环境对系统的输出结果给出评价，学习系统通过强化受奖的动作来改善自身性能。

第八节：损失函数和风险函数

损失函数与风险函数

● 损失函数：衡量当前训练模型预测质量的高低，可以理解为模型的输出为 $f(x)$ ，真实值为 y ，那么损失函数就是度量 $f(x)$ 预测真实输出 y 错误的偏离程度。

风险函数：衡量平均意义下模型预测的好坏，常常用经验风险最小化，也就是对当前所有训练样本的平均误差误差最小，就能认为模型在经验上是最好的。损失函数是 $f(x)$ 和 y 的非负实值函数，记作 $L(y, f(x))$

错误率&误差

- 错误率: 错分样本的占比
- 误差: 样本真实输出与预测输出之间的差异
 - 训练(经验)误差: 训练集上
 - 测试误差: 测试集
 - 泛化误差: 除训练集外所有样本

由于事先并不知道新样本的特征, 我们只能努力使经验误差最小化;

很多时候虽然能在训练集上做到分类错误率为零, 但多数情况下这样的学习器并不好

损失函数



Regression

What is the temperature going to be tomorrow?



Classification

Will it be Cold or Hot tomorrow?



- Loss function quantifies gap between prediction and ground truth
- For regression:
 - Mean Squared Error (MSE)
- For classification:
 - Cross Entropy Loss

Mean Squared Error

$$MSE = \frac{1}{N} \sum (t_i - s_i)^2$$

Prediction $\rightarrow s_i$

Ground Truth $\rightarrow t_i$

Cross Entropy Loss

$$CE = - \sum_i^C t_i \log(s_i)$$

Classes $\rightarrow C$

Prediction $\rightarrow s_i$

Ground Truth $\{0,1\} \rightarrow t_i$

Softmax函数

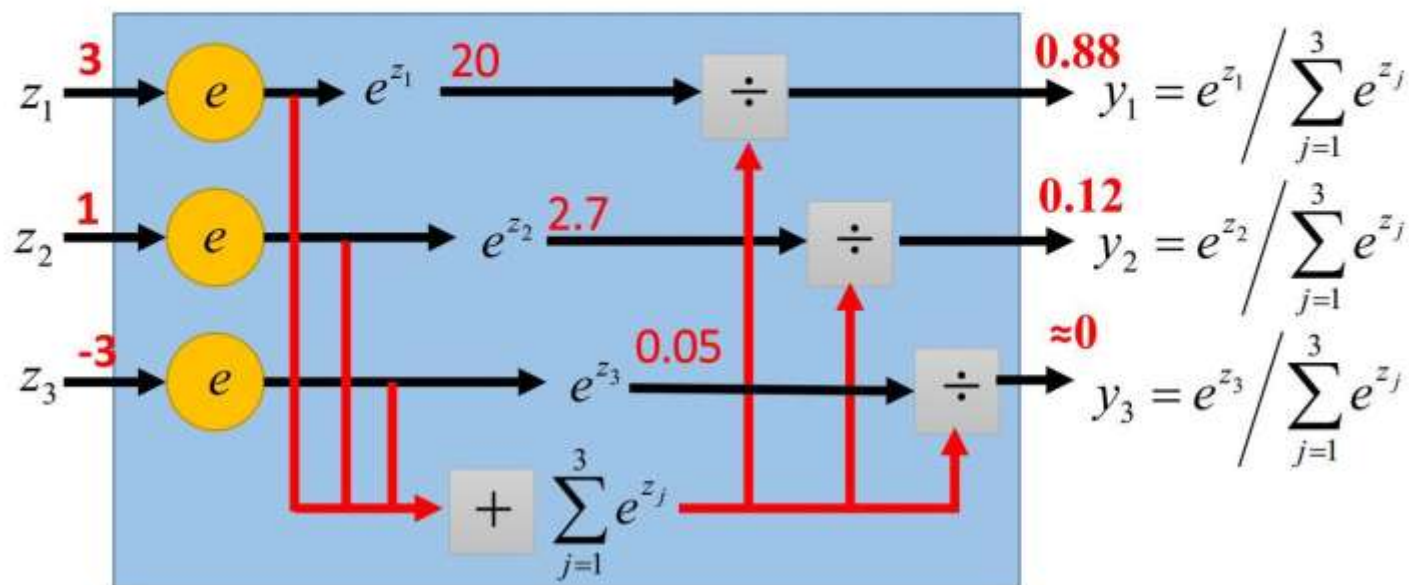
对任意的输入，都能归一化成0到1的数，从而可以表示概率，下图给了一个例子

- Softmax layer as the output layer

Probability:

- $1 > y_i > 0$
- $\sum_i y_i = 1$

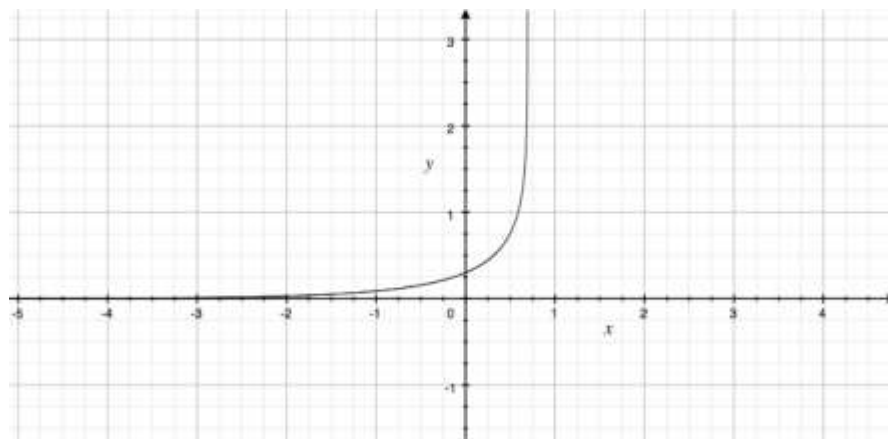
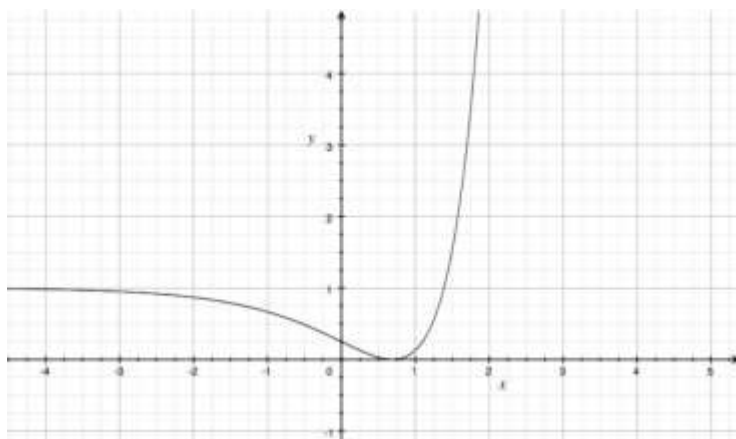
Softmax Layer



损失函数

$$loss_i = (y_i - y_{i-})^2 = (y_i - \frac{e^{y_{i-}}}{\sum_{j=1}^n e^{y_{j-}}})^2$$

$$loss_i = -\log(y_{i-})$$



- 加入softmax后的Loss图像

损失函数

8 常用的损失函数还有：

1) 0-1 损失函数 (0-1 loss function)

$$L(y, f(x)) = \begin{cases} 1, & y \neq f(x) \\ 0, & y = f(x) \end{cases}$$

2) 平方损失函数 (quadratic loss function)

$$L(y, f(x)) = (y - f(x))^2$$

3) 绝对损失函数 (absolute loss function)

$$L(y, f(x)) = |y - f(x)|$$

4) 对数损失函数 (logarithmic loss function) 或者对数似然损失函数 (log-likelihood loss function)

$$L(y, f(x)) = -\log(y|x)$$

风险函数

- 给定一个训练数据集 $T\{(x_1, y_1), (x_2, y_2), (x_3, y_3), \dots, (x_n, y_n)\}$
- 模型 $f(x)$ 关于训练数据集的平均损失称为经验风险或经验损失，记作 $R(f(x))$:

$$R(f(x)) = \frac{1}{N} \sum_{i=1}^N L(y_i, f(x_i))$$

- 损失函数值越小，模型就越好。但是由于现实中训练样本有限，甚至很小，所以经验风险对于学习泛化性能高的模型是不够的。因此学习理论中认为，学习一个泛化性能高的模型需要同时达到：经验风险最小化和结构风险最小化。结构风险最小化就是利用正则化技术实现的。

经验风险和结构风险

- 结构风险最小化（structural risk minimization, SRM），也就是正则化。在经验风险最小化的目标下加上结构风险最小化

$$R(f(x)) = \frac{1}{N} \sum_{i=1}^N L(y_i, f(x_i)) + \lambda J(f)$$

其中 $J(f)$ 代表模型的复杂度，用于控制模型的复杂度。在深度学习中，最常用的是用L2和L1损失控制 f 的权重的大小。

过拟合与欠拟合



过拟合、欠拟合的直观类比

过拟合：学习器把训练样本本身特点当做所有潜在样本都会具有的一般性质。

欠拟合：训练样本的一般性质尚未被学习器学好。

过拟合与欠拟合

- 过拟合:

学习器把训练样本学习的“太好”，将训练样本本身的特点当做所有样本的一般性质，导致泛化性能下降

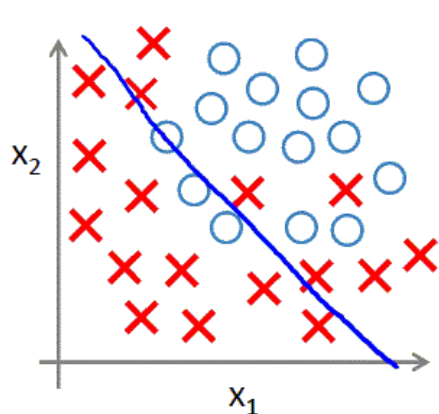
- 优化目标加正则项
- early stop

- 欠拟合:

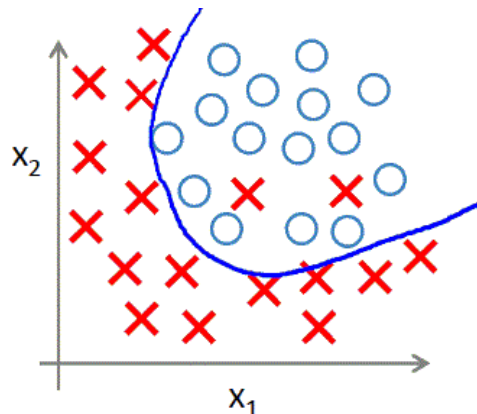
对训练样本的一般性质尚未学好

- 决策树:拓展分支
- 神经网络:增加训练轮数

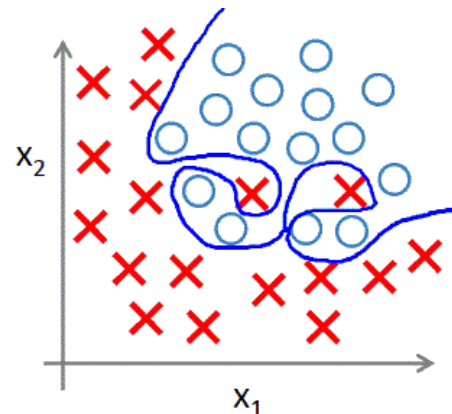
结构风险最小化的必要性



● 欠拟合



正则化



过拟合

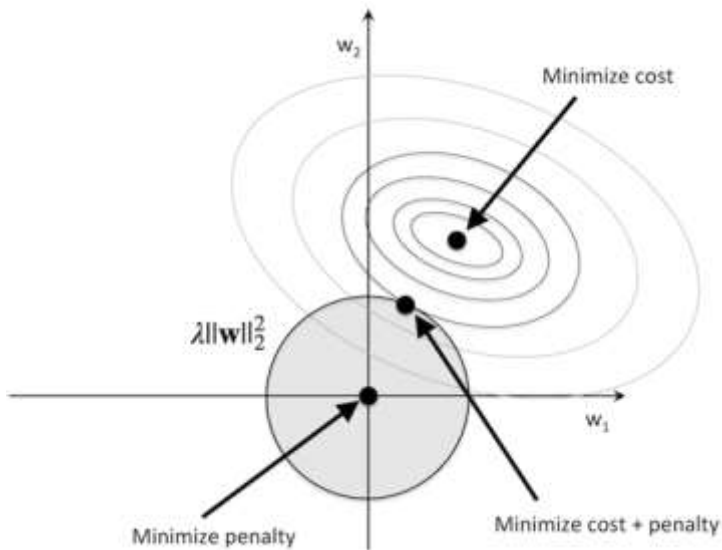
一味追求对训练数据的预测能力，所选模型的复杂度往往会比真实模型更高，这种现象称为过拟合。而正则化在这个过程中对经验风险最小化模型进行了修正，使得结构风险也能达到最小。

正则化 (regularization)

所有损害优化的方法都是正则化

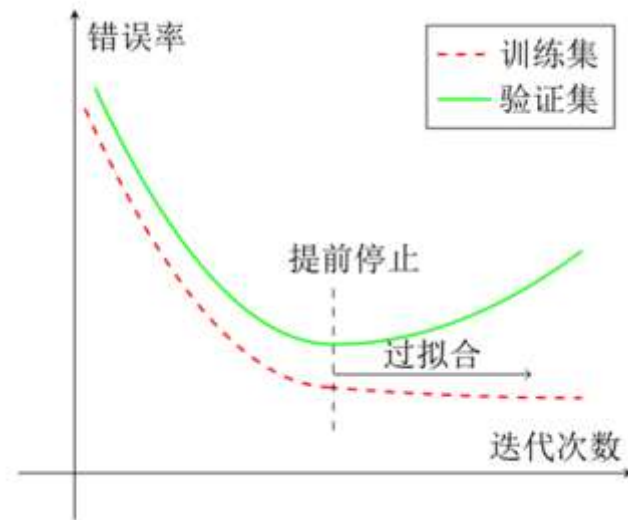
增加优化约束

L1/L2约束、数据增强



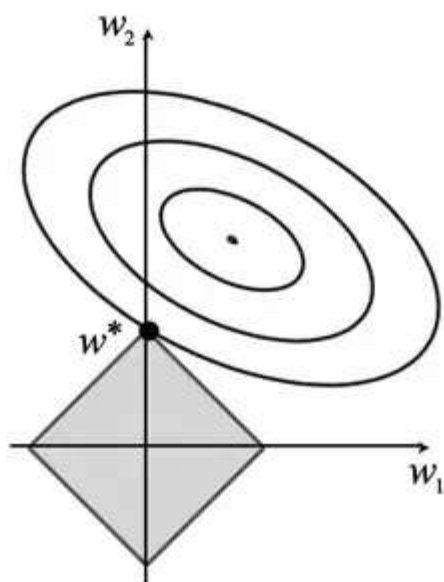
干扰优化过程

权重衰减、随机梯度下降、提前停止

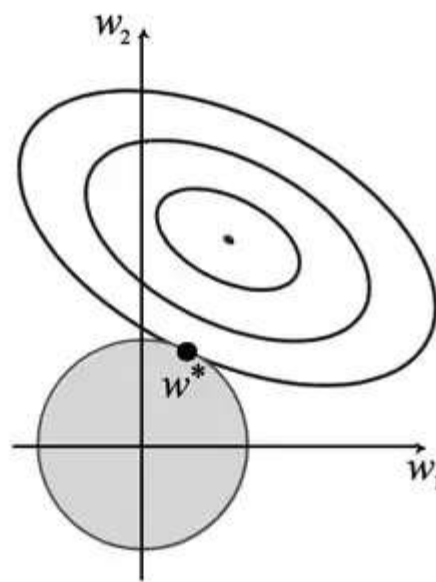


L2和L1范数对模型的影响

- **L1范数：** L1正则化可以产生稀疏参数矩阵，即产生一个稀疏模型，可以用于特征选择
- L2范数：** L2正则化可以产生一个所有参数都比较小的模型，这样可以避免模型过拟合到某一维特征



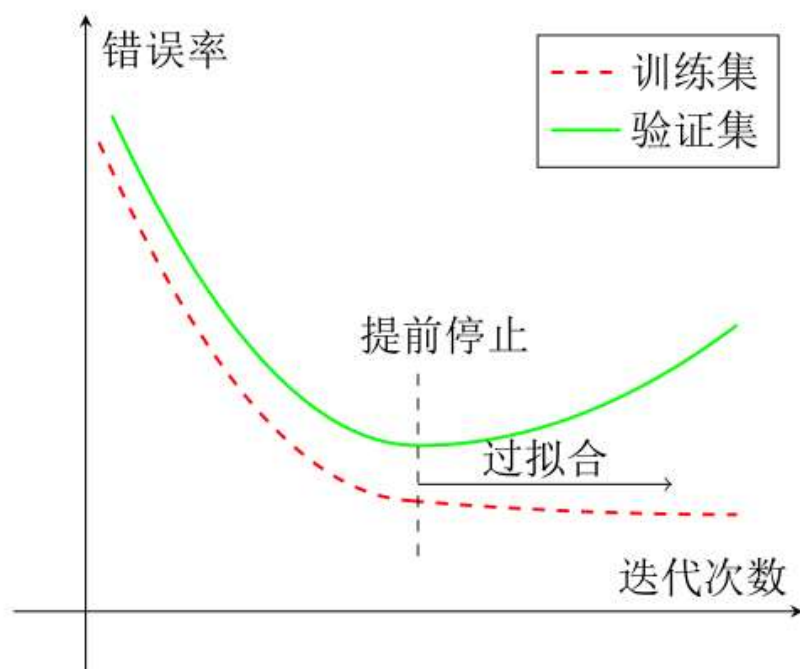
L1



L2

提前停止

- 使用一个验证集 (Validation Dataset) 来测试每一次迭代的参数在验证集上是否最优。如果在验证集上的错误率不再下降，就停止迭代。



评估方法

- 现实任务中往往会对学习器的泛化性能、时间开销、存储开销、可解释性等方面的因素进行评估并做出选择
- 假设测试集是从样本真实分布中独立采样获得，将测试集上的“测试误差”作为泛化误差的近似，所以测试集要和训练集中的样本尽量互斥。

评估方法

通常将包含个 m 样本的数据集 $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$ 拆分成训练集 S 和测试集 T :

- 留出法:
 - 直接将数据集划分为两个互斥集合
 - 训练/测试集划分要尽可能保持数据分布的一致性
 - 一般若干次随机划分、重复实验取平均值

评估方法

- 交叉验证法:

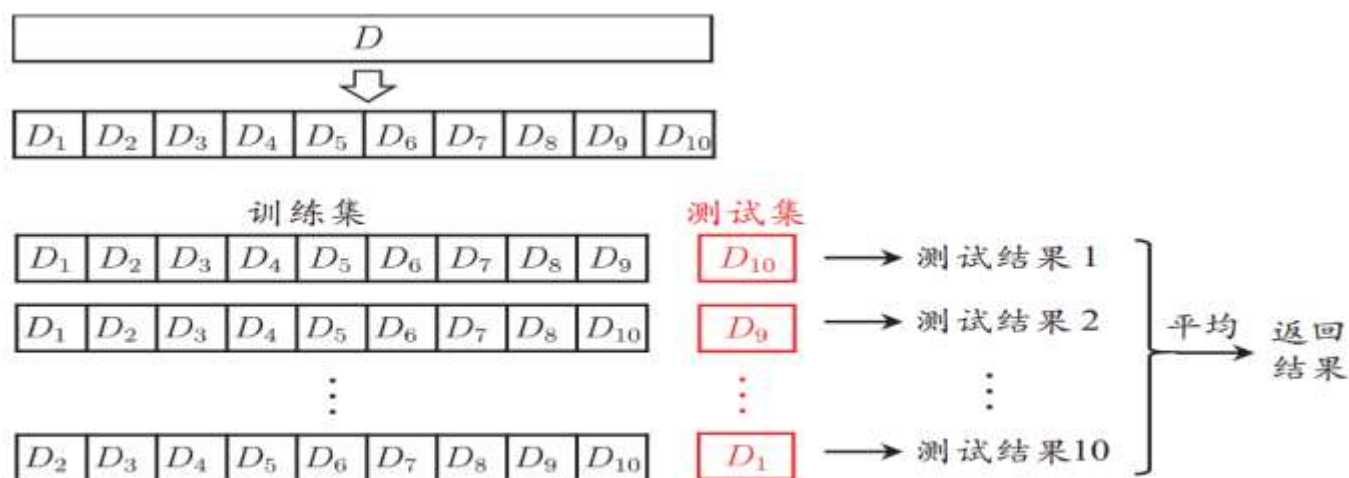
将数据集分层采样划分为 k 个大小相似的互斥子集，每次用 $k-1$ 个子集的并集作为训练集，余下的子集作为测试集，最终返回 k 个测试结果的均值， k 最常用的取值是10.

假设数据集 D 包含 m 个样本，若令 $k = m$, 则得到留一法:

- 不受随机样本划分方式的影响
- 结果往往比较准确
- 当数据集比较大时，计算开销难以忍受

评估方法

与留出法类似，将数据集 D 划分为 k 个子集同样存在多种划分方式，为了减小因样本划分不同而引入的差别， k 折交叉验证通常随机使用不同的划分重复 p 次，最终的评估结果是这 p 次 k 折交叉验证结果的均值，例如常见的“10次10折交叉验证”



10 折交叉验证示意图

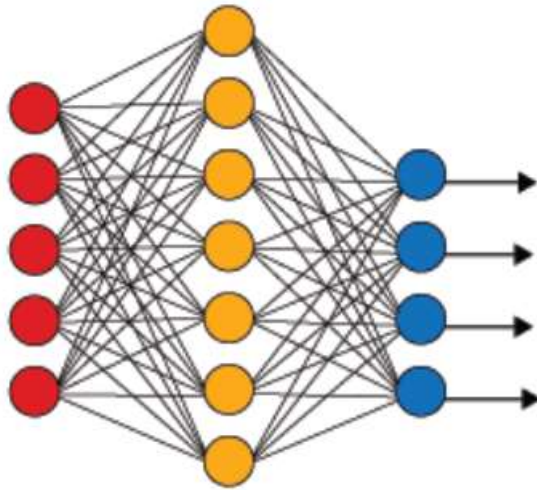
模型评估建议

- 现有做法是将已有的数据集划分为训练的测试集，只用训练集训练，测试集不参与训练，而参与模型的评估。
- 测试集和训练集的规模可以根据实际问题制定。经验的7成数据训练3成数据测试已经能够很好的评估训练模型的性能。

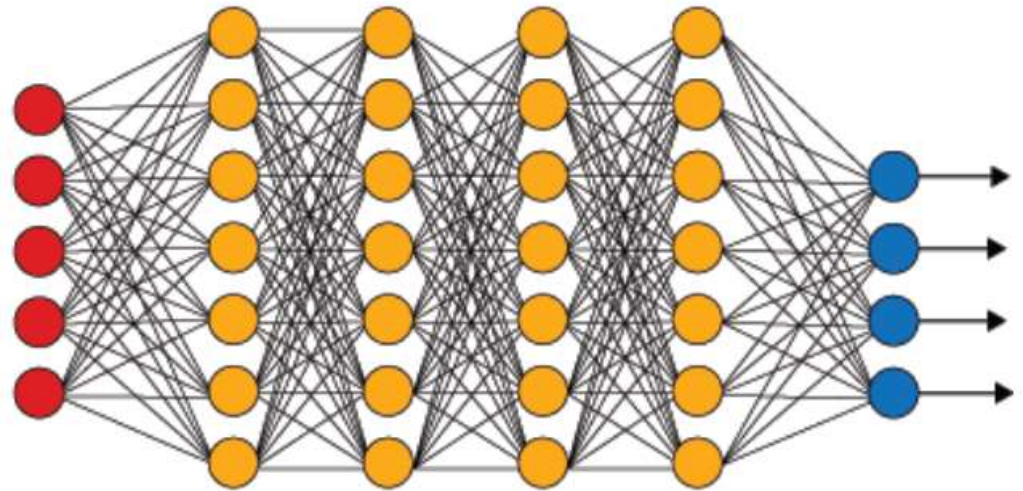
第九节：深度学习

人工神经网络模型

Simple Neural Network



Deep Learning Neural Network



● Input Layer

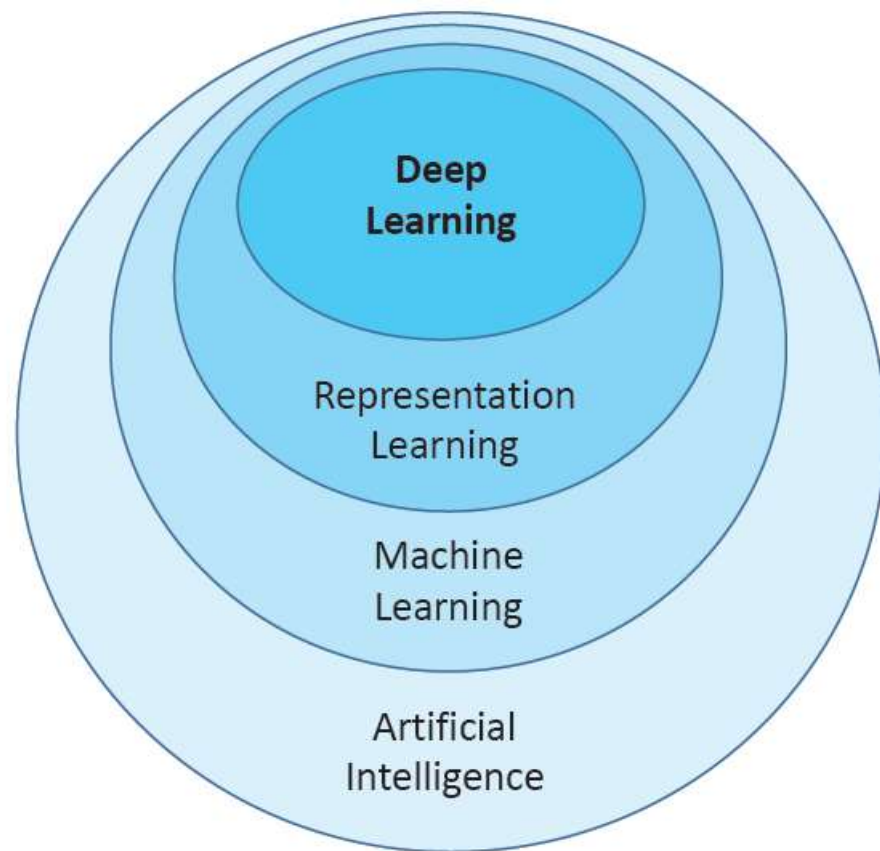
● Hidden Layer

● Output Layer

Universality: For any arbitrary function $f(x)$, there exists a neural network that closely approximate it for any input x

深度学习所属领域

- 深度学习是人工智能里面的一个部分
- $AI > ML > RL(\text{Feature Learning}) > DL$

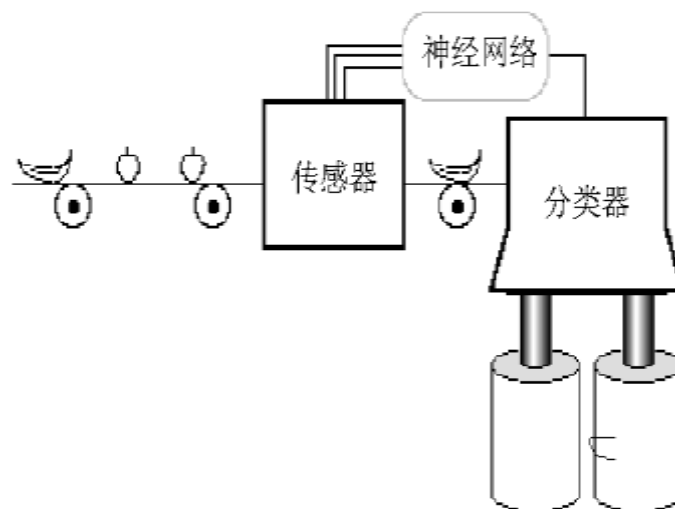


机器学习

- 专门研究计算机怎样模拟或实现人类的学习行为，以获取新的知识或技能，重新组织已有的知识结构使之不断改善自身的性能。

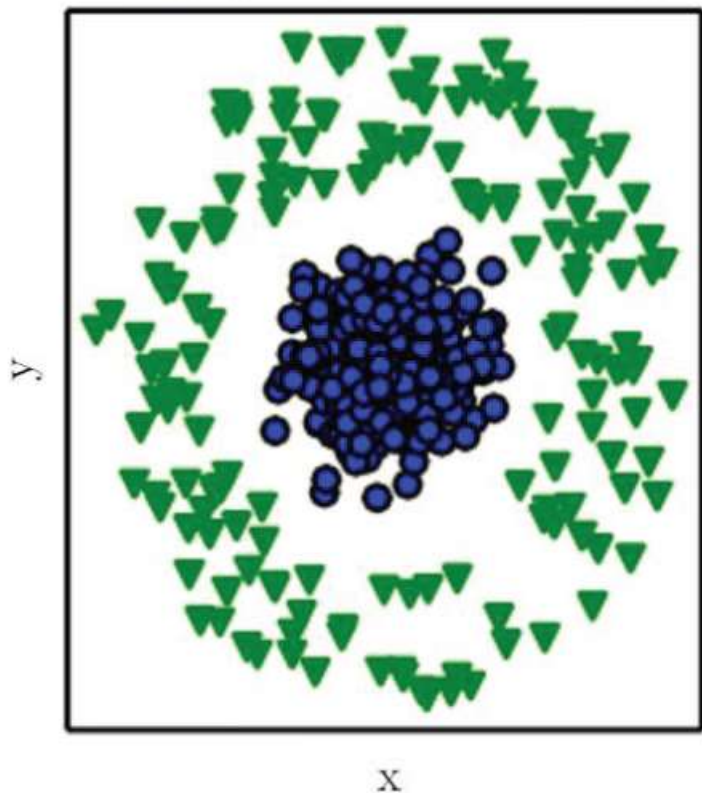


人类学习

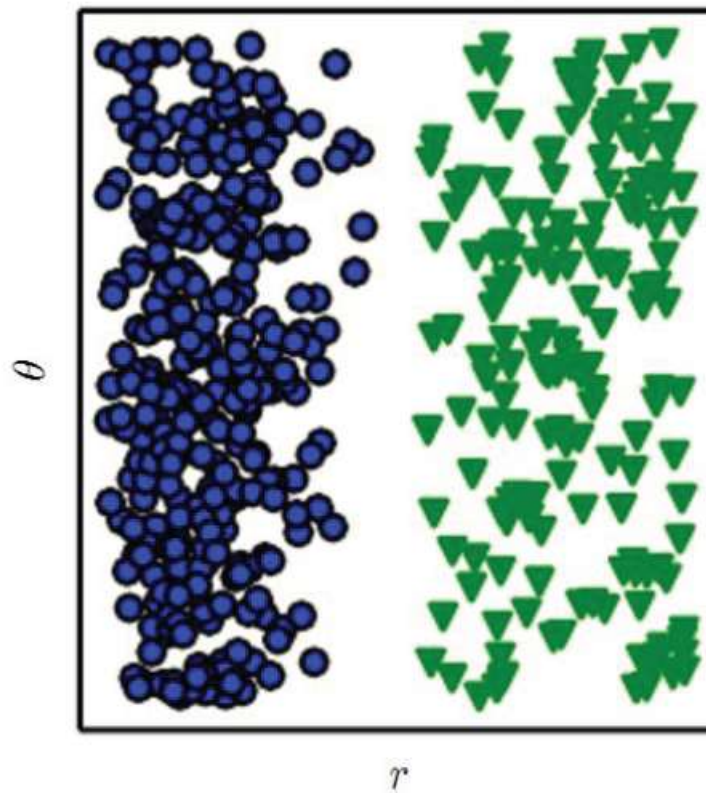


机器学习

特征学习



笛卡尔坐标

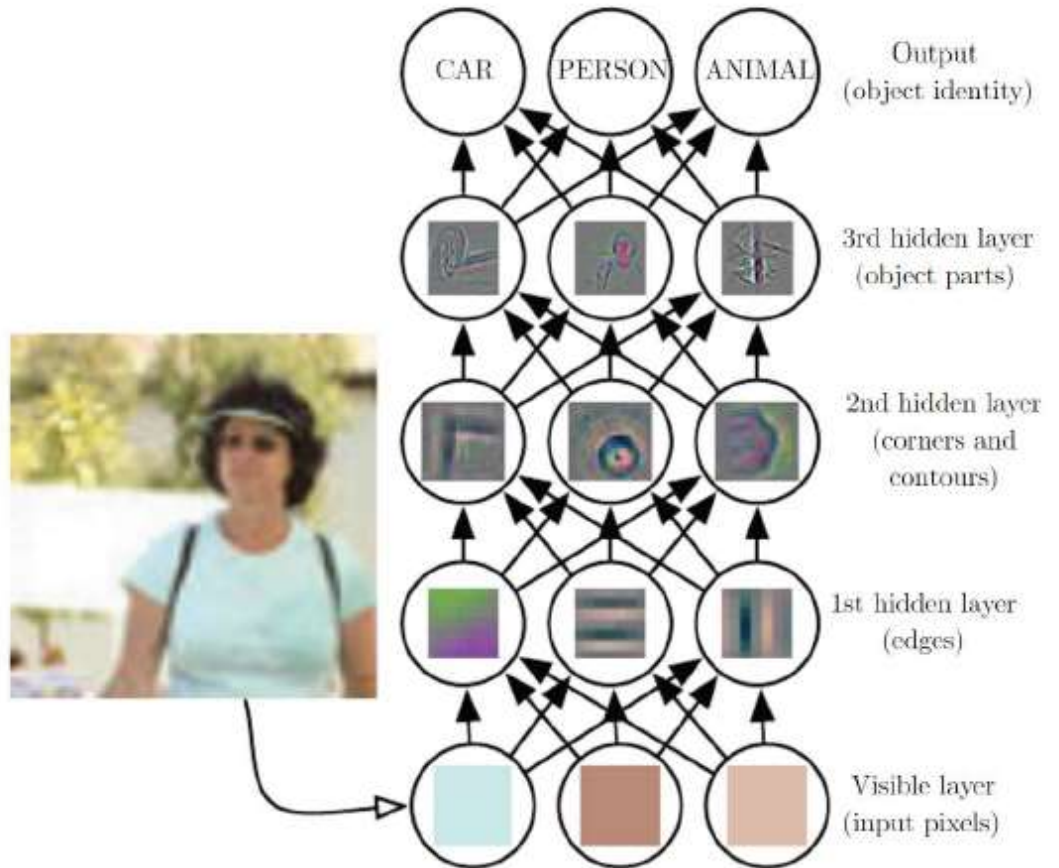
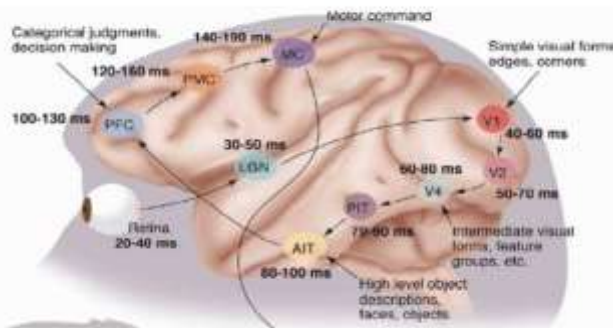


极坐标

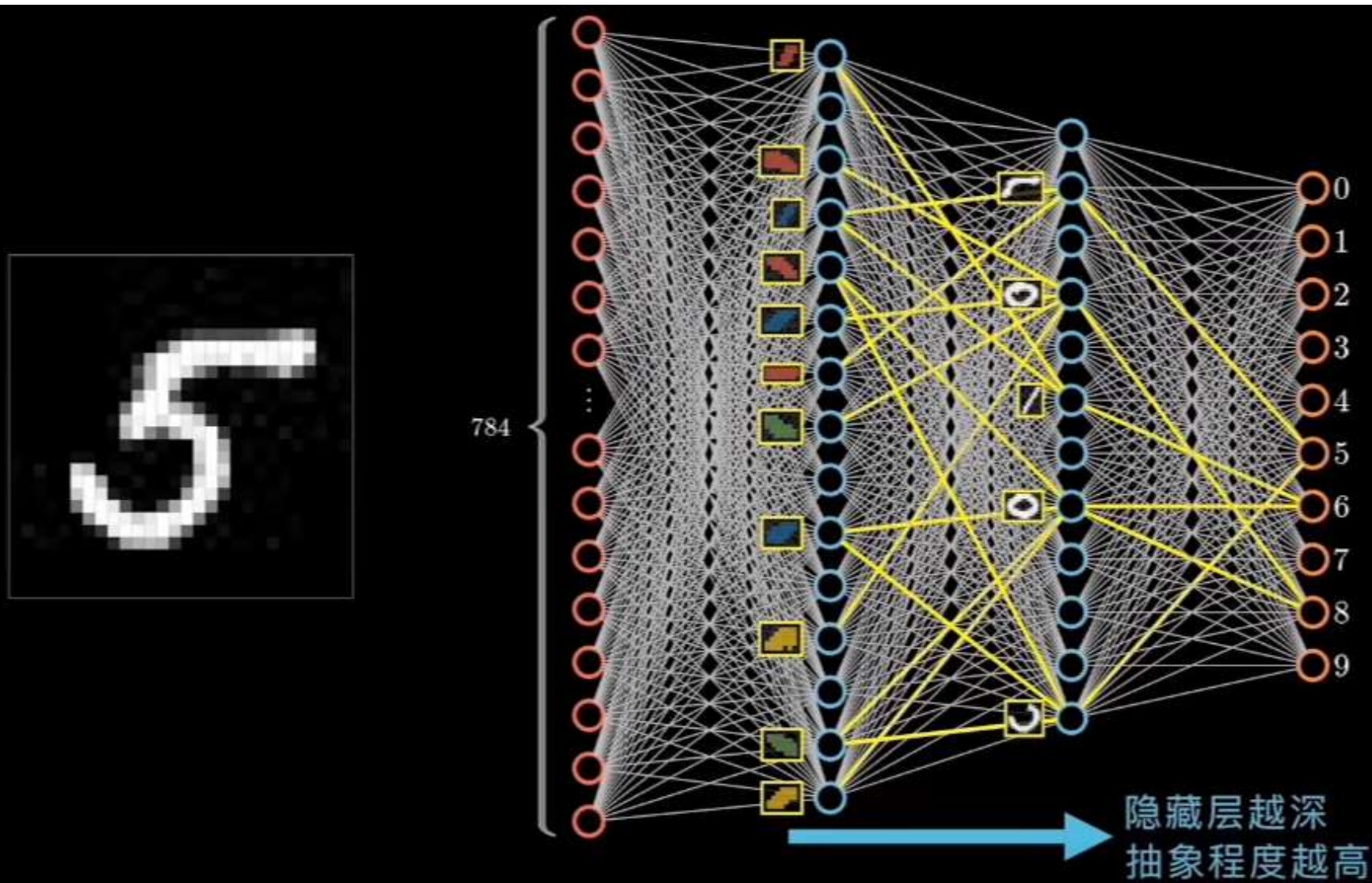
任务：画一条线将蓝点和绿点分开

深度学习

- 人类视觉分级系统
- 自动寻找特征



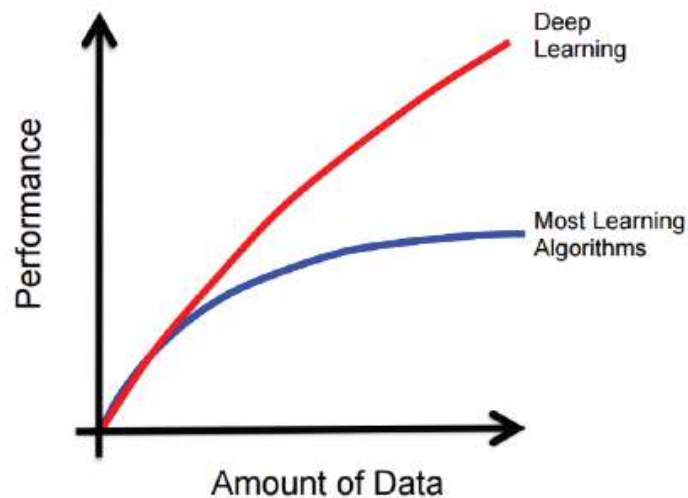
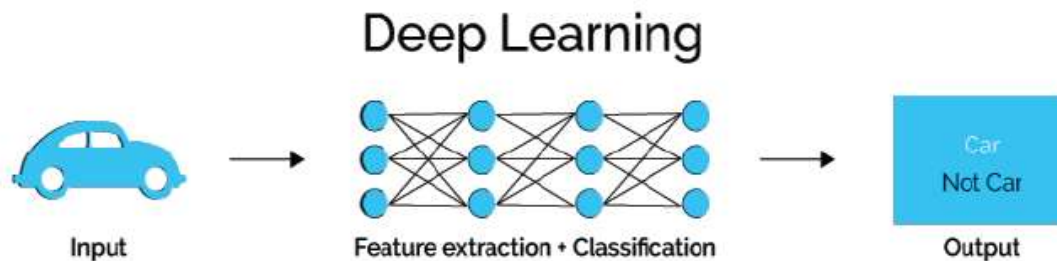
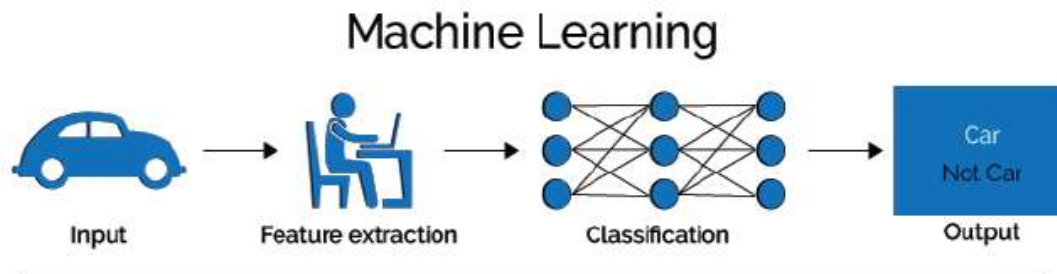
特征分层的作用



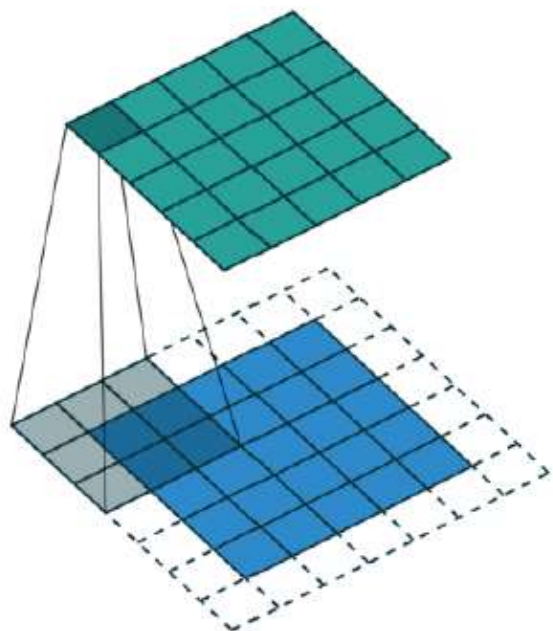
“6”下部底层特征复用了“8”的底层特征

不同层次特征有利于底层特征被复用

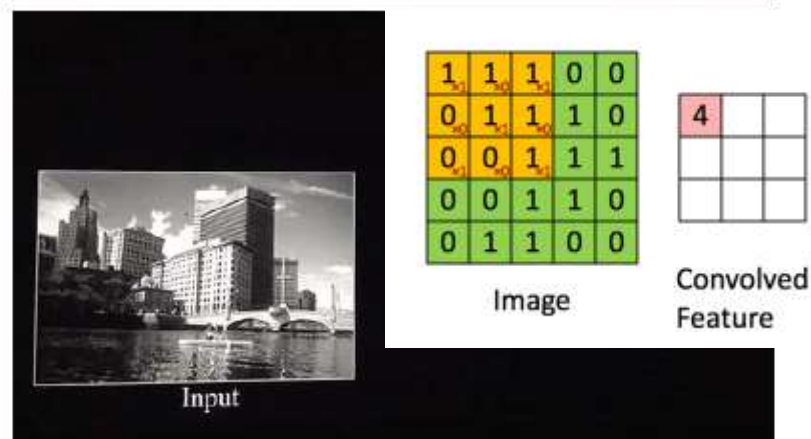
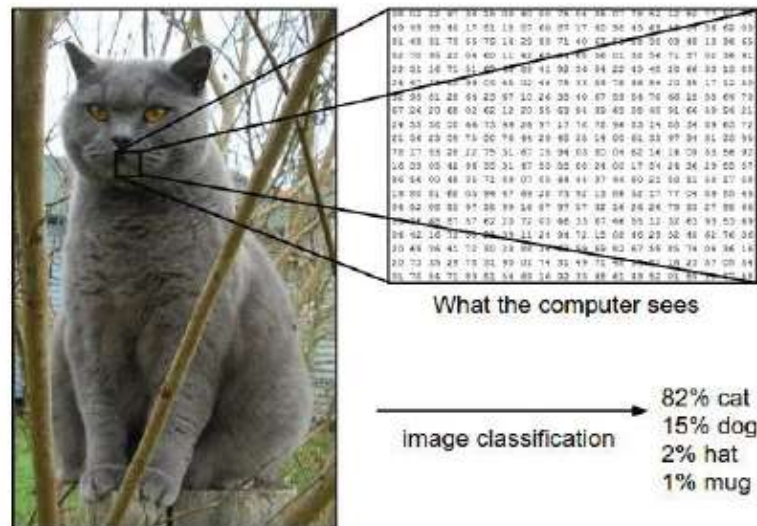
联系与区别



Convolutional Neural Networks: Image Classification



- Convolutional filters: take advantage of spatial invariance



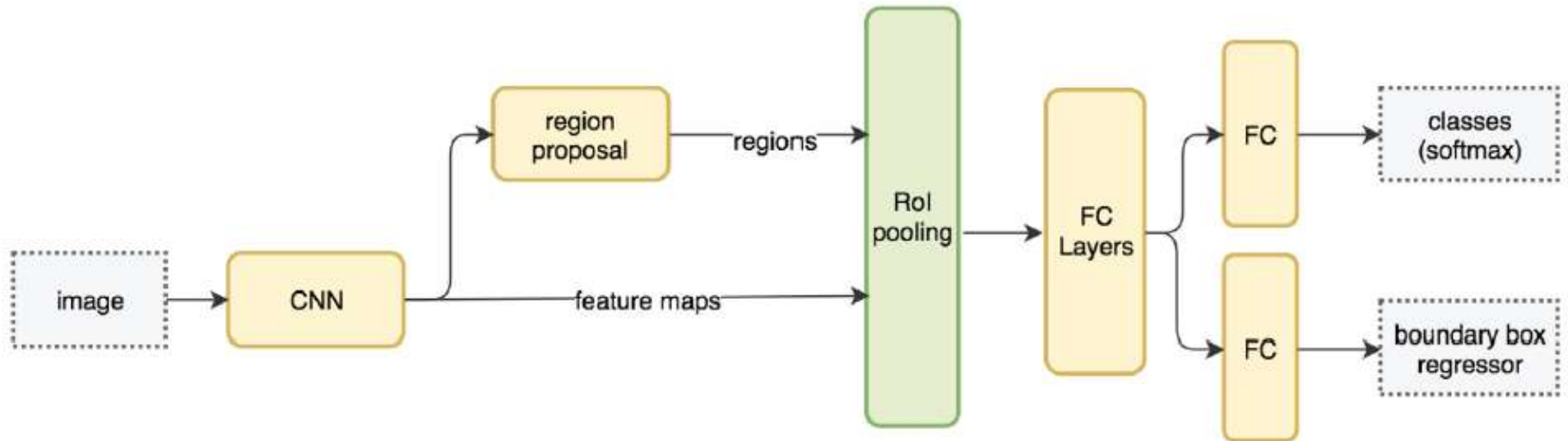


Human error (5.1%)
surpassed in 2015

- **AlexNet (2012): First CNN (15.4%)**
 - 8 layers
 - 61 million parameters
- **ZFNet (2013): 15.4% to 11.2%**
 - 8 layers
 - More filters. Denser stride.
- **VGGNet (2014): 11.2% to 7.3%**
 - Beautifully uniform: 3x3 conv, stride 1, pad 1, 2x2 max pool
 - 16 layers
 - 138 million parameters
- **GoogLeNet (2014): 11.2% to 6.7%**
 - Inception modules
 - 22 layers
 - 5 million parameters (throw away fully connected layers)
- **ResNet (2015): 6.7% to 3.57%**
 - More layers = better performance
 - 152 layers
- **CUIImage (2016): 3.57% to 2.99%**
 - Ensemble of 6 models
- **SENet (2017): 2.99% to 2.251%**
 - Squeeze and excitation block: network is allowed to adaptively adjust the weighting of each feature map in the convolutional block.

Object Detection / Localization

Region-Based Methods | Shown: Faster R-CNN

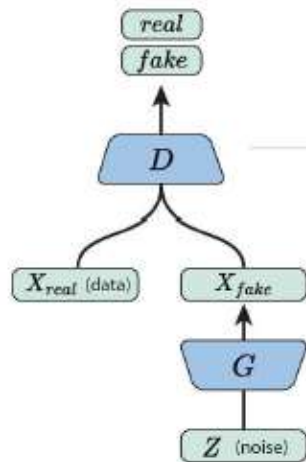


```
ROIs = region_proposal(image)
for ROI in ROIs:
    patch = get_patch(image, ROI)
    results = detector(patch)
```



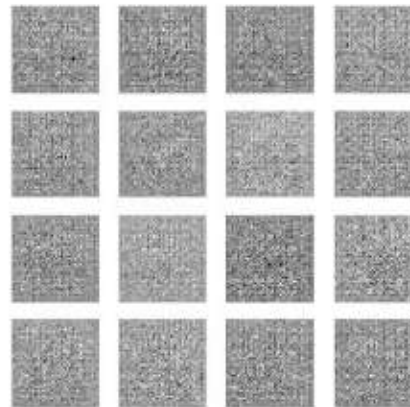
Generative Adversarial Network (GANs)

Generative Adversarial Networks (GANs) are a way to make a generative model by having two neural networks compete with each other.



The **discriminator** tries to distinguish genuine data from forgeries created by the generator.

The **generator** turns random noise into imitations of the data, in an attempt to fool the discriminator.



Progressive GAN
10/2017
1024 x 1024



第十节：应用

● 民用应用领域

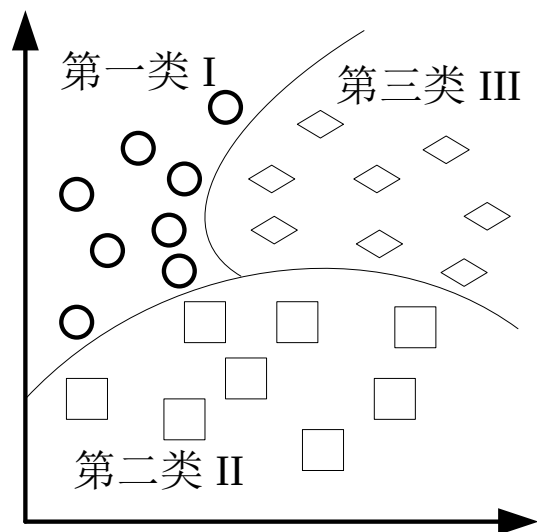
- 语言识别、图像识别与理解、计算机视觉、智能机器人故检测、实时语言翻译、企业管理、市场分析、决策优化、物资调运、自适应控制、专家系统、智能接口、神经生理学、心理学和认知科学研究等等

● 军用应用领域

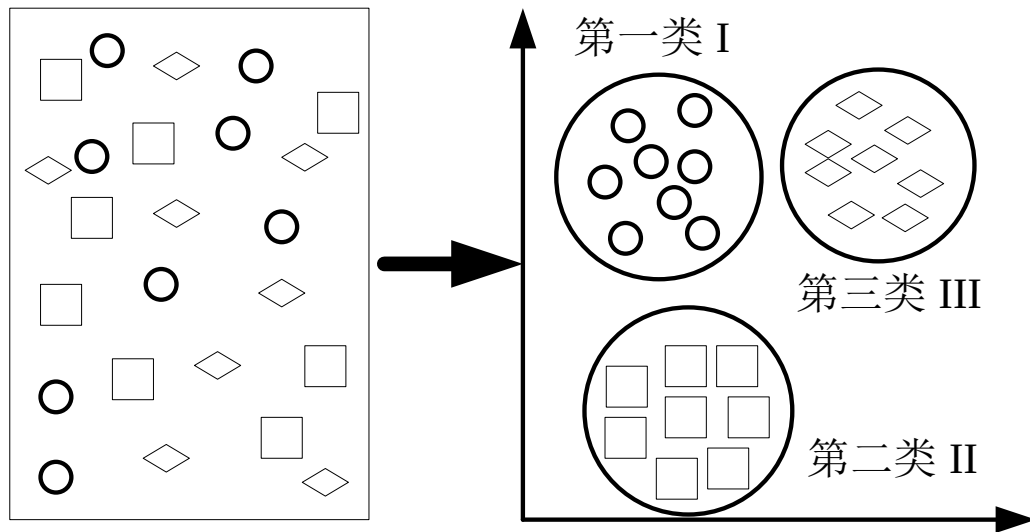
- 语音、图像信息的录取与处理、雷达、声纳的多目标识别与跟踪、战场管理和决策支持系统、军用机器人控制、各种情况、信息的快速录取、分类与查询、导弹的智能引导、保密通讯、航天器的姿态控制等等

● 识别与聚类应用

- 模式识别和分类是指通过使用一系列的训练数据对神经网络进行训练，在有教师指导的学习方法下，让神经网络在训练数据的训练下调整网络的结构和权重，以达到正确识别的目的



(a) 分类



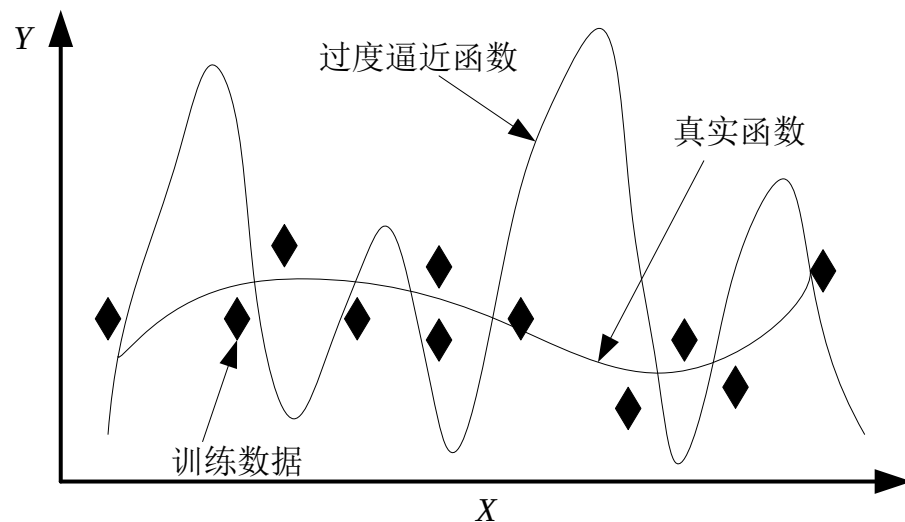
(b) 聚类

● 计算与优化应用

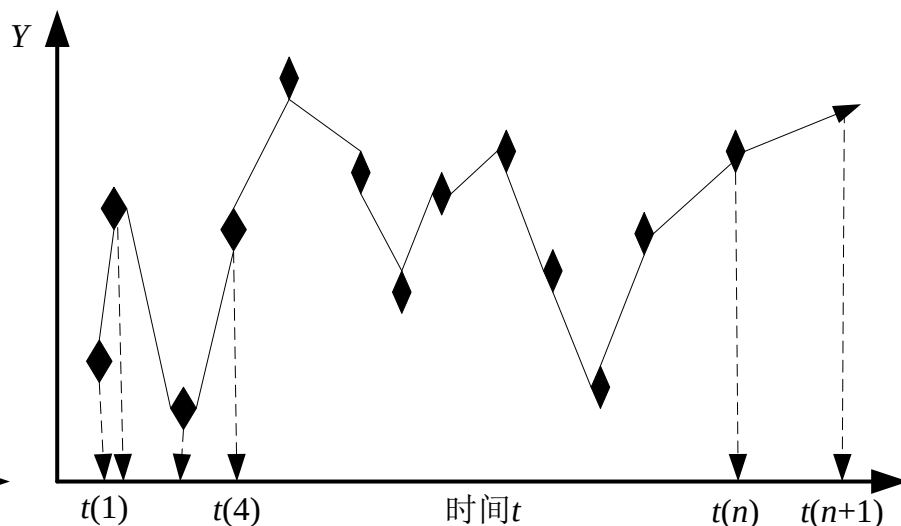
- 神经计算是人工神经网络的一个重要应用手段，也为各种优化问题提供了解决的方案。
- 优化问题就是需要在问题的解空间里面寻找一个最优的解在满足一定约束条件下使得目标函数最大化或者最小化。
- 由于神经网络具有并行搜索处理信息、联想记忆等特点，在搜索系统的全局最优或局部最优解方面，具有很好的优势，因此在优化问题上得到了广泛的研究和应用。

● 建模与预测应用

- 人工神经网络应用在非线性系统建模上的时候，实质上就是通过训练神经网络，让其在训练数据中获取知识，并且完成从输入到输出的非线性映射过程
- 所谓神经网络预测就是根据一定数量的历史样本数据（一般为表征某一种现象时间序列的数据，例如交通流量，外汇走势等等）对神经网络进行训练，然后用来对当前的，或者未来的时刻情景进行预测



(a) 建模（函数逼近）



(b) 预测

● 控制与处理应用

- 神经网络已经在自动控制和信息处理等各个领域取得了相当成功的应用。在自动控制领域，主要的应用包括系统建模和辨识、参数整定、极点配置、内模控制、优化设计、预测控制、最优控制、滤波与预测容错控制等
- 在各种信息、信号处理领域，神经网络也有重要的突破。例如在图像处理方面，神经网络能够很好地对图像进行边缘监测、图像分割、图像压缩和图像恢复等操作。此外，神经网络被广泛用于信号处理方面，例如目标检测、多目标跟踪、杂波去噪、畸形波恢复、雷达回波多目标分类、运动目标速度估计、多探测信号融合等等

人脸识别

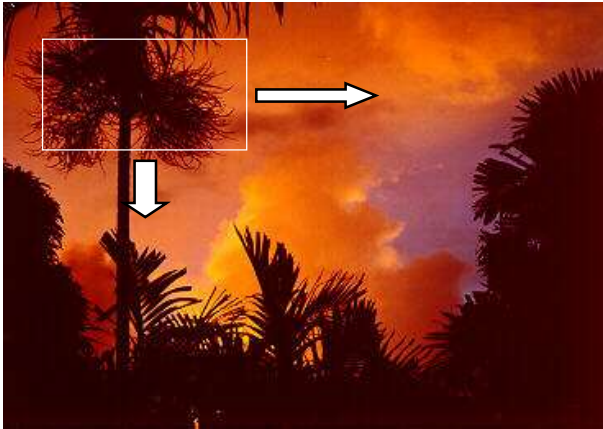
A view-based system: 15 views



Performance: 98% on 68 person database

Beymer, 1995

目标分类



Scanning in x,y and scale

Preprocessing with
overcomplete
dictionary of Haar
wavelets

SVM Classifier

TRAINING

Data Base

QP Solver

手写数字识别

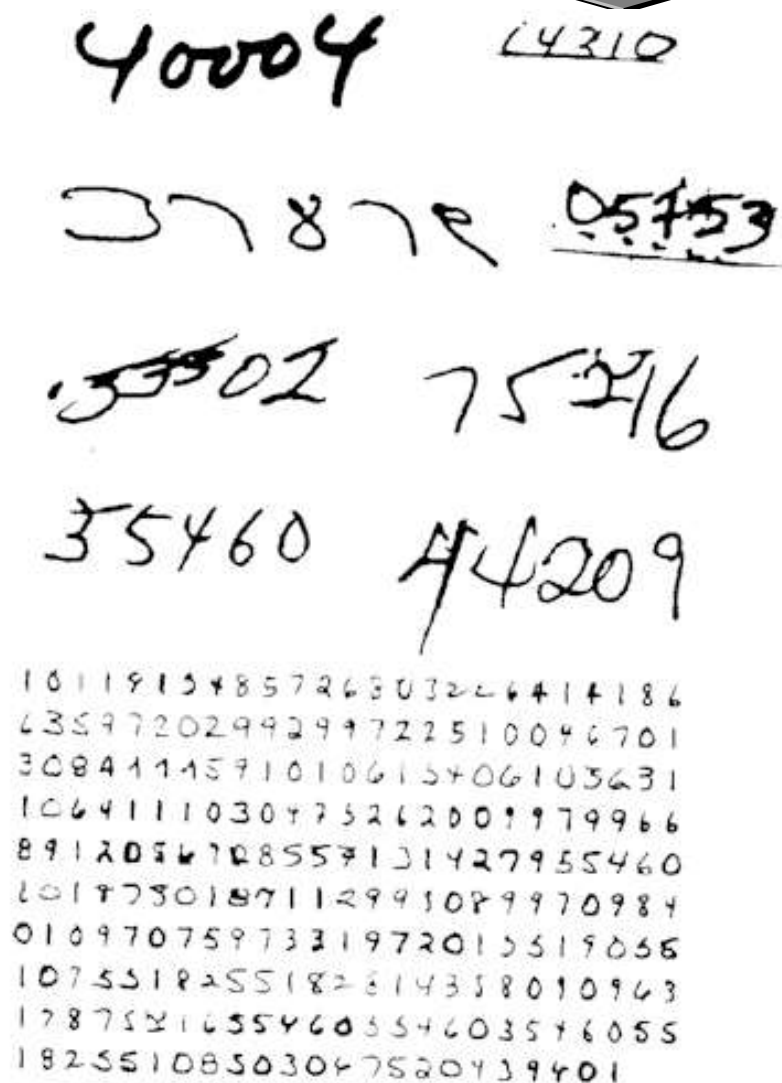
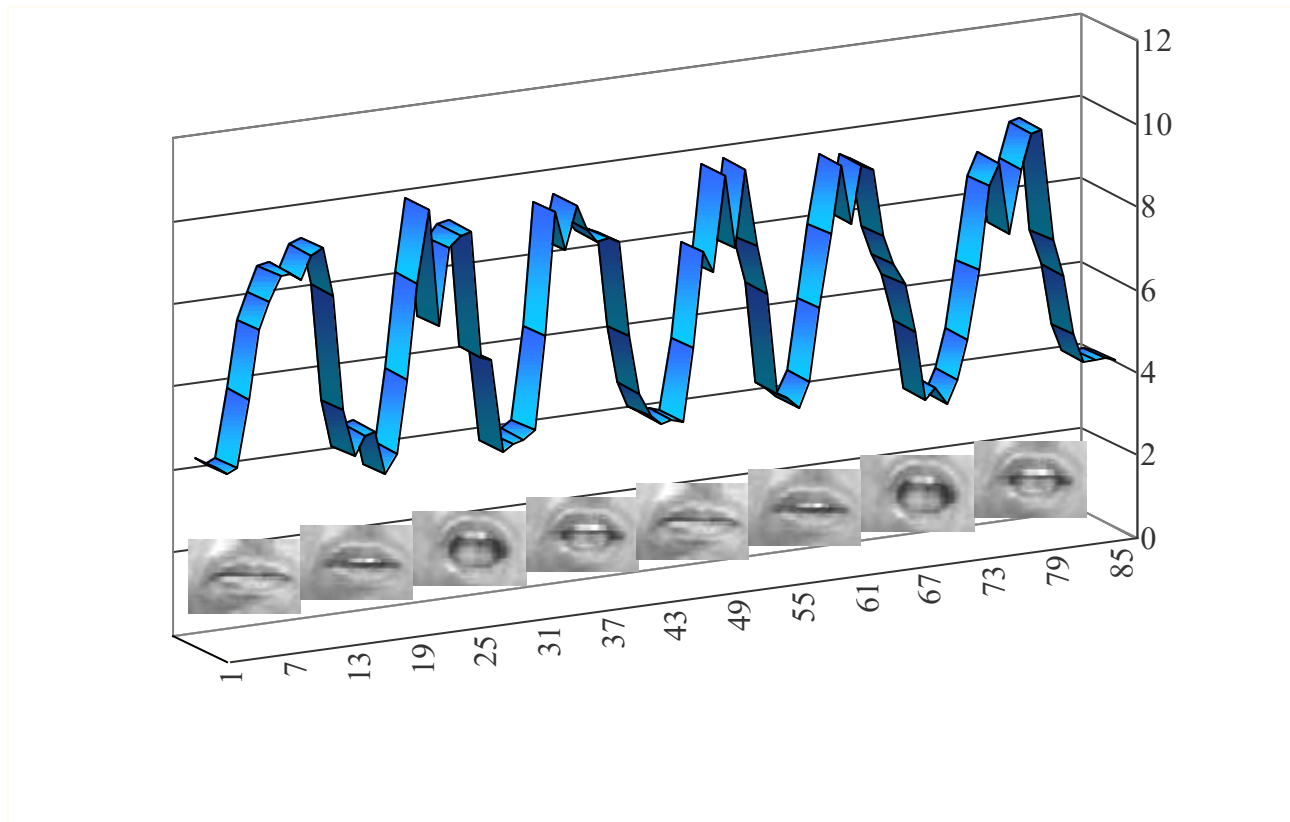


FIGURE 10.8

Examples of ZIP code image, and segmented and normalized numerals from the testing set. (Source: Reprinted with permission from Y. Le Cun, et al., "Backpropagation Applied to Handwritten Zip Code Recognition," *Neural Computation*, 1:541-551, 1989. ©1989 The MIT Press.)

图像分析：估计表情参数



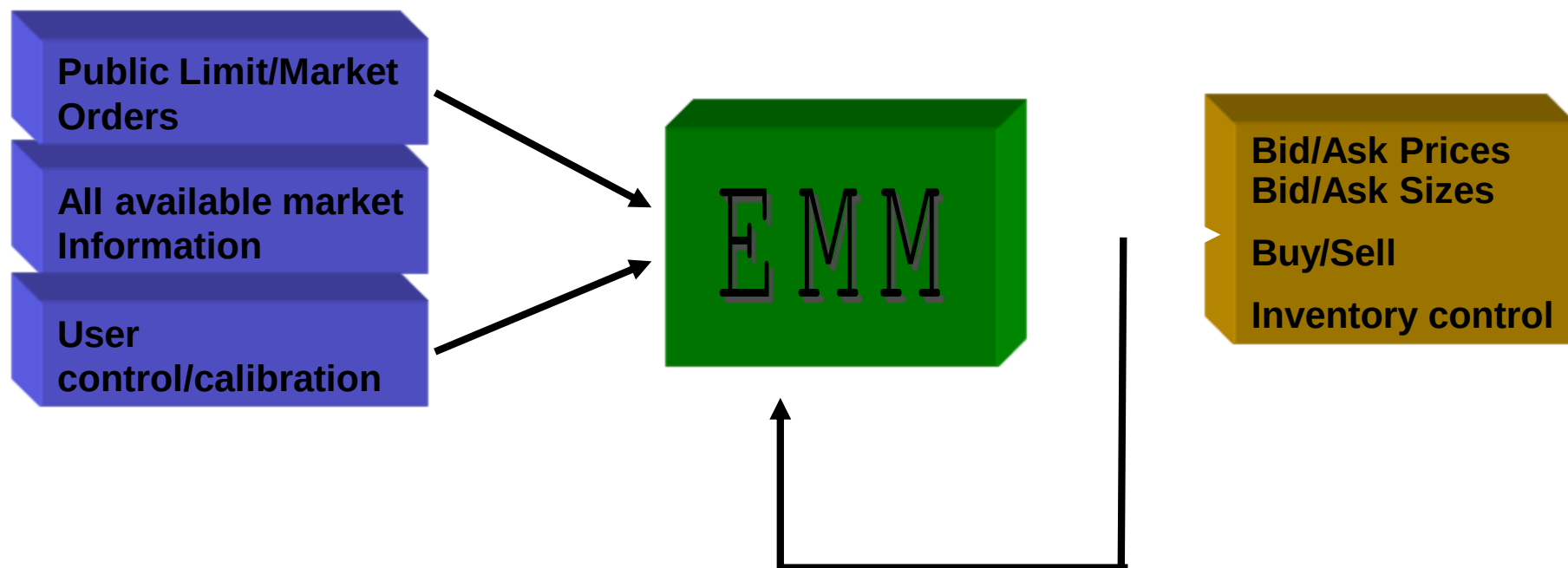
The main goal is to estimate basic facial parameters, e.g. degree of mouth openness, through learning

完整系统

Face Detection → Localization of Facial Features → Analysis of Facial parts



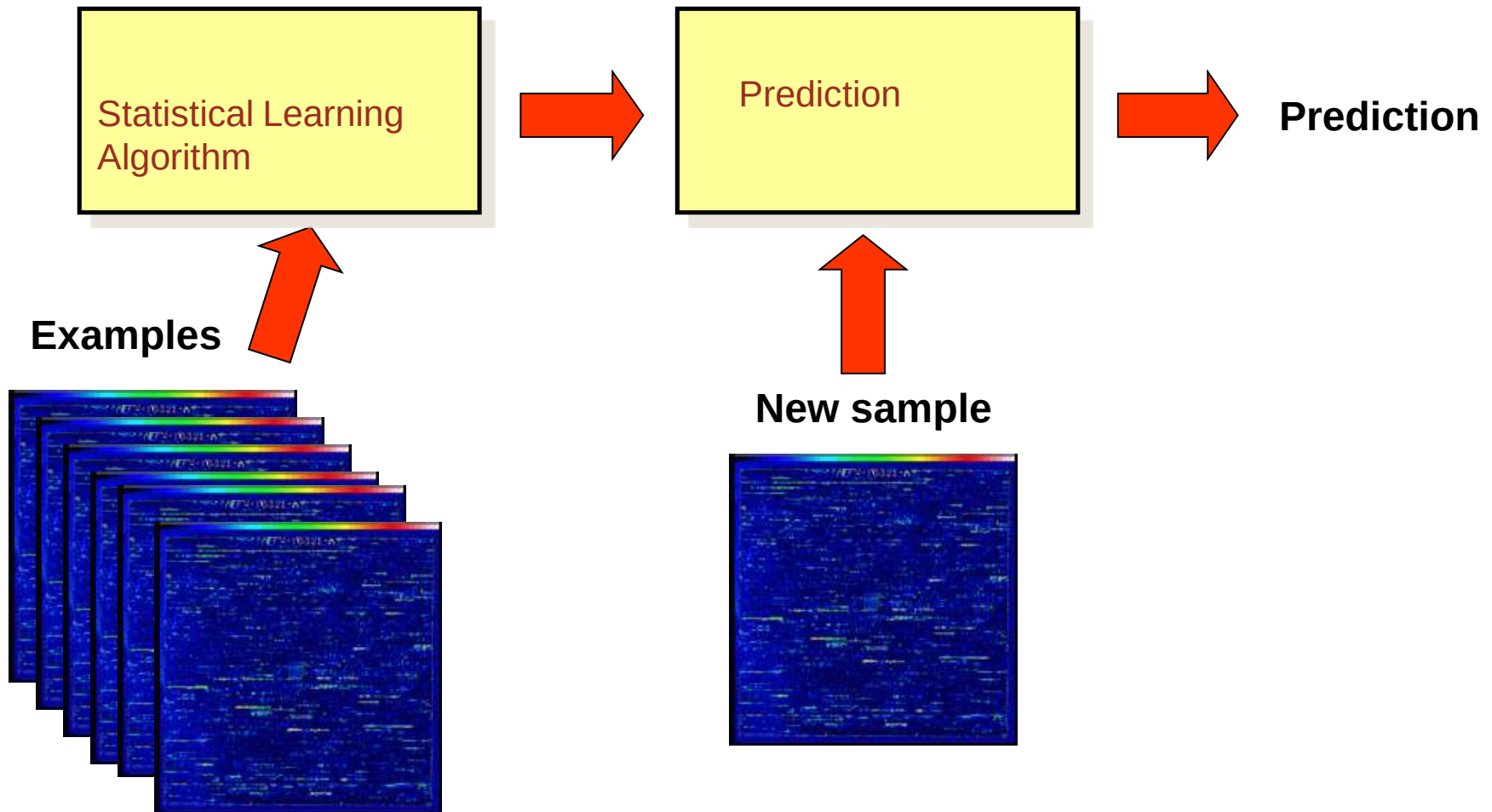
金融预测



学习反馈

生物信息学：从DNA片段信号中预测癌症

Learning from examples paradigm



如何实现人工神经网络

■ 硬件实现

高速的实时处理系统，比如汽车引擎控制，高速的实时识别系统，如人脸识别、指纹识别等，要求使用专用的硬件系统。

2000年，**Fan Yang**在嵌入式系统上数字实现了一个径向基函数(**RBF**)神经网络,能够实时的进行人脸跟踪和身份识别。近几年来科技发达国家的许多公司对神经网络芯片、生物芯片独有情钟。例如**Intel**公司、**IBM**公司等已取得了多项专利，已有产品进入市场，被国防、企业和科研部门采用。

如何实现人工神经网络

■ 软件模拟

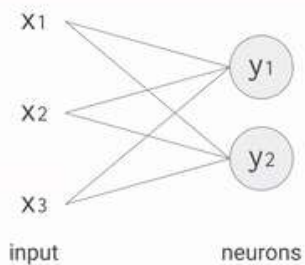
对于各种网络模型，**matlab**神经网络工具箱集成了多种学习算法，为用户提供了极大的方便。最关键的是，丰富的函数可以节省的大量的编程时间。此外，神经网络工具箱中还给出了大量的示例程序，为用户轻松的使用工具箱提供生动的实用范例。

■ 光学实现

早在**1995**年，**Jenkins**等人就研究了光学神经网络，建立了光学二维并行互连与电子学混合的光学神经网络系统，实现了光学神经元。

可并行化计算

Step 1

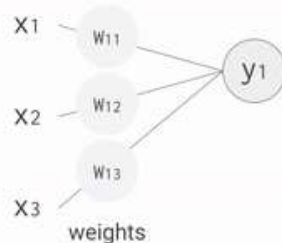


Step 4

$$y_1 = f(w_{11}x_1 + w_{12}x_2 + w_{13}x_3)$$

↑
activation function

Step 2



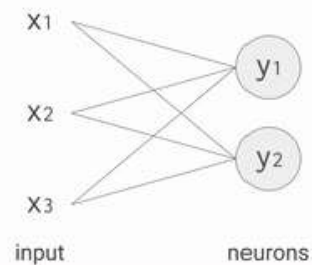
Step 5

$$y_1 = f(w_{11}x_1 + w_{12}x_2 + w_{13}x_3)$$
$$y_2 = f(w_{21}x_1 + w_{22}x_2 + w_{23}x_3)$$

Step 3

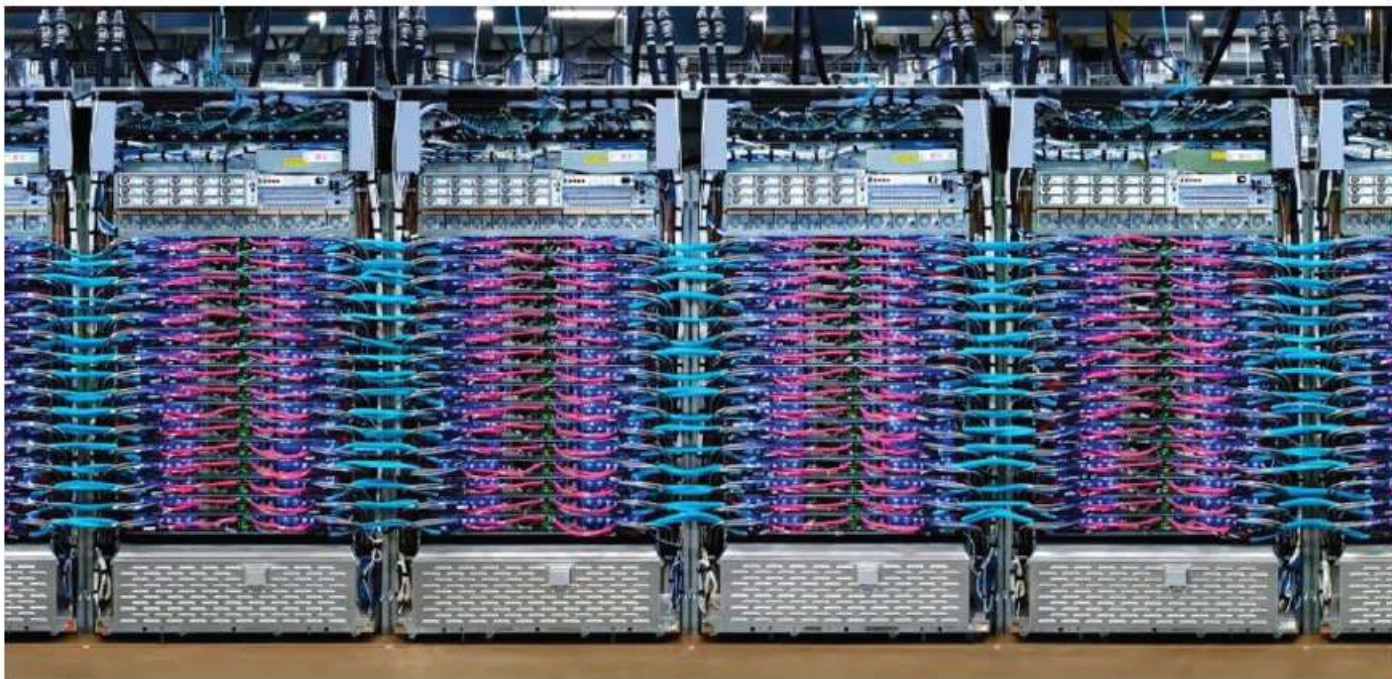
$$y_1 = w_{11}x_1 + w_{12}x_2 + w_{13}x_3$$

Animated



计算硬件

- **CPU** – serial, general purpose, everyone has one
- **GPU** – parallelizable, still general purpose
- **TPU** – custom ASIC (Application-Specific Integrated Circuit) by Google, specialized for machine learning, low precision





最高达
**20% 图形处理器
速度提升**
新增 8 个核心

最高达
**10% 中央处理器
速度提升**
配备 6 个核心

最高达
4 倍光线追踪速度
相比 A16 仿生芯片

最高达
**2 倍神经网络
引擎速度**
16 个核心运算能力
推出每秒 36 万亿次

A17 Pro 刘海屏处理器
比 iPhone 12 Pro 中的
处理器高达 70%



- 四重反射棱镜，通过镜头后方的折叠玻璃结构，对光线进行四次反射，让光线在空间不变的情况下经过的光路更长，从而带来了更长的新焦距。
- 加入 **3D 传感器** 位移式光学图像防抖和自动对焦模块，能在全部三个方向上进行位移来实现防抖，精细微调的频率可达以往两倍，每秒**10000** 次。

智能 HDR 4 对场景中的每个部分 进行优化。

借助神经网络引擎强大的机器学习能力, 智能 HDR 4 功能现在可对同一场景中多个人物分别进行调整。我们的软件和图像信号处理器 (ISP) 会自动为每个人优化对比度、光线效果和肤色, 让所有人看起来都光彩照人。



深度融合技术 对多重曝光画面中的 每个像素进行分析。

在中低光照条件下拍照时, 深度融合技术会大显身手, 运用神经网络引擎逐个像素地分析不同曝光度的画面, 再选取各个画面的最佳部分融合成最终的图像。这样拍出来的照片不仅拥有惊人的细节, 就连微妙的纹理质感也能细腻呈现。

